

UNIVERSITÀ DEGLI STUDI DI ROMA “TOR VERGATA”

Macroarea di Ingegneria
Corso di Laurea Magistrale in Ingegneria Informatica

Prodotto matrice densa per multivettore in MPI e CUDA

Progetto di fine corso

Sistemi di Calcolo Parallelo e Applicazioni
A.A. 2025/2026

Autori

Alessandro Vassallo
0366572

Andrea Galluzzi
0365026

Codice sorgente: <https://github.com/AleVassa00/parallel-computing-project>

Versione della relazione del 16 settembre 2026

Bozza di lavoro. I marcatori in rosso segnalano i punti che dipendono da campagne di misura non ancora concluse. Per la versione di consegna, commentare `\todovisibiletrue` in `preambolo.tex`.

Indice

1	Introduzione	1
1.1	Obiettivo	1
1.2	Struttura della relazione	2
2	Il nucleo di calcolo	3
2.1	Che cosa calcola il nucleo	3
2.1.1	Conteggio delle operazioni	3
2.2	Come vengono memorizzati i dati	4
2.3	Ordine dei cicli e riutilizzo dei dati	5
2.3.1	I due schemi candidati	6
2.3.2	Intensità rispetto al traffico di A	6
2.3.3	Il limite rispetto al traffico di A	7
2.3.4	Località degli accessi	7
2.4	Interfaccia comune dei backend locali	8
2.4.1	Ciclo di vita del backend	8
2.4.2	Contesto opaco del backend	9
2.4.3	Informazioni sperimentali	9
2.4.4	Selezione del backend	9
3	Parallelizzazione MPI	10
3.1	Che cosa significa parallelizzare in MPI	10
3.2	Perché si taglia A , e perché in due direzioni	11
3.2.1	Che cosa tagliare	11
3.2.2	Come tagliare: due possibilità monodimensionali	11
3.2.3	Il taglio bidimensionale	11
3.2.4	Perché i processi vengono organizzati a griglia	12
3.3	Decomposizione a griglia bidimensionale	12
3.3.1	I due sotto-comunicatori	13
3.3.2	Chi fa da root, e perché la scelta è verificata	13
3.4	Partizionamento degli indici	14
3.4.1	Le funzioni	14
3.5	Layout dei blocchi locali	14
3.5.1	Perché i conteggi delle collettive tornano	14
3.5.2	Chi contiene che cosa, e quando	15
3.5.3	Perché il multivettore non viene mai tagliato lungo k	15
3.6	Distribuzione iniziale dei dati	15
3.7	L'algoritmo distribuito	16
3.8	Modello di costo della comunicazione	17
4	Il nucleo di CPU	19

4.1	Tre backend, una sola idea	19
4.2	Il backend di riferimento	19
4.2.1	Perché specializzare k a compile-time	19
4.2.2	Il fallback generico, e perché la traccia lo richiede	20
4.3	Perché $KB = 32$	21
4.4	Ciò che il compilatore vede	21
4.5	Il primo limite: la rilettura di X	21
4.6	Secondo backend: X mattonellata in cache	22
4.7	Terzo backend: register blocking sulle righe	22
5	I backend CUDA	24
5.1	Struttura comune	25
5.2	Misurazione del backend CUDA	26
5.3	Preprocessing e occupazione della GPU	27
5.3.1	Gestione della memoria disponibile	27
5.4	<code>cuda_naive</code>	27
5.5	<code>cuda_warp</code> : un warp per riga	29
5.5.1	Gestione di k	30
5.5.2	La maschera dello shuffle	30
5.5.3	Il problema residuo: gli accessi a X	30
5.6	Primo rimedio: cambio di layout di X	31
5.7	<code>cuda_warp_smem</code> : staging di X in shared memory	31
5.7.1	Caricamento cooperativo di X	32
5.7.2	Riutilizzo del tile fra i warp del blocco	32
5.7.3	Bank conflict e padding della shared memory	33
5.7.4	Scelta della dimensione del tile	34
5.7.5	Sincronizzazione e warp inattivi	35
5.8	Due rimedi allo stesso problema	35
5.9	<code>cublas</code> : il riferimento esterno	36
5.9.1	Adattamento fra row-major e column-major	36
5.9.2	Significato del confronto	37
5.10	Più rank MPI sulla stessa GPU	37
6	Metodologia sperimentale	39
6.1	Ambiente sperimentale	39
6.2	Protocollo di misura	40
6.2.1	Temporizzazione della fase locale su GPU	41
6.3	Metriche di prestazione	41
6.3.1	Prestazione della sola fase computazionale	42
6.4	Ripetizioni e raccolta dei campioni	43
6.5	Disegno sperimentale	43
6.5.1	Criterio di selezione dei parametri	44
6.6	Fase 1 - Configurazione dei backend CUDA	45
6.6.1	Campagna 1.1 - Dimensione del blocco e confronto dei backend	45
6.6.2	Campagna 1.2 - Layout in memoria di X	46
6.6.3	Campagna 1.3 - Granularità del tile	47
6.6.4	Campagna 1.4 - Padding del tile in shared memory	48
6.7	Fase 2 - Caratterizzazione dimensionale	48
6.8	Fase 3 - Effetto della precisione numerica	50
6.9	Fase 4 - Esecuzione distribuita MPI + CUDA	51
7	Risultati	53

7.1	Fase 1 - Configurazione dei backend CUDA	53
7.1.1	Campagna 1.1 - Dimensione del blocco	53
7.1.2	Campagna 1.2 - Layout in memoria di X	55
7.1.3	Campagna 1.3 - Granularità del tile	57
7.1.4	Campagna 1.4 - Padding del tile in shared memory	59
7.1.5	Esito della Fase 1 - Configurazioni consolidate	60
7.2	Fase 2 - Caratterizzazione dimensionale	61
7.2.1	Disegno della campagna	61
7.2.2	Effetto della scala	62
7.2.3	Effetto della forma	64
7.2.4	Sintesi della Fase 2	68
7.3	Fase 3 - Effetto della precisione numerica	68
7.3.1	Confronto tra FP64 e FP32	69
7.3.2	Comportamento dei kernel custom	69
7.3.3	Piano di esecuzione di <code>cuda_warp_smem</code>	70
7.3.4	Comportamento di cuBLAS	71
7.3.5	Sintesi	72
7.4	Fase 4 - Esecuzione distribuita MPI + CUDA	72
7.4.1	Il tempo collettivo come metrica	72
7.4.2	Effetto del numero di rank P	73
7.4.3	Effetto della griglia MPI	75
7.4.4	Confronto fra backend nella composizione MPI + CUDA	77
7.4.5	Sintesi della Fase 4	78
7.5	Il costo dei dati: trasferimenti e preprocessing	79
7.6	Sintesi	80

Capitolo 1

Introduzione

1.1 Obiettivo

Il progetto realizza un nucleo di calcolo parallelo per il prodotto tra una matrice densa e un multivettore:

$$Y \leftarrow AX, \quad A \in \mathbb{R}^{M \times N}, \quad X \in \mathbb{R}^{N \times k}, \quad Y \in \mathbb{R}^{M \times k}. \quad (1.1)$$

Il multivettore X può essere visto come una matrice densa con un numero k di colonne molto più piccolo rispetto alle dimensioni della matrice A . Il codice supporta valori generici di k ed è stato collaudato sui cinque valori richiesti dalla traccia,

$$k \in \{3, 6, 8, 20, 32\}.$$

Le dimensioni globali M e N non sono fissate in fase di compilazione, ma vengono scelte a runtime. Il nucleo deve quindi adattarsi sia alla forma della matrice sia alla dimensione del multivettore.

La parallelizzazione è organizzata su due livelli. Il livello esterno è basato su MPI e distribuisce il problema fra più processi. La matrice A viene suddivisa su una griglia bidimensionale di processi, e le comunicazioni MPI rendono disponibili i dati necessari al calcolo dei contributi locali e alla successiva costruzione del risultato globale.

All'interno di ciascun processo, il prodotto sul blocco locale è demandato a un backend di calcolo attraverso un'interfaccia comune. Il progetto comprende implementazioni direttamente su CPU e diversi backend CUDA, questi ultimi utilizzati per sfruttare il parallelismo della GPU NVIDIA.

I due livelli operano quindi in modo gerarchico: MPI determina quali dati e quale parte del prodotto competono a ciascun processo, mentre il backend locale determina come quel prodotto viene eseguito all'interno del singolo processo.

Oltre alla correttezza numerica, il progetto ha come obiettivo il confronto prestazionale delle diverse strategie implementate. Le successive ottimizzazioni vengono perciò introdotte e misurate separatamente, così da poter collegare le differenze di prestazione alle specifiche scelte di distribuzione, accesso alla memoria e parallelizzazione.

Il lavoro non si limita alla realizzazione funzionale del prodotto. Le diverse implementazioni vengono confrontate sia dal punto di vista della correttezza numerica sia da quello delle

prestazioni. La correttezza viene verificata rispetto a un riferimento seriale, mentre le prestazioni vengono studiate al variare delle dimensioni della matrice, del numero di colonne del multivettore e della configurazione parallela.

La separazione tra livello MPI e backend locale permette di mantenere invariata la decomposizione distribuita del problema e di sostituire indipendentemente il nucleo di calcolo eseguito da ciascun processo. In questo modo le diverse implementazioni locali possono essere confrontate all'interno della stessa pipeline MPI.

La presenza di un k piccolo rispetto a M e N distingue inoltre il problema da una GEMM generale: ciascun elemento di A viene utilizzato per contribuire a più colonne del multivettore, rendendo il riutilizzo dei dati e il modo in cui questi vengono disposti in memoria aspetti centrali dell'ottimizzazione.

1.2 Struttura della relazione

La relazione introduce dapprima la decomposizione del prodotto e l'organizzazione dei dati, quindi descrive il livello distribuito MPI. I capitoli successivi analizzano separatamente i backend locali su CPU e GPU, mettendo in relazione le scelte implementative con il comportamento della gerarchia di memoria e con il parallelismo disponibile. Infine vengono presentate la metodologia sperimentale e l'analisi delle prestazioni ottenute.

Capitolo 2

Il nucleo di calcolo

1

Questo capitolo fissa tre cose che il resto della relazione dà per acquisite: che cosa il nucleo calcola e come se ne contano le operazioni, in che ordine conviene scandire i tre cicli, e da dove vengono i dati. Sono scelte fatte una volta sola, prima di qualunque backend: i capitoli su MPI, CPU e CUDA le ereditano senza rimetterle in discussione.

2.1 Che cosa calcola il nucleo

Il nucleo realizza il prodotto fra una matrice densa A e un multivettore X :

$$Y \leftarrow AX, \quad A \in \mathbb{R}^{M \times N}, \quad X \in \mathbb{R}^{N \times k}, \quad Y \in \mathbb{R}^{M \times k}. \quad (2.1)$$

Elemento per elemento, il risultato è definito da:

$$Y[i][c] = \sum_{j=0}^{N-1} A[i][j] X[j][c], \quad 0 \leq i < M, \quad 0 \leq c < k. \quad (2.2)$$

Le dimensioni M e N vengono scelte a runtime. Anche k è un parametro del problema e il codice deve supportarne valori generici. Come richiesto dalla traccia, il collaudo comprende nel particolare:

$$k \in \{3, 6, 8, 20, 32\}.$$

La caratteristica distintiva del problema è che k è piccolo rispetto alle dimensioni della matrice A . Il prodotto può quindi essere visto come l'applicazione della stessa matrice a un numero limitato di vettori, raccolti nelle colonne di X .

2.1.1 Conteggio delle operazioni

Per ogni elemento $Y[i][c]$ devono essere combinati N termini del tipo:

$$A[i][j] X[j][c].$$

Adottando la convenzione utilizzata dalla traccia, ogni termine viene conteggiato come due operazioni floating point, una moltiplicazione e un'addizione. Il lavoro associato a un elemento di Y è quindi pari a $2N$ FLOP.

Poiché Y contiene Mk elementi, il numero complessivo di operazioni associato al prodotto globale è:

$$\text{FLOP}(M, N, k) = 2MNk. \quad (2.3)$$

Se T indica il tempo medio necessario per una singola invocazione del nucleo, la prestazione viene espressa come:

$$\text{GFLOPS} = \frac{2MNk}{T \cdot 10^9}. \quad (2.4)$$

Le dimensioni M e N presenti nel numeratore sono sempre quelle della matrice globale, anche quando il prodotto viene distribuito fra più processi MPI: quindi il numero di operazioni associato al problema rimane invariato al variare della configurazione parallela.

La traccia richiede che T sia ottenuto ripetendo il nucleo più volte sulla stessa matrice e considerando il tempo medio per invocazione. Preprocessing, operazioni di I/O e trasferimenti dei dati da e verso la GPU non contribuiscono alla misura prestazionale ufficiale, anche se il progetto li misura separatamente per caratterizzarne il costo.

Nel caso distribuito, la definizione precisa delle fasi comprese nel tempo del prodotto e il modo in cui i tempi dei diversi rank vengono combinati sono descritti in sezione 6.3.

2.2 Come vengono memorizzati i dati

Dopo la distribuzione, ciascun processo opera su tre buffer locali:

$$A_{\text{loc}} \in \mathbb{R}^{m_{\text{loc}} \times n_{\text{loc}}}, \quad X_{\text{loc}} \in \mathbb{R}^{n_{\text{loc}} \times k}, \quad Y_{\text{loc}} \in \mathbb{R}^{m_{\text{loc}} \times k}.$$

La rappresentazione di riferimento utilizzata dal progetto è **row-major**: gli elementi appartenenti alla stessa riga sono quindi memorizzati in posizioni consecutive.

L'interfaccia dei backend associa a ciascuna matrice una *leading dimension*, indicata per ciascuna matrice rispettivamente con `lda`, `ldx` e `ldy`. La leading dimension esprime, in numero di elementi, la distanza fra l'inizio di due righe consecutive del buffer. Gli elementi vengono quindi indirizzati come

$$A[i][j] = A_{\text{loc}}[i \cdot \text{lda} + j],$$

$$X[j][c] = X_{\text{loc}}[j \cdot \text{ldx} + c], \quad Y[i][c] = Y_{\text{loc}}[i \cdot \text{ldy} + c].$$

Una leading dimension uguale al numero di colonne corrisponde a una *rappresentazione compatta*. Se invece è maggiore, fra la parte utile di due righe consecutive rimangono alcuni elementi inutilizzati, che costituiscono il *padding*.

In particolare per la matrice locale A il progetto permette:

$$lda \geq n_{loc}.$$

Il numero n_{loc} specifica quindi quanti elementi della riga appartengono effettivamente alla matrice locale, mentre lda ne descrive la disposizione fisica in memoria. Separare le due quantità permette di introdurre padding senza modificare né le dimensioni logiche del problema né l'interfaccia del backend.

La distinzione è utile anche durante la distribuzione MPI: un blocco bidimensionale estratto dalla matrice globale non è, in generale, una singola regione contigua del buffer row-major: i segmenti appartenenti a righe differenti sono separati dallo stride della matrice globale. La distribuzione utilizza datatype MPI derivati per descrivere tale struttura e depositare i dati nella rappresentazione locale rispettando la lda scelta. I dettagli della distribuzione sono descritti nel capitolo 3.

Per quanto riguarda invece X_{loc} e Y_{loc} l'interfaccia mantiene ugualmente esplicite le rispettive leading dimension, ma la pipeline distribuita utilizza buffer compatti, cioè con:

$$ldx = ldy = k.$$

Questa scelta semplifica le comunicazioni eseguite a ogni invocazione del prodotto. Le porzioni di X coinvolte nel broadcast e i contributi di Y coinvolti nella riduzione possono essere trattati come regioni contigue di memoria, senza introdurre padding o ulteriori descrizioni del layout nel percorso ripetuto dell'algoritmo.

La matrice A , al contrario, viene distribuita durante il preprocessing e rimane poi residente nel processo. Il suo buffer locale può quindi utilizzare una leading dimension più generale senza modificare le comunicazioni ripetute del prodotto.

La compattezza delle righe di X è inoltre rilevante per l'ordine dei cicli discusso nella sezione successiva, perché gli elementi:

$$X[j][0], X[j][1], \dots, X[j][k-1]$$

occupano posizioni consecutive in memoria.

Un backend CUDA considera anche una rappresentazione alternativa di X per colonne. Si tratta di una variante specifica, introdotta e discussa nel capitolo 5. Salvo quando indicato esplicitamente, nel seguito si assume il normale layout row-major.

2.3 Ordine dei cicli e riuso dei dati

La eq. (2.2) coinvolge tre indici, i , j e c . Cambiare l'ordine con cui vengono percorsi non modifica il risultato matematico, ma modifica il modo in cui i dati vengono riutilizzati e acceduti in memoria. Due implementazioni equivalenti dal punto di vista numerico possono quindi avere comportamenti prestazionali molto diversi.

Schema A ($i \rightarrow j \rightarrow c$)

```

for i:
  acc[0..k) = 0
  for j:
    a = A[i][j]
    for c:
      acc[c] += a * X[j][c]
  Y[i][0..k) = acc[0..k)

```

Schema B ($i \rightarrow c \rightarrow j$)

```

for i:
  for c:
    s = 0
    for j:
      s += A[i][j] * X[j][c]
    Y[i][c] = s

```

2.3.1 I due schemi candidati

Lo *schema B* segue direttamente la definizione elemento per elemento. Per ogni riga i e per ogni colonna c di Y , percorre l'intera dimensione j e calcola:

$$Y[i][c] = \sum_j A[i][j]X[j][c].$$

Può quindi essere interpretato come l'esecuzione di k prodotti matrice-vettore indipendenti, uno per ciascuna colonna di X .

Lo *schema A* scambia invece i due cicli interni. Fissati i e j , l'elemento

$$a = A[i][j]$$

viene utilizzato per aggiornare tutti i k elementi della riga corrente di Y :

$$\text{acc}[c] += a X[j][c], \quad 0 \leq c < k.$$

In questo modo ogni utilizzo di $A[i][j]$ necessario alle k colonne del risultato viene concentrato nella stessa iterazione del ciclo su j .

Dal punto di vista del codice, lo schema B richiede ogni elemento di A una volta per ciascuna delle k colonne, mentre nello schema A ogni elemento viene caricato una volta e riutilizzato immediatamente per tutti i k accumulatori.

Questa distinzione riguarda il *numero di accessi* espresso dall'algoritmo. Il traffico effettivamente generato verso i diversi livelli della gerarchia di memoria può essere anche inferiore, perché le riletture dello schema B possono essere servite dalle cache. Lo schema A rende tuttavia il riuso esplicito nel codice, senza dipendere dalla capacità della cache di conservare i dati fra una colonna e la successiva.

L'efficienza con cui questo riuso viene realizzato dipende poi dal backend. In particolare, mantenere gli accumulatori in registri ed evitare spill in memoria diventa un aspetto importante delle implementazioni CPU e CUDA, discusso nei rispettivi capitoli.

2.3.2 Intensità rispetto al traffico di A

Per quantificare il vantaggio dello schema A è utile considerare separatamente il traffico associato alla matrice A .

Supponiamo, come modello ideale, che ogni accesso ad A che non viene riutilizzato esplicitamente debba essere soddisfatto dal livello di memoria considerato. Indicando con s la

dimensione in byte di uno scalare, lo schema A richiede $MN s$ byte associati ad A , mentre lo schema B ne richiede $kMN s$.

Rapportando i $2MNk$ FLOP del prodotto a questo solo contributo si ottiene:

$$\mathcal{I}_A^{(A)} = \frac{2MNk}{MN s} = \frac{2k}{s}, \quad \mathcal{I}_A^{(B)} = \frac{2MNk}{kMN s} = \frac{2}{s}. \quad (2.5)$$

Queste quantità *non* rappresentano l'intensità aritmetica completa del kernel, perché non includono il traffico associato a X e Y e non modellano gli effetti delle cache. Servono invece a isolare il beneficio del riuso esplicito della matrice A introdotto dallo schema A.

Il fattore k mostra quanto l'ordine dei cicli possa aumentare il riuso esplicito della matrice dominante.

Per stabilire se questo sia sufficiente a spostare il kernel da un regime limitato dalla memoria a uno limitato dal calcolo è però necessario includere anche gli altri flussi di memoria e confrontare l'intensità complessiva con le caratteristiche dell'hardware. Questa analisi è svolta nel ??.

2.3.3 Il limite rispetto al traffico di A

Nel modello appena descritto, lo schema A raggiunge il minimo traffico possibile per la matrice A : ciascuno degli MN elementi deve essere reso disponibile almeno una volta per poter contribuire al prodotto.

Ulteriori ottimizzazioni non possono quindi ridurre questo limite algoritmico sulla matrice A , ma possono agire sul modo in cui la matrice X viene acceduta, sulla permanenza degli accumulatori nei registri e sull'effettivo sfruttamento della gerarchia di memoria.

2.3.4 Località degli accessi

Lo schema A presenta inoltre una proprietà favorevole nel codice sequenziale. Al crescere di j , gli elementi

$$A[i][0], A[i][1], \dots$$

sono attraversati nell'ordine con cui compaiono nella riga di A .

Per ogni j , il ciclo interno su c attraversa invece

$$X[j][0], X[j][1], \dots, X[j][k-1],$$

anch'essi consecutivi nel layout row-major adottato dal progetto.

Lo schema espone quindi una buona località spaziale per entrambi i buffer nel percorso sequenziale. Il modo in cui tale proprietà si traduce sull'hardware parallelo dipende però dalla mappatura del lavoro: l'ordine dei cicli descrive quali dati vengono visitati consecutivamente nell'esecuzione sequenziale, ma non dice ancora quali dati vengono elaborati nello stesso momento dai diversi thread. In particolare, sulla CPU conta quali elementi vengono elaborati consecutivamente da un core e quali cicli possono essere vettorizzati, mentre sulla GPU conta quali indirizzi vengono richiesti simultaneamente dalle lane di uno stesso warp.

La *coalescenza degli accessi* non può quindi essere dedotta dal solo ordine dei cicli e verrà analizzata separatamente per ciascun kernel CUDA.

2.4 Interfaccia comune dei backend locali

Dopo aver definito il prodotto locale, la rappresentazione dei dati e l'organizzazione di base del calcolo, resta da stabilire come il livello distribuito interagisca con le diverse implementazioni del nucleo locale.

Tutti i backend espongono la stessa interfaccia, definita in `src/kernel/kernel.h`. Il livello MPI può quindi utilizzare il nucleo locale senza conoscere i dettagli della sua implementazione, che può essere basata sulla CPU, su un kernel CUDA oppure su una libreria esterna come cuBLAS.

2.4.1 Ciclo di vita del backend

Le tre operazioni principali dell'interfaccia sono:

```
local_gemm_t *local_gemm_create(int m, int n, int k,
                                const scalar_t *A_loc,
                                int lda, int ldx, int ldy);

void local_gemm(local_gemm_t *ctx,
                const scalar_t *RESTRICT X, int ldx,
                scalar_t *RESTRICT Y, int ldy);

void local_gemm_destroy(local_gemm_t *ctx);
```

Listato 2.1: Operazioni principali dell'interfaccia definita in `src/kernel/kernel.h`.

Le tre funzioni corrispondono alle fasi di inizializzazione, esecuzione e finalizzazione del backend locale. La separazione non è soltanto organizzativa, ma riflette il diverso ciclo di vita dei dati coinvolti nel prodotto.

Il blocco locale A_{loc} rimane invariato durante tutte le ripetizioni del nucleo. Può quindi essere associato al contesto una sola volta durante `local_gemm_create`. I buffer relativi a X e Y , invece, appartengono alla singola invocazione e vengono forniti a `local_gemm` ogni volta che il prodotto deve essere eseguito.

La funzione `local_gemm_create` inizializza quindi il contesto del backend e prepara tutte le risorse che possono essere riutilizzate fra più invocazioni. Nei backend CPU ciò comprende principalmente le informazioni necessarie a descrivere il problema e il blocco locale di A . Nei backend CUDA vengono inoltre inizializzate le risorse del device, allocati i buffer persistenti e trasferita A_{loc} nella memoria della GPU.

La funzione `local_gemm` rappresenta invece il percorso ripetuto del calcolo. Riceve il blocco di X reso disponibile al processo dal livello distribuito e produce il corrispondente contributo locale in Y . Nei backend CUDA vengono riutilizzati i buffer predisposti durante l'inizializzazione: X viene trasferita sul device, viene eseguito il prodotto locale e il risultato viene infine riportato nella memoria host.

Al termine dell'esecuzione, `local_gemm_destroy` rilascia tutte le risorse associate al contesto, comprese le eventuali allocazioni e risorse CUDA.

Questa organizzazione evita di ripetere a ogni invocazione le operazioni associate ai dati persistenti e mantiene separati i costi di preparazione da quelli del percorso effettivamente ripetuto durante il benchmark.

2.4.2 Contesto opaco del backend

Lo stato specifico di ciascuna implementazione è racchiuso nel tipo opaco `local_gemm_t`. Il codice che utilizza l'interfaccia conosce soltanto un puntatore a tale tipo, mentre la struttura concreta viene definita separatamente da ciascun backend.

Questa scelta permette alle diverse implementazioni di mantenere informazioni e risorse differenti senza modificare il codice che le utilizza. Un backend CPU può, per esempio, conservare le dimensioni del problema e il riferimento al blocco locale di A , mentre un backend CUDA deve mantenere anche i puntatori ai buffer sul device e le risorse necessarie all'esecuzione e alla misurazione.

Il contesto costituisce quindi il confine fra l'interfaccia comune e lo stato specifico dell'implementazione. Il livello distribuito non deve conoscere come viene realizzato il prodotto locale: fornisce i dati attraverso `local_gemm` e riceve il risultato tramite la stessa interfaccia per tutti i backend.

2.4.3 Informazioni sperimentali

Oltre alle tre operazioni principali, l'interfaccia espone alcune funzioni di introspezione utilizzate dal driver di benchmark. Queste permettono di recuperare informazioni specifiche del backend, come tempi di inizializzazione, tempi di trasferimento, tempo del calcolo locale e parametri della configurazione effettivamente utilizzata.

Tali funzioni non partecipano alla definizione matematica del prodotto e non modificano il contratto di `local_gemm`. Servono esclusivamente a rendere osservabili separatamente i costi che compongono l'esecuzione. Il significato delle singole metriche e il modo in cui vengono aggregate sono descritti nel capitolo dedicato alla metodologia sperimentale.

2.4.4 Selezione del backend

L'implementazione del nucleo locale viene selezionata a tempo di compilazione. Tutti i backend rispettano la stessa interfaccia, per cui il driver e il livello MPI non devono essere modificati quando cambia l'implementazione utilizzata per il prodotto locale.

La configurazione di build controlla anche alcune proprietà comuni, come la precisione numerica, e parametri specifici dei singoli backend. Le configurazioni utilizzate nelle campagne di misura e la loro identificazione nei risultati vengono descritte insieme alla metodologia sperimentale.

L'interfaccia realizza quindi la separazione fra i due livelli introdotti nel capitolo 1: MPI stabilisce quali dati e quale contributo al prodotto appartengono a ciascun processo, mentre il backend locale stabilisce come quel contributo viene calcolato.

Capitolo 3

Parallelizzazione MPI

Fino a questo punto il prodotto è stato descritto come se avvenisse dentro un unico spazio di memoria: i cicli di sezione 2.3 leggono A e X e scrivono Y come se tutte e tre fossero lì, a disposizione. Questo capitolo rimuove quell'ipotesi. Il problema diventa far collaborare più processi che non condividono niente, e le scelte che ne derivano condizionano tutto il resto della relazione: la forma dei blocchi, che cosa viene cronometrato, e perfino quali configurazioni ha senso misurare.

3.1 Che cosa significa parallelizzare in MPI

MPI (*Message Passing Interface*) è uno standard per programmare macchine a **memoria distribuita**. Il modello di esecuzione è il seguente: lo stesso programma viene avviato in P copie, ciascuna un processo del sistema operativo a sé stante, con il proprio spazio di indirizzi **privato**. Nessun processo può leggere o scrivere una variabile di un altro. L'unico modo perché un dato passi da un processo all'altro è che qualcuno lo invii esplicitamente, con una chiamata di libreria, e qualcun altro lo riceva.

È una differenza sostanziale rispetto a un modello a memoria condivisa, dove i thread vedono gli stessi indirizzi e la parallelizzazione consiste essenzialmente nel ripartire le iterazioni di un ciclo. Qui ripartire le iterazioni non basta, e non è nemmeno la prima cosa da decidere: prima bisogna stabilire **chi possiede quale dato**. Da questa scelta discende tutto il resto, perché un processo può calcolare solo ciò per cui possiede gli ingredienti, e ogni ingrediente mancante è un messaggio da far viaggiare.

La parallelizzazione del nucleo si articola quindi in tre domande, in quest'ordine:

1. **decomposizione** - come si tagliano A , X e Y fra i processi;
 2. **località** - quale parte del risultato ciascun processo riesce a calcolare con i soli dati che possiede;
 3. **comunicazione** - che cosa manca, e in quale direzione deve viaggiare perché il risultato locale diventi il risultato giusto.
- Il **rank** è l'identificatore intero di un processo all'interno di un gruppo: è ciò che permette a un programma unico di comportarsi diversamente su processi diversi.
 - Un **comunicatore** è un gruppo di processi assieme a un contesto di comunicazione isolato. Ogni operazione MPI avviene *dentro* un comunicatore, e i rank hanno senso solo rispetto a quello: `MPI_COMM_WORLD` è il comunicatore predefinito che contiene tutti i processi, ma se ne possono costruire altri più piccoli, ed è esattamente ciò che il progetto fa.

- Un'operazione **collettiva** è un'operazione che tutti i processi di un comunicatore devono chiamare. Il progetto ne usa due: il *broadcast*, in cui un processo possiede un dato e al termine lo possiedono tutti; e la *reduce*, in cui ogni processo possiede un array della stessa forma, gli array vengono combinati elemento per elemento con un operatore associativo — qui la somma — e il risultato finisce su un solo processo.
- Il **root** è quel processo designato: la sorgente nel broadcast, il destinatario nella reduce.

3.2 Perché si taglia A , e perché in due direzioni

3.2.1 Che cosa tagliare

La prima decisione riguarda quale delle tre matrici vada distribuita. La risposta viene da ???: A ha MN elementi, X e Y ne hanno rispettivamente Nk e Mk , e con k piccolo per definizione A è più grande delle altre due di un fattore che cresce con la taglia del problema.

Replicare A su ogni processo sarebbe quindi la scelta peggiore possibile, e per due ragioni indipendenti: ogni nodo dovrebbe contenerla per intero, mettendo un tetto alla taglia massima trattabile a prescindere da quanti processi si usino; e ogni processo la rileggerebbe tutta, quando è proprio il traffico su A a determinare il tempo. A va dunque **tagliata**. X e Y , che sono piccole, possono invece permettersi di essere replicate lungo un asse, e vedremo che è esattamente ciò che conviene fare.

3.2.2 Come tagliare: due possibilità monodimensionali

Tagliare A significa assegnare a ogni processo una sottomatrice. Le decomposizioni più semplici la tagliano lungo una sola dimensione, e la eq. (2.2) dice subito che cosa comportano.

Taglio a bande di righe. Ogni processo possiede righe *interi* di A . Per calcolare $Y[i][c]$ serve tutta la riga i di A e tutte le righe di X : un processo che possiede righe intere possiede quindi tutti i fattori del primo tipo, ma gli serve X per intero. Il vantaggio è che il risultato che calcola è già *definitivo*: nessuna somma finale fra processi. Il costo è che X deve essere replicata su tutti i P processi.

Taglio a bande di colonne. Ogni processo possiede colonne di A , cioè, per ciascuna riga, solo una parte degli addendi della sommatoria. Gli basta quindi la porzione di X corrispondente alle proprie colonne — X viene tagliata e non replicata — ma ciò che produce è un risultato **parziale**, che riguarda tutte le righe di Y e va sommato con i parziali di tutti gli altri. La somma finale coinvolge P processi su un oggetto della taglia piena di Y .

Le due alternative sono speculari: la prima paga la replicazione di X su tutti, la seconda la riduzione di Y su tutti. In entrambe la comunicazione coinvolge sempre l'intero insieme dei processi, e questo è il difetto strutturale: aumentare P non rende i messaggi più piccoli, li rende soltanto più numerosi.

3.2.3 Il taglio bidimensionale

La terza possibilità taglia A in entrambe le direzioni: ogni processo possiede un *rettangolo* della matrice. Le due necessità di prima compaiono ora entrambe — servono le righe di X corrispondenti alle proprie colonne di A , e il proprio contributo a Y è parziale e va sommato — ma ciascuna coinvolge soltanto un **sottoinsieme** dei processi: quelli che condividono le stesse colonne di A per la prima, quelli che condividono le stesse righe per la seconda. Nessuna delle due comunicazioni è più globale.

È questa la decomposizione richiesta dalla traccia, ed è quella che il progetto implementa. Che sia effettivamente superiore alle due monodimensionali non è però qualcosa da accettare sulla fiducia: è una proprietà quantificabile, e la sezione 3.8 la dimostra con un modello di costo, mostrando che le due decomposizioni 1D ne sono i casi degeneri. Il confronto viene poi misurato direttamente in capitolo 7.

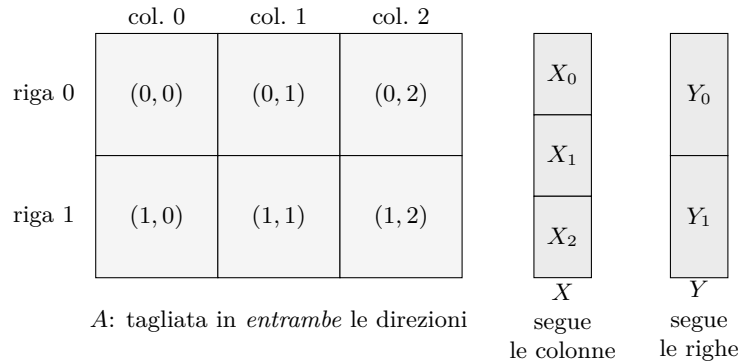


Figura 3.1: Decomposizione dei dati su una griglia 2×3 . Ogni processo possiede un *rettangolo* di A , individuato dalle sue coordinate nella griglia. Il multivettore X è tagliato per bande di righe, una per ciascuna banda di colonne di A ; Y per bande di righe, una per ciascuna banda di righe di A . Nessuna delle due è mai tagliata lungo k .

3.2.4 Perché i processi vengono organizzati a griglia

Resta un ultimo passaggio, che è di comodità ma non solo. Se il taglio del dato è bidimensionale, conviene che lo sia anche il modo in cui si identificano i processi: un processo individuato da una coppia di coordinate (r, c) invece che da un intero rende immediate le due relazioni che servono di continuo. «I processi che possiedono le stesse colonne di A » diventa «i processi della stessa colonna della griglia», e non un calcolo aritmetico sui rank da rifare ogni volta.

MPI offre questa struttura nativamente, sotto il nome di **topologia virtuale cartesiana**. È virtuale nel senso che non descrive come i nodi sono fisicamente cablati: descrive come il programma vuole *pensare* all'insieme dei propri processi, lasciando però all'implementazione la possibilità di sfruttare quell'informazione per disporli meglio sulla macchina reale.

3.3 Decomposizione a griglia bidimensionale

I P processi vengono organizzati in una griglia cartesiana $P_r \times P_c = P$, costruita con `MPI_Cart_create`. La chiamata restituisce un nuovo comunicatore nel quale ogni processo ha, oltre al proprio rank, una coppia di coordinate recuperabile con `MPI_Cart_coords`: sono le (r, c) della sezione precedente, ed è su di esse che il codice ragiona da qui in avanti.

Due parametri della chiamata meritano una nota:

- la griglia è dichiarata **non periodica**. La periodicità serve agli schemi in cui ogni processo comunica con i propri vicini e il bordo deve richiudersi sul lato opposto; qui le comunicazioni sono collettive lungo un'intera riga o colonna, e un collegamento fra l'ultimo processo e il primo non avrebbe alcun significato;
- `reorder = 1` autorizza l'implementazione a **rinumerare** i processi rispetto a `MPI_COMM_WORLD`, per far corrispondere meglio la griglia logica alla topologia fisica della macchina. È la libertà che rende utile una topologia virtuale, ma ha una conseguenza da non dimenticare: da quel momento l'unico rank valido è quello del nuovo comunicatore, e assumere che coincida con quello di partenza sarebbe un errore.

La forma della griglia può essere imposta dall'utente con i flag `-pr` e `-pc`. In assenza di indicazione, `grid_default_shape` sceglie la fattorizzazione più vicina alla forma quadrata, prendendo come P_r il più grande divisore di P non superiore a $\sqrt{P}$; ne segue $P_r \leq P_c$, e nel caso limite di P primo la griglia degenera in $1 \times P$. La ragione per cui il quadrato è un buon default, e i casi in cui non lo è, sono in sezione 3.8.

3.3.1 I due sotto-comunicatori

La griglia da sola non basta: le collettive vanno eseguite lungo una riga o lungo una colonna, non sull'insieme di tutti i processi, e una collettiva agisce su un comunicatore intero. Servono quindi comunicatori più piccoli, uno per ogni riga e uno per ogni colonna della griglia.

Li produce `MPI_Cart_sub`, a cui si indica quali dimensioni lasciar variare e quali tenere fisse. Fissando la coordinata di riga e lasciando variare quella di colonna si ottiene `row_comm`, che raccoglie i processi di una stessa *riga*; fissando la colonna si ottiene `col_comm`, che raccoglie quelli di una stessa *colonna*. Ogni processo appartiene esattamente a un `row_comm` e a un `col_comm`, e sono i due assi lungo cui viaggiano le uniche due collettive del cammino cronometrato.

3.3.2 Chi fa da root, e perché la scelta è verificata

Entrambe le collettive hanno bisogno di un root, e in entrambi i casi il processo giusto è determinato dal problema, non arbitrario: il broadcast di X deve partire dal processo di riga 0, che è l'unico a possedere inizialmente quella fetta, e la reduce di Y deve arrivare al processo di colonna 0, che è il proprietario designato di quella porzione del risultato.

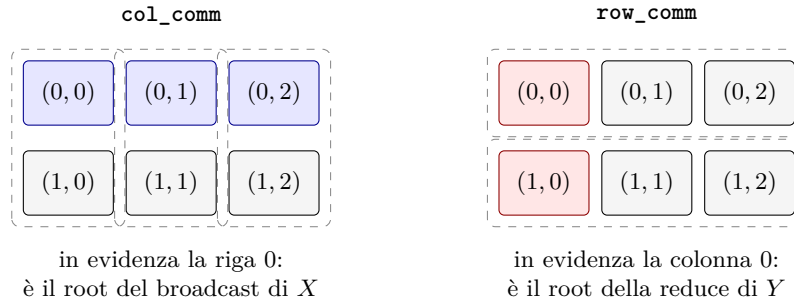


Figura 3.2: La stessa griglia, raggruppata nei due modi. Fissando la coordinata di riga si ottengono i comunicatori `row_comm`, fissando quella di colonna i comunicatori `col_comm`. Ogni processo appartiene a esattamente uno di ciascun tipo, e i due si intersecano soltanto in lui.

Indicare il root a una collettiva significa però indicarne il *rank nel sotto-comunicatore*, non le coordinate nella griglia. Il codice sfrutta allora una proprietà di `MPI_Cart_sub`: in un sotto-comunicatore con una sola dimensione residua i rank risultano ordinati per coordinata crescente. Ne segue che il rank 0 di `col_comm` è il processo di riga 0, e il rank 0 di `row_comm` è il processo di colonna 0: in entrambi i casi il root cercato, e le due collettive possono usare semplicemente 0.

La proprietà viene comunque **verificata** in `grid_create`, confrontando il rank in ciascun sotto-comunicatore con la coordinata cartesiana corrispondente. Il motivo è la libertà concessa da `reorder = 1`: se l'ipotesi cadesse, le collettive continuerebbero a funzionare regolarmente, ma con il root sbagliato, e il programma produrrebbe risultati errati senza segnalare nulla.

3.4 Partizionamento degli indici

Decidere la forma della griglia non basta: bisogna ancora stabilire, per ciascuna delle due dimensioni, quali indici globali finiscano su quale parte. La regola è una sola e viene applicata due volte — alle righe di A fra le P_r righe della griglia, alle colonne fra le P_c colonne — ed è una partizione **a blocchi**: ogni parte riceve un intervallo contiguo di indici, e il resto della divisione viene distribuito un elemento per volta sulle prime parti.

Il blocco contiguo non è una scelta di comodo. L'alternativa naturale sarebbe una distribuzione ciclica, che assegna gli indici a turno; ma qui il lavoro per elemento è uniforme, quindi il ciclico non bilancerebbe nulla che il blocco già non bilanci, e in cambio distruggerebbe la contiguità — cioè proprio la proprietà da cui dipendono sia il datatype derivato della distribuzione (sezione 3.6) sia il passo unitario degli accessi dello schema A (sezione 2.3).

3.4.1 Le funzioni

Posti $q = \lfloor n/p \rfloor$ e $r = n \bmod p$, le prime r parti hanno $q + 1$ elementi e le restanti q . Da qui le prime due funzioni, che dicono *quanti* elementi tocchino alla parte i e *da dove* cominci:

$$\text{size}(n, p, i) = q + \begin{cases} 1 & i < r \\ 0 & \text{altrimenti} \end{cases} \quad \text{start}(n, p, i) = i q + \min(i, r). \quad (3.1)$$

Il $\min(i, r)$ nell'offset conta quante parti *lunghe* precedono la parte i : finché $i < r$ sono tutte lunghe e se ne accumula una per ciascuna, dopo il loro numero resta fermo a r . La funzione è definita anche per $i = p$, dove vale n : una sentinella che rende l'ultima parte un caso come gli altri.

Lo sbilanciamento massimo è di un elemento per dimensione: fra due blocchi locali qualsiasi c'è al più una riga e una colonna di differenza.

3.5 Layout dei blocchi locali

Applicando la partizione alle due dimensioni si ottiene la forma dei tre blocchi che ogni processo possiede:

$$A_{\text{loc}} : m_{\text{loc}} \times n_{\text{loc}}, \quad X_{\text{loc}} : n_{\text{loc}} \times k, \quad Y_{\text{loc}} : m_{\text{loc}} \times k, \quad (3.2)$$

con $m_{\text{loc}} = \text{size}(M, P_r, \text{my_row})$ e $n_{\text{loc}} = \text{size}(N, P_c, \text{my_col})$.

3.5.1 Perché i conteggi delle collettive tornano

Nelle due formule qui sopra c'è una proprietà che vale la pena rendere esplicita, perché è la condizione di consistenza delle due collettive: m_{loc} dipende **solo** da **my_row** e n_{loc} **solo** da **my_col**.

Ne segue che tutti i processi di una stessa riga della griglia concordano su m_{loc} , e tutti quelli di una stessa colonna concordano su n_{loc} . Sono esattamente i due gruppi su cui agiscono le collettive: il broadcast lungo **col_comm** usa un conteggio di $n_{\text{loc}} \cdot k$ elementi, e i suoi partecipanti condividono la colonna, dunque n_{loc} ; la reduce lungo **row_comm** usa $m_{\text{loc}} \cdot k$, e i suoi partecipanti condividono la riga, dunque m_{loc} . Nessuno dei due conteggi va negoziato o comunicato: viene dalla stessa formula, valutata su coordinate che i partecipanti hanno in comune.

3.5.2 Chi contiene che cosa, e quando

Il buffer X_{loc} esiste su *tutti* i processi — il broadcast deve avere dove scrivere — ma prima del broadcast contiene dati significativi soltanto sui processi della riga 0. Dopo il prodotto locale ogni processo possiede un contributo *parziale* a Y , che riguarda le proprie righe ma soltanto le proprie colonne di A ; la `MPI_Reduce` lungo `row_comm` li somma e produce il blocco definitivo sui processi della colonna 0.

3.5.3 Perché il multivettore non viene mai tagliato lungo k

Le k colonne restano indivise su tutti i processi. Tre ragioni, in ordine di peso:

- **costringerebbe a replicare A .** Per calcolare un qualunque sottoinsieme di colonne di Y serve comunque *tutta* A : i processi di una stessa partizione lungo k vorrebbero perciò lo stesso identico blocco di A . Si duplicherebbe il dato dominante (??), che è l'opposto di ciò per cui la decomposizione esiste;
- **ridurrebbe il riuso di A in registro.** Nello schema A il fattore di riuso è pari al numero di colonne che il processo possiede localmente (sezione 2.3): frazionare k lo frazionerebbe nella stessa misura, abbassando l'intensità aritmetica del nucleo;
- **produrrebbe messaggi troppo piccoli.** k è piccolo per definizione del problema, e frammentarlo ulteriormente darebbe comunicazioni dominate dalla latenza invece che dalla banda.

Invariante critico

Il numero di righe locali di X deve venire dalla *stessa* partizione con cui si tagliano le colonne di A , e il numero di righe locali di Y dalla stessa con cui se ne tagliano le righe. Non è un controllo da eseguire, è una proprietà garantita *per costruzione*: esistono soltanto m_{loc} e n_{loc} , calcolati una volta sola in `layout_init`, e non due coppie di valori da tenere allineate. La struttura `layout_t` è il punto unico di verità sulla forma dei blocchi, e finché resta tale l'invariante non può essere violato senza modificarla.

3.6 Distribuzione iniziale dei dati

Le modalità di inizializzazione di A e X sono indipendenti (`-a-mode`, `-x-mode`), e tutte e quattro le combinazioni sono valide. Le due modalità di ciascuna matrice producono gli stessi elementi globali a parità di seed, per la proprietà del generatore discussa in ??.

-a-mode global. Il grid rank 0 materializza A completa e la distribuisce. È il caso interessante dal punto di vista MPI: i sottoblocchi 2D **non sono contigui** nella matrice row-major globale. Il mittente descrive il blocco da inviare con un `MPI_Type_vector` di m_{loc} blocchi da n_{loc} elementi e stride N . Il destinatario ne descrive uno con stride `lda`, che può essere diverso: è il caso generale - due layout differenti sui due lati della stessa comunicazione - e non richiede nessun buffer intermedio né nessuna copia manuale. Il rank 0 fa eccezione per il proprio blocco, che copia direttamente senza passare da MPI.

-a-mode local. Ogni processo genera direttamente il proprio blocco 2D di A dagli indici globali. In questo modo non è necessario materializzare una copia globale aggiuntiva della matrice su un singolo processo.

-x-mode global Il grid rank 0 materializza la X compatta $N \times k$ e un `MPI_Scatterv` sul `row_comm` della riga 0 ne distribuisce i blocchi di righe: entrambi i lati sono contigui, perché $ldx = k$ ovunque, quindi non serve nessun datatype derivato.

-x-mode local I processi della riga 0 generano direttamente le proprie righe di X . Il successivo broadcast lungo `col_comm` è identico alla modalità `global`.

Generazione e distribuzione iniziale fanno parte del **preprocessing**, perciò sono escluse dai tempi ufficiali. La modalità `global` rappresenta uno scenario in cui l'input è inizialmente centralizzato.

3.7 L'algoritmo distribuito

Un'invocazione del prodotto esegue tre passi:

```
MPI_Bcast(X_loc, n_loc*k, SCALAR_MPI_TYPE, 0, grid->col_comm);

local_gemm(local_gemm_context, X_loc, layout->ldx, Y_loc_part, layout->ldy);

MPI_Reduce(Y_loc_part, Y_row_col0, m_loc*k, SCALAR_MPI_TYPE,
           MPI_SUM, 0, grid->row_comm);
```

Listato 3.1: Il corpo di `mpi_matmul`, senza i cronometri.

1. **Broadcast di X lungo `col_comm`:** tutti i processi di una colonna della griglia lavorano sulle stesse colonne di A , quindi servono loro le stesse righe di X . Il root è il processo di riga 0, l'unico che possiede inizialmente quella fetta, ne è il *responsabile*.
2. **Prodotto locale:** il risultato è *parziale*: il processo in posizione (r, c) calcola il contributo delle sole colonne $[col_0, col_0 + n_{loc})$ di A alle righe $[row_0, row_0 + m_{loc})$ di Y (dove row_0 e col_0 sono gli offset globali del blocco locale).
3. **Reduce lungo `row_comm`:** i P_c contributi parziali della stessa banda di righe vivono sui processi di una stessa riga della griglia: sommandoli tra loro si ottiene Y . Il count è $m_{loc} \cdot k$ (e non 1) dato che la riduzione agisce lungo la dimensione dei processi e preserva la posizione, producendo dunque $m_{loc} \cdot k$ somme indipendenti. Il root è il processo di colonna 0, che è già il *responsabile* designato di quella porzione di Y : nessuna ridistribuzione finale è necessaria.

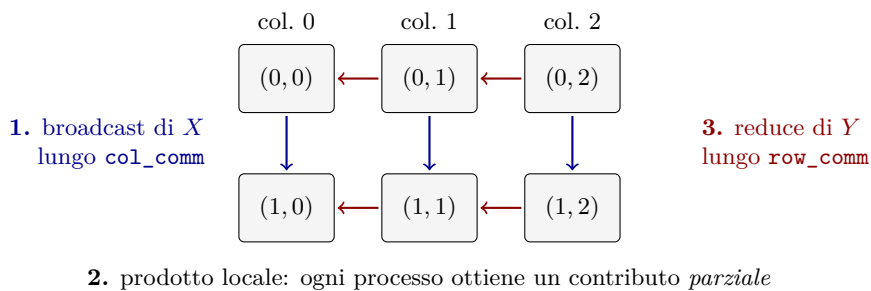


Figura 3.3: Le due collettive di una singola invocazione. Il broadcast fa scendere la fetta di X dal processo di riga 0 a tutti i processi della sua colonna, che ne hanno bisogno perché condividono le stesse colonne di A . Dopo il prodotto locale ogni processo possiede un contributo *parziale* a Y : la reduce somma i P_c parziali di una stessa banda di righe e deposita il risultato sul processo di colonna 0, che ne è il proprietario designato.

3.8 Modello di costo della comunicazione

Il costo di un messaggio di n byte fra due processi si modella, seguendo le dispense del corso, come

$$T(n) = \lambda + \frac{n}{B}, \quad (3.3)$$

dove λ è la *latenza*, un costo fisso che si paga per ogni messaggio indipendentemente dalla sua taglia, e B è la *banda*. In letteratura lo stesso modello compare spesso nella forma $\alpha + \beta n$: il dizionario è $\alpha \equiv \lambda$ e $\beta \equiv 1/B$.

Le due collettive non sono però messaggi singoli. Il costo logaritmico che segue è quello del *recursive doubling*, l'algoritmo di broadcast visto a lezione, che al posto dei $p-1$ invii sequenziali della versione ingenua costa $T(p) = \lceil \log_2 p \rceil$ tempi di messaggio. Il logaritmo si ottiene grazie a un'ipotesi precisa sulla rete — che essa sostenga più messaggi contemporaneamente, purché le coppie di nodi coinvolte non si sovrappongano — ed è quell'ipotesi, più che la struttura ad albero, a rendere possibile il guadagno.

Detti s i byte per scalare, il costo di comunicazione per invocazione è quindi

$$T_{\text{comm}} \approx \underbrace{\lceil \log_2 P_r \rceil \left(\lambda + \frac{s k N / P_c}{B} \right)}_{\text{broadcast di } X} + \underbrace{\lceil \log_2 P_c \rceil \left(\lambda + \frac{s k M / P_r}{B} + \gamma k \frac{M}{P_r} \right)}_{\text{reduce di } Y}, \quad (3.4)$$

dove γ è il costo per elemento della somma eseguita dalla reduce. Il termine in γ non compare nel modello del corso, che descrive il costo del solo trasferimento: lo aggiungiamo perché la reduce, a differenza del broadcast, a ogni livello dell'albero esegue anche $m_{\text{loc}} \cdot k$ addizioni.

Il modello va letto per quello che è: i fattori logaritmici valgono se l'implementazione usa il recursive doubling, ma le librerie MPI cambiano algoritmo al crescere del messaggio e per messaggi grandi passano a schemi a pipeline o scatter/allgather, nei quali il termine di banda perde il fattore logaritmico e resta proporzionale al solo volume trasferito.

Il modello va letto per quello che è: i fattori logaritmici valgono se l'implementazione usa il recursive doubling, ma — come le dispense stesse osservano — una buona libreria MPI commuta internamente fra algoritmi diversi a seconda dell'operazione, della rete e soprattutto della quantità di dati. Per messaggi grandi si passa a schemi a pipeline o scatter/allgather, nei quali ogni byte attraversa la rete una volta sola invece di $\log_2 p$ volte: il termine di banda perde il fattore logaritmico e resta proporzionale al solo volume, al prezzo di un numero maggiore di messaggi e quindi di più latenza.

Alle taglie della campagna i messaggi cadono in quel regime, quindi il $\log_2$ va inteso come limite superiore sui termini di latenza più che come andamento atteso della banda. Questo non intacca però la conclusione che segue: il *volume*, cioè l'unica quantità da cui dipende la scelta della forma della griglia, è lo stesso per qualunque algoritmo collettivo, perché è quanto ciascun processo deve comunque ricevere o inviare.

Il termine che decide la forma ottima della griglia è il volume:

$$V(P_r, P_c) = k \left(\frac{N}{P_c} + \frac{M}{P_r} \right), \quad P_r P_c = P. \quad (3.5)$$

Minimizzando con il vincolo $P_r P_c = P$ si ottiene

$$P_r^* = \sqrt{P \frac{M}{N}}, \quad P_c^* = \sqrt{P \frac{N}{M}}, \quad V^* = \frac{2k\sqrt{MN}}{\sqrt{P}}. \quad (3.6)$$

Due letture, entrambe verificate sperimentalmente in capitolo 7:

- **Perché 2D e non 1D.** Le due decomposizioni monodimensionali sono i casi degeneri $P_r = 1$ (volume kM , nessun broadcast ma reduce su tutti i P processi) e $P_c = 1$ (volume kN , nessuna reduce ma broadcast su tutti i P). In entrambi il volume *non decresce* con P , mentre la griglia 2D lo fa scendere come $1/\sqrt{P}$. È il motivo per cui la traccia richiede una griglia bidimensionale, ed è ciò che la campagna C6 misura direttamente confrontando tutte le fattorizzazioni di P , forme degeneri comprese.
- **Perché la forma ottima dipende da $M:N$.** Per $M = N$ la griglia migliore è quella quadrata, cioè proprio quella che `grid_default_shape` sceglie da sé — ed è anche quella che lo standard MPI sceglierebbe da sé, visto che `MPI_Dims_create` costruisce la griglia più vicina possibile alla forma quadrata. La eq. (3.6) dice però che quel default è ottimo *soltanto* per $M = N$; per $M = 3N$ conviene invece $P_r > P_c$. La campagna C1 varia il rapporto $M:N$ a lavoro costante e rende visibile questo effetto separandolo da quello della taglia.

Un'osservazione che va tenuta presente leggendo il weak scaling (campagna C5): su una griglia 2D con blocco locale costante, M segue P_r e N segue P_c , quindi cambiare P cambia *anche* il rapporto $M:N$. È inerente alla decomposizione, ed è la ragione per cui l'effetto della forma viene misurato separatamente in C1 e C6.

Capitolo 4

Il nucleo di CPU

Il nucleo di CPU non è la parte più veloce del progetto, ma è quella di cui tutto il resto ha bisogno per essere giudicato: è il denominatore di ogni speedup, ed è l'unica implementazione che gira ovunque, anche su una macchina senza `nvcc`. Merita quindi lo stesso lavoro dei backend CUDA, e per le stesse ragioni.

4.1 Tre backend, una sola idea

Tutti e tre i backend di CPU implementano lo schema A di sezione 2.3: cambia come il lavoro viene organizzato attorno a quell'ordine dei cicli, non l'ordine. Si scelgono a tempo di compilazione con la variabile `KERNEL` del `Makefile`, espongono la stessa interfaccia (??) e i binari coesistono, così un confronto è una sola campagna sugli stessi dati.

- **scheme_a**: l'implementazione di riferimento, quella che corrisponde alla consegna;
- **scheme_a_jblock**: l'implementazione di riferimento è la stessa del caso precedente, ma con X in cui è stato applicato un meccanismo di *tiling* in cache;
- **scheme_a_jblock_rb**: come sopra, ma all'implementazione è stato aggiunto il *register blocking* sulle righe di A .

I tre non sono tre implementazioni alternative, ma riflettono la **catena** di sviluppo: ciascuno rimuove il collo di bottiglia che il precedente lascia scoperto. Il capitolo li presenta in quest'ordine, e ogni sezione si apre con il limite che motiva il backend successivo.

4.2 Il backend di riferimento

4.2.1 Perché specializzare k a compile-time

Nello schema A i k accumulatori sono variabili locali del ciclo su j , e tutto il vantaggio dello schema dipende dal fatto che questi accumulatori vivano nei **registri**: è lì che il valore $A[i][j]$, letto una volta sola, può essere riutilizzato k volte.

Affinché il compilatore possa rappresentare gli accumulatori come un insieme fisso di valori scalari candidati a vivere nei registri, il numero di accumulatori deve essere *noto a compile-time*. Con un array dimensionato dinamicamente, invece, l'accesso `acc[c]` conserva un indice runtime e il compilatore non dispone della stessa struttura esplicita.

Se il numero di accumulatori è noto solo quando il programma parte, la variabile `acc` diventa un array in memoria e ogni operazione di accumulazione (`acc[c] += ...`) è un accesso a quella memoria: esattamente il traffico che lo schema A serviva a evitare.

Il codice risolve generando **cinque funzioni specializzate**, per $k \in 3, 6, 8, 20, 32$, cioè per i cinque valori richiesti dalla traccia, a partire da una sola definizione via macro:

```
#define COLS_8(M) COLS_6(M) M(6) M(7)
#define DECLARE_ACC(c) scalar_t acc##c = (scalar_t)0;
#define UPDATE_ACC(c) acc##c += a * xrow[c];
#define STORE_ACC(c) yrow[c] = acc##c;

/* per ogni riga i: */
COLS(DECLARE_ACC)
for (j = 0; j < n_loc; j++) {
    const scalar_t a = arow[j];          /* letto UNA volta */
    const scalar_t *restrict xrow = X_loc + (size_t)j * ldx;
    COLS(UPDATE_ACC)                     /* k aggiornamenti espliciti */
}
COLS(STORE_ACC)
```

Listato 4.1: Generazione delle specializzazioni (estratto di `src/kernel/scheme_a.c`).

Vale la pena leggere che cosa producono queste righe: `COLS_8` è un elenco di indici, applicarlo a `DECLARE_ACC` genera 8 dichiarazioni di variabili scalari *distinte* (`acc0`, `acc1`, ...), non un array, mentre applicarlo a `UPDATE_ACC` genera 8 aggiornamenti scritti per esteso, senza alcun ciclo interno con estremo runtime.

L'hot path che il compilatore vede non contiene quindi nessuna indicizzazione su `c`: contiene k istruzioni indipendenti su k variabili indipendenti, che fornisce al compilatore una forma particolarmente favorevole all'allocazione degli accumulatori nei registri e alla successiva vettorizzazione.

La generazione effettiva di codice vettoriale e l'eventuale uso di FMA sono proprietà del codice macchina prodotto dal compilatore e vengono quindi verificate separatamente tramite profilazione.

In questo contesto, vettorizzare un ciclo significa che il compilatore riesce a trasformare le k istruzioni indipendenti in un'unica istruzione vettoriale, che opera su W scalari per volta, dove W è la larghezza del registro vettoriale della CPU, utilizzando istruzioni SIMD. Se k è minore di W , il compilatore può comunque generare un'istruzione vettoriale, ma non sarà completamente utilizzata. Se k è maggiore di W , il compilatore genererà più istruzioni vettoriali per coprire tutti gli accumulatori.

4.2.2 Il fallback generico, e perché la traccia lo richiede

La traccia chiede però che il codice funzioni per k *generico*, e cinque specializzazioni non possono coprire tutti i valori possibili. Per ogni altro k interviene un **fallback generico**, che non tenta di tenere k accumulatori distinti, perché sarebbero troppi e finirebbero in memoria, ma lavora a gruppi di al più $KB = 32$ colonne per volta. Ogni gruppo richiede una passata completa su A , con un numero di passaggi pari a:

$$\text{passaggi su } A = \left\lceil \frac{k}{KB} \right\rceil. \quad (4.1)$$

Per $k \leq KB$ il ciclo esterno sui gruppi è *degenere*: esiste nel codice ma gira una volta sola, A viene letta una volta sola e il fallback si differenzia dalle specializzazioni soltanto per l'array di accumulatori invece delle variabili distinte. Per $k > 32$ il numero di passaggi cresce secondo $\lceil k/32 \rceil$, ad esempio tra 33 e 64 sono necessari due passaggi, tra 65 e 96 tre, e così via.

La conseguenza va tenuta presente leggendo i risultati: le righe dei k oltre la soglia e quelle dei k entro la soglia **non sono direttamente confrontabili**, perché non stanno facendo la stessa quantità di lavoro di memoria.

4.3 Perché $KB = 32$

Il valore copre tutti i k del collaudo, ma questa da sola non è una ragione sufficiente. La ragione vera è un budget di registri.

Infatti con KB accumulatori il compilatore deve tenere KB valori vivi attraverso l'intero ciclo su j . Vettorizzati, occupano KB/W registri vettoriali, dove W è il numero di scalari che entrano in un registro, a fronte di 16 registri vettoriali sulle configurazioni AVX2 e 32 sulle configurazioni AVX-512. Al crescere di KB il riuso di A migliora, perché i passaggi della eq. (4.1) diminuiscono, ma cresce la pressione sui registri fino al fenomeno del *register spilling*, cioè fino al punto in cui il compilatore è costretto a parcheggiare in memoria gli accumulatori che non stanno più nei registri e a quel punto si va a perdere più di quanto si guadagni.

Il valore 32 è quindi il compromesso scelto dall'implementazione: è sufficiente a coprire in un solo passaggio tutti i 5 k richiesti dalla traccia, ma mantiene il numero di accumulatori entro una dimensione compatibile con un'implementazione a registri. Il fatto che tale scelta eviti effettivamente lo spilling e sia ottimale sulla macchina di riferimento non è assunto come proprietà teorica: viene verificato sperimentalmente nella campagna C3 (??).

4.4 Ciò che il compilatore vede

Affinché questo basti, il codice deve rispondere da sé alle tre domande che un compilatore si pone prima di vettorizzare un ciclo.

- «**Questi puntatori possono riferirsi alla stessa memoria?**» No: A , X e Y sono dichiarati **restrict**. La garanzia va però *ripristinata* dentro `local_gemm`, con copie locali dei puntatori: il contesto opaco tiene il puntatore ad A come membro di una struttura, e senza quella copia i kernel specializzati perderebbero l'informazione di non aliasing che avevano quando A era un parametro. Se l'aliasing fosse possibile, ogni scrittura su Y potrebbe invalidare A e il ciclo andrebbe eseguito in ordine.
- «**Le iterazioni dipendono l'una dall'altra?**» No: i k aggiornamenti `acc[c] += a * X[j][c]` riguardano accumulatori distinti e sono indipendenti fra loro. Essendo già srotolati dalle macro, si presentano come catene di moltiplicazione-accumulo indipendenti, che è la forma che una unità FMA può eseguire in parallelo.
- «**Gli accessi sono regolari?**» Sì, e a passo unitario: come osservato in sezione 2.3, sia la riga di A sia la riga di X sono contigue in memoria.

Da completare con i dati della campagna

Verificare sul server l'effettiva vettorizzazione del percorso caldo, per esempio con `gcc -fopt-info-vec` oppure ispezionando l'assembly di `kernel_k8` e `kernel_k32`. Il punto da documentare è se e a quale k compaiano spill degli accumulatori.

4.5 Il primo limite: la riletture di X

Lo schema A legge A una volta sola, ed è il suo pregio. Ma per ogni riga i di A percorre **tutta** X , e questo è il suo prezzo: il traffico in lettura su X vale

$$\text{elementi di } X \text{ letti} = m_{\text{loc}} \cdot n_{\text{loc}} \cdot k, \quad (4.2)$$

cioè k volte il traffico su A .

Finché X resta in una cache vicina al calcolo questo traffico non raggiunge la memoria centrale e semplicemente non si vede. Quando X non ci sta più, viene ristreamata da un livello più lontano una volta per ogni riga di A , e il tempo per elemento smette di essere governato dal calcolo per essere governato dalla banda di quel livello.

La soglia è esplicita e dipende da entrambi i parametri del problema: il blocco locale X_{loc} occupa $n_{\text{loc}} \cdot k \cdot s$ byte. A parità di matrice, quindi, è k **che decide da che parte della soglia ci si trova**, ed è per questo che l'effetto si manifesta come un calo dei GFLOPS al crescere di k — cioè nella direzione opposta a quella che l'intensità aritmetica della ?? lascerebbe prevedere. È la chiave di lettura dei risultati di ??, ed è il limite che il backend successivo rimuove.

4.6 Secondo backend: X mattonellata in cache

L'idea di `scheme_a_jblock` è di non percorrere più tutta X per ogni riga di A , ma di prenderne un **tile** — un blocco di righe consecutive — e di applicarlo a tutte le righe di A prima di passare al successivo:

```
for j0 (tile di X di BJ righe):
  for i (TUTTE le righe di A):
    acc = (primo tile) ? 0 : Y[i][*]
    for j nel tile: a = A[i][j]; acc[c] += a * X[j][c]
    Y[i][*] = acc
```

Listato 4.2: Struttura di `src/kernel/scheme_a_jblock.c`.

Il tile viene caricato dalla memoria una volta e riusato m_{loc} volte restando in cache. A continua a essere letta *una volta sola*: ogni sua riga viene semplicemente consumata a segmenti, uno per tile, invece che tutta di seguito.

Il prezzo. Y non può più vivere nei registri per tutta la durata di una riga, perché ci si torna sopra a ogni tile: va riletta e riscritta ogni volta. Il traffico aggiuntivo vale $2k$ scalari per riga e per tile, cioè un rapporto $2k/\text{BJ}$ rispetto al traffico su A . Il tile va quindi scelto *grande* abbastanza da rendere questo termine trascurabile e *piccolo* abbastanza da stare in cache: le due richieste tirano in direzioni opposte, e il punto di equilibrio non è deducibile a priori.

Per questo la dimensione del tile è un parametro di build, `JBLOCK_BYTES`, e non una costante scelta una volta per tutte: il codice ne ricava il numero di righe per tile dividendo per $k \cdot s$, il default è un valore che sta nella cache di secondo livello di qualunque core x86 recente, e quale sia il valore migliore è oggetto di misura.

Un principio già visto. È esattamente la stessa mossa del backend CUDA con tile in shared memory (??): portare un blocco di X in una memoria vicina al calcolo e riusarlo per molte righe prima di passare al blocco successivo. Cambia il nome della memoria vicina — lì la shared memory, qui la cache — non il ragionamento.

4.7 Terzo backend: register blocking sulle righe

Tolto il limite di banda, ne resta scoperto un altro, che si manifesta ai k *piccoli*: in tutti i backend visti finora ogni `acc[c]` è una catena *seriale* di moltiplicazioni-accumulo lungo j — ogni aggiornamento deve attendere il precedente — e l'unico parallelismo a livello di istruzione disponibile è quello fra le k colonne. Poiché una unità FMA ha una latenza di diversi cicli e

la CPU ne ha più di una, per tenerle occupate servono più catene indipendenti di quante k piccolo ne offra: il processore aspetta, e i GFLOPS crescono con k invece di essere costanti.

`scheme_a_jblock_rb` risolve lavorando su RB righe di A per volta:

```
for j0 (tile di X, come in scheme_a_jblock):
  for i a passi di RB:
    acc[r][c] = (primo tile) ? 0 : Y[i+r][c]
    for j nel tile:
      x[c] = X[j][c] /* caricato UNA volta */
      for r: a = A[i+r][j]; acc[r][c] += a * x[c] /* riusato RB volte */
    Y[i+r][c] = acc[r][c]
```

Listato 4.3: Struttura di `src/kernel/scheme_a_jblock_rb.c`.

Le righe di A sono indipendenti per costruzione — nessun elemento di Y dipende da un'altra riga — quindi il numero di catene indipendenti passa da $\lceil k/W \rceil$ a $RB \cdot \lceil k/W \rceil$. In più, ogni valore di X caricato viene riusato RB volte direttamente dai registri, quindi anche il traffico su X — già in cache grazie al tile — si riduce dello stesso fattore.

Chi sceglie RB. Anche qui il limite sono i registri: $RB \cdot \lceil k/W \rceil$ accumulatori più RB valori di A devono starci tutti, altrimenti si torna allo spill e si perde più di quanto si guadagna. Il codice sceglie perciò RB *a tempo di compilazione e per ogni k* , dal numero di registri vettoriali e dalla loro ampiezza dell'architettura di destinazione (`VEC_NREGS` e `VEC_W`, fissati dalle macro del compilatore), prendendo il più grande fra i valori ammessi che soddisfi il vincolo; la variabile di build `RB_ROWS` permette di forzarlo per misurarne l'effetto. Le righe che avanzano quando m_{loc} non è multiplo di RB sono chiuse dal kernel a una riga, che è il caso $RB = 1$ dello stesso schema.

I tre backend rispondono quindi a tre limiti diversi e successivi: il traffico su A , risolto dallo schema A stesso; la banda verso X , risolta dal tile in cache; il parallelismo di istruzione, risolto dal blocco di righe. Quanto ciascuno dei tre passi valga davvero sulla macchina di misura è ciò che la campagna riportata in capitolo 7 quantifica.

Capitolo 5

I backend CUDA

L'implementazione su GPU è stata sviluppata attraverso tre kernel CUDA, affiancati da un backend basato su cuBLAS, usato come riferimento esterno. Le diverse versioni non sono state progettate come soluzioni indipendenti: partono dalla stessa operazione locale, $Y = AX$, e introducono progressivamente modifiche mirate alla mappatura del lavoro e agli accessi alla memoria.

In particolare, il percorso seguito è il seguente:

- **cuda_naive** costituisce la baseline CUDA. A ciascun thread viene assegnato un singolo elemento Y_{ic} : il thread fissa quindi una riga i di A e una colonna c di X , e percorre l'intera dimensione interna j per calcolarne il prodotto scalare. La mappatura è semplice e offre un primo termine di confronto, ma non sfrutta il riuso della riga di A tra le k colonne del multivettore.
- **cuda_warp** cambia la granularità della parallelizzazione: non più un thread per elemento di Y , ma un *warp* per un'intera riga di Y . Le 32 lane del warp si suddividono gli elementi della corrispondente riga di A e ciascuna lane riutilizza il valore di A appena caricato per aggiornare i k risultati della riga. Al termine, i contributi prodotti dalle lane vengono sommati mediante primitive di warp shuffle.
 - **cuda_warp** nella variante **xcol** mantiene esattamente la stessa organizzazione del calcolo, ma modifica la rappresentazione di X . Il blocco locale del multivettore viene convertito una sola volta, durante il preprocessing, da una disposizione in memoria per righe (*row-major*) a una disposizione per colonne (*column-major*). In questo modo quando le lane del warp avanzano su indici j consecutivi, accedono anche a elementi consecutivi di X .
 - **cuda_warp_smem** affronta lo stesso problema degli accessi a X senza però andarne a modificare il layout. Un blocco di thread carica *cooperativamente* una porzione di X dalla memoria globale alla **shared memory**. I warp appartenenti allo stesso blocco possono quindi riutilizzare quella stessa porzione durante il calcolo delle rispettive righe di Y .

Il passaggio da **cuda_naive** a **cuda_warp** modifica la mappatura del lavoro e introduce il riuso di A all'interno del warp. A questo punto è emerso un problema: con X nel normale layout row-major, le lane che avanzano lungo j non leggono elementi consecutivi del multivettore.

La variante **xcol** interviene sulla *rappresentazione del dato*, convertendo preventivamente X in un layout più favorevole agli accessi del warp, mentre **cuda_warp_smem** conserva il layout originario e interviene sul *percorso del dato*, caricando e riutilizzando X attraverso la shared

memory. Le due versioni permettono così di studiare due strategie diverse per lo stesso collo di bottiglia, senza sovrapporle nello stesso kernel.

Infine, si propone anche un'implementazione su GPU basata su cuBLAS, la libreria NVIDIA per algebra lineare ottimizzata per GPU:

- **cublas** non rappresenta un'ulteriore ottimizzazione del kernel sviluppato nel progetto. È un riferimento esterno, inserito dietro la stessa interfaccia degli altri backend, che permette di confrontare le implementazioni precedenti con una libreria GPU general-purpose sulla stessa macchina e nelle stesse condizioni di misura.

5.1 Struttura comune

Tutti i backend GPU rispettano lo stesso ciclo di vita a tre fasi introdotto in ??, che permette di confrontare le diverse implementazioni misurando, in ciascun caso, la stessa porzione dell'esecuzione.

La distinzione fondamentale deriva dal diverso ruolo dei dati. Il blocco locale di A non cambia tra un'invocazione del prodotto e la successiva e può quindi essere trasferito sulla GPU una sola volta. Al contrario, la fetta locale di X deve essere resa disponibile a ciascun processo a ogni invocazione del prodotto: le sue porzioni sono inizialmente collocate sulla prima riga della griglia di processi e vengono replicate lungo ciascuna colonna mediante `MPI_Bcast.`, mentre il risultato parziale Y deve essere riportato sull'host prima della successiva `MPI_Reduce.` Di conseguenza, le operazioni comuni ai backend CUDA sono distribuite nelle tre fasi nel modo seguente:

- **local_gemm_create** prepara il backend prima dell'inizio delle ripetizioni cronometrate. In questa fase viene inizializzato il contesto CUDA, vengono allocati in VRAM i buffer destinati ad A , X e Y , vengono creati gli eventi CUDA usati per le misure e il blocco locale di A viene copiato dall'host al device. Poiché A rimane invariata per tutte le invocazioni successive, questo trasferimento viene eseguito una sola volta. L'intera fase appartiene al preprocessing e non entra nel tempo ufficiale del prodotto. I suoi costi principali vengono comunque registrati separatamente come inizializzazione del device, allocazione della memoria e trasferimento H2D di A .
- **local_gemm** rappresenta invece il cammino percorso a ogni invocazione. Prima del calcolo, il nuovo blocco di X viene trasferito dall'host alla GPU. Segue l'esecuzione del kernel CUDA, oppure della chiamata cuBLAS nel backend di riferimento. Infine il risultato parziale Y viene copiato nuovamente verso l'host. Il tempo `t_local` comprende quindi l'intera sequenza H2D di X , calcolo e D2H di Y .

All'interno della stessa funzione, tre coppie distinte di `cudaEvent` delimitano separatamente il trasferimento H2D di X , il solo calcolo sul device e il trasferimento D2H di Y . Si ottengono in questo modo tre misurazioni: rispettivamente $t_{H2D X}$, t_{kernel} e $t_{D2H Y}$. Questa separazione è necessaria perché la specifica del progetto richiede di escludere dal tempo ufficiale i trasferimenti da e verso la scheda: per i backend CUDA è quindi t_{kernel} , e non l'intero t_{local} , a rappresentare il contributo del calcolo locale alla misura prestazionale.

- **local_gemm_destroy** conclude infine il ciclo di vita del backend, liberando i buffer allocati sulla GPU, gli eventi CUDA e il contesto mantenuto dall'implementazione. Anche questa fase avviene al di fuori delle ripetizioni misurate.

La stessa suddivisione viene mantenuta in tutti i backend GPU. Cambia quindi il modo in cui viene eseguito il prodotto all'interno della seconda fase, ma non cambiano né la posizione

dei trasferimenti né i confini delle misure. Il confronto tra i kernel può così essere effettuato sul solo t_{kernel} , mentre t_{local} rimane disponibile per osservare anche il costo reale dell'intera invocazione lato GPU.

5.2 Misurazione del backend CUDA

La natura asincrona dell'esecuzione CUDA richiede di distinguere il tempo osservato dall'host dal tempo trascorso effettivamente sul device. Il lancio di un kernel, infatti, restituisce il controllo alla CPU senza attendere necessariamente il completamento dell'operazione. Un cronometro host posto semplicemente prima e dopo il lancio misurerebbe semplicemente il costo della chiamata al runtime CUDA, non il tempo di esecuzione del kernel, ciò che invece ci interessa.

Per isolare le singole operazioni eseguite sul device, i backend CUDA utilizzano coppie di **cudaEvent** registrate sullo stesso stream delle operazioni misurate: in questo modo gli eventi permettono di determinare separatamente il tempo necessario al trasferimento H2D di X , all'esecuzione del calcolo e al trasferimento D2H di Y .

Il tempo della fase locale osservato dal driver, indicato con t_{local} , ha invece un significato diverso: misura l'intera invocazione del backend e comprende quindi sia il lavoro eseguito sul device sia l'overhead necessario per richiederlo e sincronizzarlo.

Schematizzando il calcolo, otteniamo:

$$t_{\text{local}} = t_{\text{H2D } X} + t_{\text{kernel}} + t_{\text{D2H } Y} + t_{\text{overhead}}, \quad (5.1)$$

dove t_{overhead} non viene misurato direttamente, ma rappresenta il residuo fra il tempo complessivo osservato dall'host e i tre intervalli misurati mediante eventi CUDA. Esso può comprendere il costo delle chiamate al runtime, delle sincronizzazioni e della gestione della strumentazione.

La distinzione è rilevante anche per la valutazione delle prestazioni, perché come richiesto dalla specifica del progetto, i trasferimenti tra host e device non devono contribuire al tempo utilizzato per calcolare i FLOPS. Per i backend CUDA il contributo computazionale è quindi t_{kernel} , mentre $t_{\text{H2D } X}$ e $t_{\text{D2H } Y}$ vengono conservati come misure separate e analizzati successivamente.

Oltre al tempo, il backend comunica al driver anche il numero di byte trasferiti nelle tre operazioni principali: la copia iniziale di A , la copia di X eseguita a ogni invocazione e la copia di ritorno di Y . Tempo e volume trasferito permettono di ricavare una **banda effettiva**, calcolata come:

$$B_{\text{eff}} = \frac{D}{t}, \quad (5.2)$$

dove D è il numero di byte trasferiti e t il tempo della corrispondente copia.

Il solo tempo di trasferimento sarebbe infatti poco informativo: una copia più grande richiede naturalmente più tempo di una più piccola. La banda effettiva permette invece di confrontare trasferimenti di dimensioni differenti e di valutare quanto efficacemente venga utilizzato il collegamento tra host e GPU.

5.3 Preprocessing e occupazione della GPU

Anche il costo di creazione del backend viene registrato, pur rimanendo esterno alla regione utilizzata per la misura ufficiale. Il tempo t_{setup} viene distinto in tre contributi, perché corrispondono a operazioni di natura differente.

Il primo è l'inizializzazione dell'ambiente CUDA, necessaria affinché il processo possa utilizzare il device. Si tratta principalmente di un costo di avvio e non dipende direttamente dalle dimensioni del prodotto da calcolare.

Il secondo contributo riguarda l'allocazione in memoria device dei buffer necessari per A , X e Y . Questo costo è legato alla preparazione delle risorse del backend e non rappresenta un trasferimento di dati.

Il terzo contributo è invece la copia H2D del blocco locale di A . Poiché A rimane invariata durante tutte le successive invocazioni del prodotto, questa copia viene effettuata una sola volta in `local_gemm_create`. Il suo costo viene quindi escluso dal cammino cronometrato, ma misurato separatamente.

Questa distinzione permette di quantificare direttamente il vantaggio della persistenza di A sul device: se il suo trasferimento fosse ripetuto a ogni chiamata di `local_gemm`, il relativo costo entrerebbe nel percorso critico insieme alla copia di X e a quella di Y .

5.3.1 Gestione della memoria disponibile

La disponibilità della memoria device non viene verificata mediante una stima preventiva utilizzata per decidere se procedere con il calcolo. Il backend tenta invece direttamente le allocazioni necessarie mediante `cudaMalloc` e, a questo punto, se si verifica un errore di allocazione, viene registrato un messaggio di errore e si procede con il fallimento della creazione del backend.

Questa scelta evita di basare la decisione su una precedente interrogazione della memoria libera: infatti, in presenza di più processi MPI che condividono la stessa GPU, il valore restituito da una precedente osservazione dello stato della memoria può cambiare prima dell'allocazione effettiva. In caso di errore, `cudaMemGetInfo` viene comunque utilizzata per produrre una diagnostica che riporta la memoria richiesta e quella disponibile.

I principali buffer persistenti richiedono complessivamente una quantità di byte pari a:

$$(m_{\text{loc}} \cdot \text{lda} + n_{\text{loc}} \cdot \text{ldx} + m_{\text{loc}} \cdot \text{ldy}) \cdot \text{sizeof}(\text{scalar_t}) \quad (5.3)$$

Poiché nel problema considerato k è piccolo rispetto alle dimensioni della matrice, il termine associato ad A è normalmente dominante.

Aumentare il numero di processi MPI e scegliere opportunamente la griglia bidimensionale riduce le dimensioni dei blocchi locali e, di conseguenza, la quantità di memoria device richiesta da ciascun processo.

5.4 `cuda_naive`

La prima implementazione CUDA adotta la mappatura più diretta possibile: a ciascun thread viene assegnato un singolo elemento del risultato locale Y .

Indicando con t l'identificativo globale del thread, gli indici dell'elemento da calcolare sono

$$i = \left\lfloor \frac{t}{k} \right\rfloor, \quad c = t \bmod k. \quad (5.4)$$

Il thread t è quindi responsabile di Y_{ic} e ne calcola il valore percorrendo interamente la dimensione interna del prodotto:

$$Y_{ic} = \sum_{j=0}^{n_{\text{loc}}-1} A_{ij} X_{jc}. \quad (5.5)$$

Ogni thread mantiene un unico accumulatore e non comunica con gli altri thread.

Il kernel costituisce per questo una baseline semplice, utile sia per verificare il corretto funzionamento dell'intera pipeline CUDA, sia come riferimento interno rispetto al quale valutare le ottimizzazioni successive.

La semplicità della mappatura comporta però una prima inefficienza fondamentale. Infatti, per una stessa riga i , gli elementi:

$$Y_{i0}, Y_{i1}, \dots, Y_{i,k-1}$$

vengono calcolati da thread distinti. Durante una stessa iterazione j , tutti questi thread hanno bisogno dello stesso coefficiente A_{ij} , ma ciascuno lo utilizza soltanto per la propria colonna di Y . Il kernel non esprime quindi alcun riuso di A_{ij} all'interno del thread: lo stesso elemento di A compare logicamente in k operazioni di caricamento differenti.

Questo è il principale limite che motiva il backend successivo: in `cuda_warp`, un valore di A viene invece caricato una volta da una lane e riutilizzato per aggiornare tutti i k risultati associati alla stessa riga.

La mappatura del kernel assegna thread consecutivi alle colonne consecutive di Y . Per un j fissato, tutte le lane che lavorano sulla stessa riga i di Y richiedono quindi lo stesso elemento A_{ij} . Le lane associate alla riga successiva richiedono invece $A_{i+1,j}$, che in memoria row-major si trova a distanza `lda` elementi da A_{ij} .

Per $k < 32$, cioè quando una riga di Y contiene meno elementi della dimensione di un warp, un singolo warp comprende lane associate a più righe di Y e coinvolge quindi più gruppi di accessi ad A distanziati tra loro.

Gli accessi a X hanno invece un comportamento differente. Per ogni gruppo di lane associato alla stessa riga di Y , thread consecutivi leggono gli elementi consecutivi

$$X_{j0}, X_{j1}, \dots, X_{j,k-1},$$

che sono contigui in memoria nel layout row-major. Se il warp comprende più righe di Y , questa stessa sequenza di accessi a X si ripete.

All'aumentare di k , un numero crescente di lane del warp lavora sulla stessa riga di Y , fino al caso $k = 32$, in cui un intero warp corrisponde esattamente a una riga. A ogni iterazione j , tutte le lane richiedono quindi lo stesso elemento A_{ij} e, contemporaneamente, leggono i 32 elementi consecutivi

$$X_{j0}, \dots, X_{j31}.$$

Il parametro k influenza quindi direttamente il pattern di accesso del kernel. Questo aspetto sarà utile nel confronto con `cuda_warp`, dove la mappatura viene rovesciata: un intero warp viene assegnato a una sola riga di Y e le lane vengono distribuite lungo la dimensione j .

5.5 `cuda_warp`: un warp per riga

Il secondo backend modifica radicalmente l'approccio alla parallelizzazione. In `cuda_naive` ogni thread era responsabile di un singolo elemento Y_{ic} , ma in `cuda_warp`, invece, un intero warp collabora ed è responsabile del calcolo di una riga di Y .

Fissata la riga i , le 32 lane del warp si dividono la dimensione interna j . La lane l visita gli indici:

$$j = l, l + 32, l + 64, \dots$$

fino a raggiungere n_{loc} .

A ogni iterazione, lane consecutive accedono quindi a elementi consecutivi della stessa riga della matrice A :

$$A_{i,j}, A_{i,j+1}, \dots, A_{i,j+31}.$$

La mappatura produce così un pattern di accesso contiguo, favorevole alla coalescenza delle letture dalla memoria globale. A differenza di quanto accadeva in `cuda_naive`, questo comportamento non dipende da k , perché è la dimensione j , e non quella delle colonne del multivettore, a essere distribuita fra le lane.

La nuova mappatura consente soprattutto di sfruttare esplicitamente il riuso degli elementi di A . Una volta caricato un elemento:

$$a = A_{ij},$$

la lane non lo utilizza per una sola colonna del risultato, ma per aggiornare tutti i k accumulatori associati alla riga di Y :

$$\begin{aligned} acc[0] &+= aX_{j0}, \\ acc[1] &+= aX_{j1}, \\ &\vdots \\ acc[k-1] &+= aX_{j,k-1}. \end{aligned}$$

Un singolo caricamento di A_{ij} viene quindi riutilizzato nelle k moltiplicazioni che coinvolgono lo stesso elemento della matrice, ed è questa la principale differenza rispetto al primo approccio.

Al termine del ciclo su j , ciascuna lane possiede però soltanto una parte del risultato. Per ogni colonna c , la lane l ha accumulato i termini corrispondenti agli indici:

$$j = l, l + 32, l + 64, \dots$$

e il valore completo può essere scritto come

$$Y_{ic} = \sum_{l=0}^{31} acc_l[c]. \quad (5.6)$$

I 32 contributi vengono quindi ridotti all'interno del warp mediante `__shfl_down_sync`. Al termine, la lane 0 possiede i k valori completi della riga e li scrive in Y .

5.5.1 Gestione di k

I valori di k richiesti dalla traccia, 3, 6, 8, 20 e 32, sono implementati mediante cinque istanze template distinte del kernel. In queste versioni k è quindi noto a compile time: la dimensione dell'array degli accumulatori è costante e i cicli sulle colonne possono essere srotolati dal compilatore.

La stessa interfaccia supporta comunque valori arbitrari di k . Quando k non appartiene all'insieme precedente viene utilizzato un kernel runtime che elabora quattro colonne alla volta (`RUNTIME_TILE = 4`). In questo modo il numero di accumulatori simultaneamente mantenuti da ciascuna lane rimane limitato, pur continuando a riutilizzare ogni elemento di A per più colonne.

5.5.2 La maschera dello shuffle

Un dettaglio di correttezza riguarda la maschera utilizzata nelle operazioni di shuffle. Il kernel usa la maschera completa `0xffffffff`, perché ogni warp valido porta tutte le 32 lane fino alla riduzione. Il controllo sul limite delle righe viene infatti eseguito uniformemente sull'intero warp: se la riga assegnata non esiste, tutte le lane terminano prima; altrimenti tutte raggiungono la `__shfl_down_sync`.

Questo rimane vero anche quando $n_{loc} < 32$. In questo caso alcune lane possono non avere alcun indice j da elaborare, ma non terminano l'esecuzione: i loro accumulatori, inizializzati a zero, rimangono nulli e partecipano comunque alla riduzione con contributo nullo, senza alterare il risultato finale.

5.5.3 Il problema residuo: gli accessi a X

La distribuzione di j fra le lane rende favorevoli gli accessi ad A , ma determina contemporaneamente un pattern meno favorevole per X quando il multivettore è memorizzato in row-major.

Consideriamo un'iterazione nella quale le 32 lane del warp stanno elaborando gli indici consecutivi

$$j_0, j_0 + 1, \dots, j_0 + 31.$$

Durante l'aggiornamento di una stessa colonna c , le lane leggono quindi

$$X_{j_0,c}, X_{j_0+1,c}, \dots, X_{j_0+31,c}.$$

Si tratta di elementi appartenenti alla stessa colonna di X , ma a 32 righe differenti. Poiché X è memorizzata in row-major, due elementi consecutivi di questa sequenza non sono adiacenti in memoria: i loro indirizzi distano `ldx` elementi, cioè una quantità di byte pari a:

$$\text{ldx} \cdot \text{sizeof}(\text{scalar_t})$$

dove ricordiamo che nel caso compatto, utilizzato dal progetto, si ha sempre $\text{ldx} = k$.

Il comportamento è quindi opposto a quello delle letture di A : per la matrice A , lane consecutive leggono elementi consecutivi della stessa riga, mentre per X durante il calcolo di una colonna c , lane consecutive leggono elementi della stessa colonna ma appartenenti a righe differenti.

All'aumentare di k aumenta anche la distanza in memoria fra gli elementi di X richiesti da lane consecutive. Le richieste del warp risultano quindi progressivamente più sparse e possono richiedere un numero maggiore di transazioni di memoria rispetto a un accesso contiguo.

Il kernel ha quindi risolto il principale limite di `cuda_naive`, cioè il mancato riuso esplicito di A , ma ha lasciato un nuovo collo di bottiglia nel pattern di accesso a X . Le due varianti introdotte nelle sezioni successive affrontano esattamente questo problema senza modificare la mappatura warp-per-row.

5.6 Primo rimedio: cambio di layout di X

Il problema descritto nella sezione precedente nasce dall'incompatibilità fra la mappatura delle lane e il layout row-major di X . Le lane di `cuda_warp` avanzano lungo la dimensione j , mentre nel layout row-major sono gli elementi che differiscono per l'indice di colonna c a essere contigui in memoria.

Il primo rimedio consiste quindi nel modificare la rappresentazione di X senza cambiare la mappatura del kernel. Durante il preprocessing, il blocco locale del multivettore viene riscritto in layout *column-major*, in modo che

$$X_{jc} = \mathbf{x_loc}[c \cdot n_{\text{loc}} + j]. \quad (5.7)$$

In questa rappresentazione, tutti gli elementi appartenenti a una stessa colonna di X sono consecutivi in memoria. Di conseguenza, quando le 32 lane del warp elaborano gli indici

$$j_0, j_0 + 1, \dots, j_0 + 31$$

per una stessa colonna c , leggono anche 32 posizioni consecutive del buffer:

$$X_{j_0,c}, X_{j_0+1,c}, \dots, X_{j_0+31,c}.$$

Lo stride fra gli indirizzi richiesti da lane consecutive passa quindi da k elementi (nel layout row-major) a un singolo elemento (in questo layout). La mappatura warp-per-row e il riuso di A introdotti da `cuda_warp` rimangono invariati: cambia esclusivamente il modo in cui X è organizzata e indirizzata in memoria.

5.7 `cuda_warp_smem`: staging di X in shared memory

Il secondo rimedio conserva sia la mappatura warp-per-row di `cuda_warp` sia il normale layout row-major di X . La differenza riguarda il percorso seguito dagli elementi del multivettore: invece di essere letti direttamente dalla global memory durante il calcolo, vengono prima

caricati cooperativamente in **shared memory** e successivamente consumati dai warp del blocco.

Il percorso di un elemento di X diventa quindi:

$$\text{global memory} \longrightarrow \text{shared memory} \longrightarrow \text{registri},$$

mentre in `cuda_warp` il passaggio intermedio non era presente.

La struttura del calcolo rimane per il resto invariata. Ogni warp è ancora responsabile di una riga di Y , le sue 32 lane si dividono gli indici j , ogni elemento di A viene riutilizzato per aggiornare i k accumulatori e la riduzione finale avviene mediante `__shfl_down_sync`.

5.7.1 Caricamento cooperativo di X

Il caricamento del tile di X non utilizza la stessa mappatura con cui viene successivamente eseguito il prodotto. In questa prima fase partecipano tutti i thread del blocco e gli elementi del tile vengono assegnati mediante un indice lineare.

Indicando con q tale indice, vengono ricavati

$$r = \left\lfloor \frac{q}{k} \right\rfloor, \quad c = q \bmod k,$$

e il thread carica l'elemento X_{rc} nella corrispondente posizione del tile in shared memory.

Con X compatta in row-major, thread consecutivi ricevono valori consecutivi di q e leggono quindi posizioni consecutive del buffer. Il caricamento dalla global memory segue così l'ordine naturale con cui X è memorizzata, evitando il pattern strided che caratterizzava `cuda_warp`.

Una volta completato il caricamento cooperativo, una `__syncthreads()` garantisce che il tile sia interamente disponibile prima che i warp inizino a utilizzarlo. La shared memory è infatti una risorsa comune a tutti i thread dello stesso blocco.

A questo punto il calcolo riprende la stessa struttura di `cuda_warp`. Per ogni warp, la lane l elabora gli indici

$$j = l, l + 32, l + 64, \dots$$

ma gli elementi di X corrispondenti vengono ora letti dal tile in shared memory anziché direttamente dalla global memory.

5.7.2 Riutilizzo del tile fra i warp del blocco

L'introduzione della shared memory produce inoltre un secondo effetto. I warp appartenenti allo stesso blocco calcolano righe differenti di Y , e utilizzano quindi righe differenti di A , ma tutti devono attraversare la stessa matrice X .

Nel backend `cuda_warp`, ciascun warp emette le proprie letture di X . In `cuda_warp_smem`, invece, un tile viene caricato una sola volta dall'intero blocco e viene poi utilizzato da tutti i suoi warp.

Se W indica il numero di warp per blocco, il numero di elementi di X caricati esplicitamente dalla global memory passa quindi, idealmente, da

$$m_{\text{loc}} n_{\text{loc}} k$$

a

$$\left\lceil \frac{m_{\text{loc}}}{W} \right\rceil n_{\text{loc}} k.$$

Per esempio, con il valore predefinito `BLOCK_THREADS=256`, il blocco contiene $W = 8$ warp.

Questa riduzione descrive il numero di letture esplicitamente richieste dal kernel alla gerarchia di memoria globale. Non implica però che il traffico effettivo verso la DRAM si riduca dello stesso fattore, perché una parte delle letture di `cuda_warp` può già essere servita dalle cache della GPU. Il vantaggio effettivo della shared memory deve quindi essere determinato sperimentalmente.

5.7.3 Bank conflict e padding della shared memory

Una volta caricato il tile, il problema non è più la coalescenza degli accessi alla global memory, ma il modo in cui gli indirizzi richiesti dalle lane vengono distribuiti sui banchi della shared memory.

La shared memory è organizzata in 32 banchi, e parole consecutive da 32 bit vengono assegnate a banchi consecutivi. Se più lane richiedono indirizzi differenti appartenenti allo stesso banco, gli accessi non possono essere serviti simultaneamente e vengono parzialmente serializzati.

Durante il caricamento cooperativo del tile, thread consecutivi scrivono elementi consecutivi della shared memory. Questo pattern segue linearmente il buffer e non introduce il problema che interessa il successivo consumo del tile.

Durante il calcolo, invece, la lane l accede alla posizione

$$X_{\text{tile}}[l \cdot \text{tile_row_stride} + c],$$

dove

$$\text{tile_row_stride} = k + \text{SMEM_PAD}.$$

Lane consecutive leggono quindi elementi separati da un'intera riga del tile. Se questo stride produce una mappatura periodica sugli stessi banchi, si genera un bank conflict.

Il caso $k = 32$ in doppia precisione è particolarmente sfavorevole senza padding. Con

$$\text{tile_row_stride} = 32,$$

la distanza fra gli elementi letti da lane consecutive corrisponde a 64 parole da 32 bit e quindi

$$64 \equiv 0 \pmod{32}.$$

Gli accessi ricadono così ripetutamente sugli stessi banchi.

Aggiungendo un elemento di padding per riga si ottiene invece

```
tile_row_stride = 33,
```

e la distanza diventa

$$66 \equiv 2 \pmod{32},$$

distribuendo gli accessi sui diversi banchi e rimuovendo il caso patologico precedente.

Il padding non rappresenta tuttavia una scelta universalmente ottimale per ogni k . Il suo effetto dipende dallo stride risultante. Per questo `SMEM_PAD` è un parametro di compilazione e il progetto mantiene separate le configurazioni con e senza padding, così da misurarne direttamente l'effetto.

5.7.4 Scelta della dimensione del tile

Il blocco locale di X non viene trasferito interamente in shared memory. La dimensione della shared memory disponibile per blocco è limitata e contribuisce, insieme ai registri e al numero di thread, a determinare quanti blocchi possono risiedere contemporaneamente su uno SM.

Il backend divide quindi X in tile contenenti `x_rows_per_tile` righe. Indicando questa quantità con R , la memoria richiesta da un tile è

$$R(k + \text{SMEM_PAD}) \text{sizeof}(\text{scalar_t}).$$

Il valore di R modifica la quantità di shared memory occupata da ogni blocco, ma non il volume complessivo di X che quel blocco deve attraversare. Un tile più piccolo produce semplicemente un numero maggiore di iterazioni del ciclo di tiling.

La dimensione del tile influenza invece direttamente la frequenza delle sincronizzazioni. Ogni tile richiede due chiamate a `__syncthreads()`: la prima impedisce che il tile precedente venga sovrascritto mentre qualche warp lo sta ancora utilizzando, la seconda garantisce che il nuovo tile sia stato completamente caricato prima di iniziare il calcolo. Aumentando il numero di righe per tile diminuisce quindi il numero di barriere incontrate durante l'attraversamento di X .

Non è tuttavia possibile aumentare arbitrariamente il tile. Una maggiore quantità di shared memory per blocco può ridurre il numero di blocchi residenti contemporaneamente su uno SM e quindi l'occupancy. La scelta implementata nel backend cerca perciò un compromesso: utilizzare un tile sufficientemente grande senza fare della shared memory un ulteriore vincolo alla residenza dei blocchi.

La funzione `plan_tile` esegue questa scelta una sola volta durante `local_gemm_create`, quindi fuori dalla regione cronometrata.

Come primo passo viene interrogata `cudaOccupancyMaxActiveBlocksPerMultiprocessor` assumendo nulla la shared memory dinamica. Il risultato rappresenta il numero di blocchi per SM consentito dal kernel compilato in assenza del vincolo aggiuntivo introdotto dal tile.

La shared memory disponibile per SM viene quindi suddivisa rispetto a questo numero di blocchi, ottenendo un budget iniziale per ciascun blocco. Da tale budget viene ricavato il numero di righe che possono essere contenute nel tile.

Il valore così ottenuto viene infine verificato interrogando nuovamente l'API di occupancy con la quantità effettiva di shared memory richiesta. Se il numero di blocchi residenti risulta inferiore all'obiettivo iniziale, la dimensione del tile viene progressivamente ridotta.

La pianificazione viene eseguita sull'istanza concreta del kernel specializzato per il valore di k corrente. Questa distinzione è necessaria perché le diverse istanze possono avere richieste differenti di registri e quindi differenti limiti di occupancy.

Il parametro `TILE_GRANULARITY` controlla infine l'arrotondamento del numero di righe scelto dal piano. Il valore predefinito è 32, pari alla dimensione del warp.

Un tile con un numero di righe multiplo di 32 permette a tutte le lane di partecipare a ciascun passo completo del ciclo su j . Questo non è però un requisito di correttezza: il kernel gestisce anche tile con un numero di righe non multiplo di 32.

L'arrotondamento introduce quindi un compromesso. Una granularità elevata può lasciare inutilizzata una parte del budget di shared memory e aumentare il numero di tile, mentre una granularità più fine può produrre un ultimo passo nel quale alcune lane non hanno elementi da elaborare. Il parametro viene mantenuto configurabile proprio per poter misurare questo compromesso.

5.7.5 Sincronizzazione e warp inattivi

L'uso di `__syncthreads()` introduce un vincolo di correttezza che non era presente in `cuda_warp`. La barriera coinvolge tutti i thread del blocco e può essere superata soltanto quando tutti i thread che partecipano al blocco l'hanno raggiunta.

L'ultimo blocco della griglia può contenere più warp di quanti siano necessari per coprire le ultime righe di Y . In `cuda_warp` questi warp possono terminare immediatamente quando la riga assegnata supera m_{loc} .

In `cuda_warp_smem` questo non è possibile prima del completamento del ciclo sui tile. Se alcuni warp uscissero mentre gli altri continuano a eseguire le `__syncthreads()`, i thread rimanenti potrebbero attendere indefinitamente a una barriera che gli altri non raggiungeranno mai.

Per questo il kernel associa a ciascun warp un flag `active`. Anche i warp ai quali non corrisponde una riga valida partecipano al caricamento cooperativo e attraversano tutte le sincronizzazioni del blocco. Soltanto l'accumulo dei prodotti e la successiva scrittura di Y vengono eseguiti dai warp attivi.

Terminato l'ultimo tile, quando non rimangono più barriere di blocco, i warp inattivi possono infine uscire prima della riduzione finale.

5.8 Due rimedi allo stesso problema

Le soluzioni descritte nelle sezioni 5.6 e 5.7 nascono dallo stesso problema: in `cuda_warp`, con X row-major, lane consecutive leggono elementi della stessa colonna di X separati da uno stride pari a k elementi.

I due approcci intervengono però in punti differenti.

La variante `X_LAYOUT=column` modifica la rappresentazione del dato prima dell'esecuzione del kernel. Disponendo consecutivamente gli elementi di una stessa colonna di X , la mappatura warp-per-row può essere mantenuta inalterata e le lane trovano direttamente in global memory gli elementi richiesti in posizioni consecutive. Il costo associato a questa soluzione è la conversione preliminare del multivettore.

`cuda_warp_smem`, invece, conserva X nel normale layout row-major e modifica il percorso attraverso il quale i dati raggiungono le lane. Gli elementi vengono prima caricati cooperativamente e in modo contiguo dalla global memory alla shared memory, quindi letti dal tile durante il calcolo. Questa soluzione evita la conversione del dato, ma introduce il costo dello staging, delle sincronizzazioni e dell'utilizzo della shared memory.

Le due implementazioni rappresentano quindi due strategie alternative studiate separatamente nel progetto: la prima modifica il layout del dato, la seconda modifica il meccanismo di accesso mantenendone invariata la rappresentazione.

La combinazione dei due approcci non è stata inclusa tra i backend sperimentali, poiché l'obiettivo del confronto è isolare separatamente l'effetto delle due strategie rispetto alla stessa baseline `cuda_warp`.

5.9 cublas: il riferimento esterno

L'ultimo backend GPU non introduce un nuovo kernel sviluppato nel progetto, ma utilizza la routine GEMM della libreria cuBLAS. Il suo ruolo è fornire un riferimento esterno con cui confrontare le implementazioni CUDA precedenti.

Perché il confronto sia significativo, cuBLAS viene inserita dietro la stessa interfaccia degli altri backend. La creazione del contesto, le allocazioni device e il trasferimento iniziale di A avvengono quindi in `local_gemm_create`, mentre ogni invocazione di `local_gemm` esegue, nello stesso ordine degli altri backend GPU, il trasferimento H2D di X , il prodotto sul device e il trasferimento D2H di Y .

Anche la misura segue gli stessi confini: il tempo del solo prodotto cuBLAS viene delimitato mediante `cudaEvent`, mentre i trasferimenti vengono misurati separatamente. Il confronto con i kernel sviluppati nel progetto avviene quindi nelle stesse condizioni sperimentali.

5.9.1 Adattamento fra row-major e column-major

Il progetto memorizza le matrici in row-major, mentre l'interfaccia GEMM tradizionale di cuBLAS interpreta i propri operandi in column-major. Non è tuttavia necessario convertire o trasporre fisicamente i dati.

Cioè gli stessi dati che rappresentano una matrice in row-major, se interpretati come column-major senza modificarne il contenuto in memoria, rappresentano la sua trasposta. Cambia soltanto la convenzione con cui gli indici vengono associati agli indirizzi.

Di conseguenza, i buffer del progetto possono essere reinterpretati da cuBLAS come

$$A_{\text{cm}} = A_{\text{rm}}^T, \quad X_{\text{cm}} = X_{\text{rm}}^T, \quad Y_{\text{cm}} = Y_{\text{rm}}^T.$$

Applicando questa reinterpretazione al prodotto desiderato si ottiene:

$$Y_{\text{cm}} = Y_{\text{rm}}^T = (A_{\text{rm}} X_{\text{rm}})^T = X_{\text{rm}}^T A_{\text{rm}}^T = X_{\text{cm}} A_{\text{cm}}.$$

Quindi la chiamata cuBLAS deve utilizzare X come primo operando e A come secondo. Nella convenzione utilizzata internamente da cuBLAS si ha:

$$C_{M \times N} = A_{M \times K} B_{K \times N},$$

con:

$$C_{M \times N} = Y_{\text{cm}}^T, \quad A_{M \times K} = X_{\text{cm}}^T, \quad B_{K \times N} = A_{\text{cm}}^T$$

e le dimensioni passate alla libreria sono:

$$M = k, \quad N = m_{\text{loc}}, \quad K = n_{\text{loc}}.$$

Entrambi gli operandi vengono utilizzati con `CUBLAS_OP_N`: la trasposizione non viene eseguita dalla libreria, ma è già implicita nella reinterpretazione column-major degli stessi buffer.

5.9.2 Significato del confronto

Il backend cuBLAS costituisce un riferimento esterno e non un ulteriore passo della sequenza di ottimizzazione sviluppata nel progetto. La libreria sceglie internamente l'implementazione del prodotto e il progetto non controlla direttamente parametri quali tiling, distribuzione del lavoro od occupancy.

Il confronto viene quindi utilizzato in senso sperimentale: permette di stabilire quale prestazione ottengano i kernel specializzati del progetto rispetto a una routine GEMM di libreria eseguita sulla stessa GPU e con gli stessi dati.

Nel problema considerato una delle dimensioni del prodotto, k , è intenzionalmente piccola rispetto a m e n . Questo rende il confronto particolarmente interessante, perché i kernel sviluppati nel progetto sono costruiti esplicitamente attorno alla struttura di un multivettore stretto, mentre cuBLAS riceve il problema attraverso la propria interfaccia GEMM generale. L'eventuale vantaggio di una delle due soluzioni deve però essere stabilito dalle misure, non assunto a priori.

5.10 Più rank MPI sulla stessa GPU

La piattaforma utilizzata per le misure dispone di una sola GPU e i backend CUDA del progetto selezionano esplicitamente il device 0. Quando l'applicazione viene eseguita con più rank MPI sullo stesso nodo, ogni processo crea quindi il proprio contesto CUDA sulla medesima scheda.

In assenza di NVIDIA Multi-Process Service (MPS), il lavoro proveniente da contesti CUDA appartenenti a processi differenti non viene eseguito contemporaneamente sugli SM, ma viene alternato mediante time slicing. I rank possono quindi procedere contemporaneamente sul lato CPU, ma competono per l'esecuzione sulla stessa GPU.

Questa situazione modifica il significato delle misure ottenute mediante `cudaEvent`. Per ciascun rank, t_{kernel} misura l'intervallo compreso fra gli eventi che delimitano il proprio kernel. Se il contesto viene sospeso durante tale intervallo per consentire l'esecuzione di un altro rank, il tempo osservato risente anche della contesa per la GPU.

Di conseguenza, con più rank per GPU, t_{kernel} non deve essere interpretato come il tempo che lo stesso kernel otterrebbe disponendo della scheda in modo esclusivo.

Il driver conserva due prospettive differenti sulla fase locale.

La prima è il tempo del kernel restituito dal backend CUDA. Al termine delle ripetizioni, come per le altre fasi, viene considerato il massimo fra i rank, poiché l'operazione distribuita non può essere considerata conclusa prima del processo più lento. Questa misura rimane utile

per osservare il comportamento dei singoli backend, ma con più contesti sulla stessa GPU è influenzata dalla loro contesa.

Per misurare invece il tempo effettivamente richiesto dall'intero gruppo di rank per completare la fase locale, il benchmark mette a disposizione la modalità `-group-timing`. In questo caso una barriera MPI viene posta immediatamente prima di `local_gemm` e una seconda subito dopo:

$$\text{barriera} \longrightarrow \text{fase locale dei rank} \longrightarrow \text{barriera}.$$

L'intervallo fra le due barriere misura quindi il tempo trascorso dal momento in cui tutti i rank sono pronti fino al completamento dell'ultimo. È questo il denominatore utilizzato per la metrica `gflops_local_group`, che descrive il throughput complessivo del gruppo durante la fase locale.

Per le campagne dedicate alla caratterizzazione del kernel viene quindi utilizzato un solo rank MPI. In questo modo il backend dispone esclusivamente della GPU e differenze dovute, per esempio, alla dimensione del blocco CUDA, al padding della shared memory o alla scelta del layout di X non vengono confuse con la schedulazione fra contesti differenti.

A parità di dimensioni globali M , N e k , il numero complessivo di operazioni rimane lo stesso, ma con un solo rank cambia naturalmente la dimensione del blocco locale assegnato al processo. Le misure a rank singolo devono quindi essere interpretate come caratterizzazione del backend sulla GPU dedicata, mentre quelle multi-rank descrivono l'esecuzione della configurazione MPI effettiva sulla piattaforma disponibile.

Capitolo 6

Metodologia sperimentale

6.1 Ambiente sperimentale

Tutte le misure riportate nella relazione sono state eseguite sul *server di calcolo del Dipartimento*.

L'ambiente software viene predisposto tramite il sistema di moduli del server, utilizzando versioni fissate della toolchain GNU, di Open MPI e di CUDA:

```
module purge
module load gnu/13.3.0
module load openmpi/5.0.9
module load cuda/12.8
```

Tutte le campagne vengono quindi compilate ed eseguite con la stessa toolchain software.

Le implementazioni CPU sono compilate con ottimizzazione `-O3` e con ottimizzazioni specifiche per l'architettura host tramite `-march=native`. È inoltre abilitata la contrazione delle operazioni floating-point mediante `-ffp-contract=fast`.

I backend CUDA sono compilati con `-O3` e per l'architettura `sm_75`, corrispondente alla Compute Capability 7.5 della GPU utilizzata. La precisione numerica viene selezionata a tempo di compilazione.

Nelle esecuzioni MPI con più processi, i rank vengono associati esplicitamente ai core mediante:

```
--bind-to core --map-by core
```

in modo da mantenere stabile il placement dei processi durante l'esperimento ed evitare che la migrazione dei rank fra core introduca un'ulteriore sorgente di variabilità nelle misure.

Il nodo utilizzato dispone di una sola GPU e tutti i backend CUDA selezionano il device 0. Quando più rank MPI utilizzano contemporaneamente un backend CUDA, essi condividono quindi lo stesso acceleratore. Nel contesto sperimentale adottato non viene utilizzato NVIDIA MPS, per cui l'esecuzione multi-rank sulla GPU non rappresenta una configurazione con acceleratori indipendenti.

Per questo motivo le campagne dedicate alla caratterizzazione dei kernel CUDA vengono eseguite con un solo rank MPI. In questo modo le differenze osservate fra configurazioni CUDA non vengono confuse con la contesa fra più processi sulla stessa scheda. Le conseguenze dell'utilizzo di più contesti CUDA sullo stesso device sono discusse nella sezione 5.10.

Per ogni esecuzione il runner salva inoltre informazioni sufficienti a ricostruire la configurazione con cui sono state prodotte le misure, fra cui il comando eseguito, il file di configurazione, i principali parametri sperimentali, il commit Git, lo stato del working tree e le versioni della toolchain utilizzata. I risultati possono così essere ricondotti alla specifica versione del codice e dell'ambiente software che li ha generati.

6.2 Protocollo di misura

Ogni campione temporale corrisponde a una singola invocazione del prodotto distribuito. Prima di ciascuna repetition misurata viene eseguita una `MPI_Barrier` sul comunicatore dell'intera griglia, in modo che tutti i processi abbiano completato la repetition precedente prima di iniziare quella successiva.

La barriera precede l'avvio del cronometro e non contribuisce quindi ai tempi riportati.

Una repetition segue quindi la sequenza

$$\text{MPI_Bcast}(X) \longrightarrow \text{local_gemm} \longrightarrow \text{MPI_Reduce}(Y). \quad (6.1)$$

Il broadcast viene eseguito sul comunicatore di colonna della griglia MPI e rende disponibile a ciascun processo il blocco di X necessario al prodotto locale. Ogni rank calcola quindi, mediante `local_gemm`, il proprio contributo parziale a Y . I contributi appartenenti alla stessa banda di righe vengono infine sommati mediante una `MPI_Reduce` sul corrispondente comunicatore di riga.

La temporizzazione delle tre fasi viene effettuata localmente da ciascun rank mediante `MPI_Wtime`. Indicando con

$$t_0 : \quad \text{inizio del broadcast}, \quad (6.2)$$

$$t_1 : \quad \text{fine del broadcast}, \quad (6.3)$$

$$t_2 : \quad \text{fine della fase locale}, \quad (6.4)$$

$$t_3 : \quad \text{fine della reduce}, \quad (6.5)$$

si definiscono

$$t_{\text{bcast}} = t_1 - t_0, \quad (6.6)$$

$$t_{\text{local}} = t_2 - t_1, \quad (6.7)$$

$$t_{\text{reduce}} = t_3 - t_2, \quad (6.8)$$

$$t_{\text{total}} = t_3 - t_0. \quad (6.9)$$

Il tempo t_{total} è quindi una misura wall-clock diretta dell'intera invocazione distribuita osservata dal singolo rank, dall'inizio del broadcast fino al completamento della reduce. Le operazioni di preparazione dei dati e di inizializzazione del backend sono già terminate prima dell'inizio delle repetition e non appartengono a questo intervallo.

6.2.1 Temporizzazione della fase locale su GPU

Il significato di t_{local} dipende dal backend utilizzato. Nei backend CPU esso misura direttamente l'intervallo trascorso all'interno del nucleo locale. Nei backend CUDA, invece, `local_gemm` comprende l'intero percorso necessario a utilizzare l'acceleratore durante una singola invocazione:

$$X_{\text{host}} \longrightarrow X_{\text{device}} \longrightarrow \text{kernel CUDA} \longrightarrow Y_{\text{device}} \longrightarrow Y_{\text{host}}.$$

Di conseguenza t_{local} è il tempo wall-clock della fase locale completa e comprende sia il calcolo sul device sia i trasferimenti necessari alla singola invocazione.

Per poter distinguere questi contributi, i backend CUDA utilizzano inoltre gli eventi CUDA per misurare separatamente il trasferimento H2D di X , il tempo del kernel e il trasferimento D2H di Y . Tali misure sono interne alla fase delimitata da t_1 e t_2 e vengono registrate come metriche distinte.

Il trasferimento iniziale di A non appartiene invece alla repetition: A_{loc} è invariata fra le diverse invocazioni e viene trasferita sul device durante l'inizializzazione del backend.

6.3 Metriche di prestazione

Il protocollo descritto nella sezione precedente produce, per ciascuna repetition e per ciascun rank MPI, i tempi delle tre fasi che compongono il prodotto distribuito. Queste misure devono essere combinate in una singola quantità da utilizzare al denominatore della metrica in GFLOP/s.

La traccia richiede che la misura includa il tempo necessario al prodotto matrice-multivettore, escludendo preprocessing, I/O e, nel caso CUDA, i trasferimenti fra memoria host e device. Le comunicazioni MPI necessarie all'esecuzione distribuita rimangono invece parte del prodotto.

Indicando con i la repetition e con p il rank MPI, viene quindi definito un tempo di prestazione $T_{i,p}$ associato al singolo processo.

Per i backend CPU la fase locale coincide con l'esecuzione del nucleo di calcolo. Si utilizza quindi

$$T_{i,p}^{\text{CPU}} = t_{\text{bcast},i,p} + t_{\text{local},i,p} + t_{\text{reduce},i,p}. \quad (6.10)$$

Poiché i tre intervalli sono consecutivi, questa quantità coincide con il tempo wall-clock

$$t_{\text{total},i,p} = t_{3,i,p} - t_{0,i,p}.$$

Per i backend CUDA, invece, il tempo wall-clock della fase locale contiene anche i trasferimenti H2D di X e D2H di Y . Poiché tali trasferimenti sono esclusi dalla metrica richiesta dalla traccia, il contributo della fase locale viene sostituito dal tempo del solo kernel:

$$T_{i,p}^{\text{CUDA}} = t_{\text{bcast},i,p} + t_{\text{kernel},i,p} + t_{\text{reduce},i,p}. \quad (6.11)$$

$T_{i,p}^{\text{CUDA}}$ non coincide quindi con il tempo wall-clock completo della repetition. È la quantità utilizzata per valutare il prodotto distribuito dopo aver escluso i trasferimenti host-device richiesti dalla pipeline CUDA.

Per ogni repetition, i tempi vengono prima composti separatamente per ciascun rank e soltanto successivamente aggregati fra i processi:

$$T_i = \max_{0 \leq p < P} T_{i,p}, \quad (6.12)$$

dove P è il numero complessivo di rank MPI.

L'operazione di massimo riflette il fatto che l'esecuzione distribuita non può essere considerata completata prima del processo più lento. È importante che la composizione delle fasi preceda tale operazione. Nel caso CUDA viene quindi calcolato

$$\max_p (t_{\text{bcast},p} + t_{\text{kernel},p} + t_{\text{reduce},p}),$$

e non

$$\max_p t_{\text{bcast},p} + \max_p t_{\text{kernel},p} + \max_p t_{\text{reduce},p}.$$

Quest'ultima espressione potrebbe combinare i massimi di fasi osservati su processi differenti e costruirebbe quindi un tempo che non corrisponde al percorso di alcun rank.

Dalle R repetition misurate si ottiene infine il tempo medio

$$\bar{T} = \frac{1}{R} \sum_{i=1}^R T_i. \quad (6.13)$$

Utilizzando il conteggio del lavoro definito in eq. (2.3), la prestazione complessiva è

$$P_{\text{GFLOP/s}} = \frac{2MNk}{\bar{T}} \cdot 10^{-9}. \quad (6.14)$$

M e N sono sempre le dimensioni della matrice globale. Il numeratore rappresenta quindi lo stesso lavoro matematico indipendentemente dal numero di processi utilizzati per eseguirlo.

6.3.1 Prestazione della sola fase computazionale

Accanto alla metrica complessiva viene mantenuta una misura che esclude le comunicazioni MPI e considera soltanto la fase computazionale del backend.

Per ogni repetition e rank si definisce

$$C_{i,p} = \begin{cases} t_{\text{local},i,p}, & \text{backend CPU,} \\ t_{\text{kernel},i,p}, & \text{backend CUDA.} \end{cases} \quad (6.15)$$

La stessa regola di aggregazione utilizzata per il tempo complessivo viene applicata anche a questa quantità:

$$C_i = \max_p C_{i,p}, \quad \bar{C} = \frac{1}{R} \sum_{i=1}^R C_i.$$

La prestazione della sola fase computazionale è quindi

$$P_{\text{compute}} = \frac{2MNk}{C} \cdot 10^{-9}. \quad (6.16)$$

Nel file dei risultati questa quantità viene riportata come `gflops_compute` e per i backend CUDA il valore basato sul solo kernel è riportato anche come `gflops_kernel`.

Le due metriche rispondono a domande differenti:

- La metrica complessiva **gflops** caratterizza l'esecuzione del prodotto distribuito, includendo il costo delle comunicazioni MPI ma rispettando l'esclusione dei trasferimenti CUDA richiesta dalla traccia.
- La metrica `gflops_compute` isola invece il backend locale.

Nelle campagne dedicate al tuning dei kernel CUDA, eseguite con un solo rank MPI, il confronto viene quindi effettuato principalmente mediante `gflops_kernel`, così da misurare direttamente l'effetto delle scelte interne al kernel senza includere le collettive MPI degeneri.

Quando l'oggetto dell'esperimento è invece l'esecuzione distribuita, la metrica di riferimento rimane **gflops**.

6.4 Ripetizioni e raccolta dei campioni

Ogni configurazione sperimentale viene eseguita più volte all'interno della stessa esecuzione del programma. Prima delle repetition utilizzate per le misure vengono effettuate alcune invocazioni di warm-up, escluse dalla raccolta dei tempi, in modo da non attribuire alla configurazione eventuali costi transitori della prima esecuzione.

Tutte le campagne utilizzano lo stesso protocollo: 5 invocazioni di warm-up seguite da $R = 20$ repetition misurate. Per ciascuna repetition viene ottenuto il tempo T_i secondo la procedura descritta nelle sezioni precedenti, e il valore utilizzato per il calcolo della prestazione è il tempo medio $\bar{T}$.

I singoli campioni vengono inoltre conservati nei file di output, così da poter verificare a posteriori la regolarità delle misure e individuare eventuali esecuzioni anomale senza perdere l'informazione contenuta nelle singole repetition.

Il seed utilizzato per la generazione dei dati viene mantenuto fisso a 20252026. Tutte le configurazioni di una campagna lavorano quindi sugli stessi dati, e lo stesso protocollo di misura viene mantenuto nelle diverse fasi sperimentali.

La *dispersione* delle repetition viene utilizzata come controllo della stabilità delle misure e viene riportata nei risultati quando rilevante.

6.5 Disegno sperimentale

Il codice espone diversi parametri che possono influenzare le prestazioni: la dimensione del blocco CUDA, il layout di X , il padding della shared memory, la granularità del tile, il backend locale e la precisione numerica. Farli variare tutti contemporaneamente produrrebbe un numero elevato di configurazioni e, soprattutto, renderebbe difficile attribuire le differenze osservate a una causa specifica.

Gli esperimenti sono quindi organizzati in fasi successive. Quando possibile viene modificato un solo fattore alla volta, mantenendo invariati gli altri. I parametri che interagiscono direttamente vengono invece esplorati congiuntamente.

- La *prima fase* è dedicata alla configurazione dei backend CUDA e determina i parametri da utilizzare nei confronti successivi.
- La *seconda fase* verifica come le implementazioni così configurate reagiscano alla variazione della taglia e della forma del problema.
- Una *terza fase* studia l'effetto della precisione numerica, confrontando FP32 e FP64 a parità delle altre condizioni sperimentali.

Le prime tre fasi utilizzano un singolo processo MPI, così da caratterizzare il backend locale senza introdurre contesa sulla GPU.

- Infine, una *quarta fase* considera invece l'applicazione distribuita completa e studia l'interazione fra decomposizione MPI, comunicazioni collettive ed esecuzione CUDA quando più processi condividono l'unico acceleratore disponibile sul nodo.

In questo modo le proprietà dei backend locali vengono determinate prima in un ambiente controllato, mentre l'ultima fase valuta il comportamento della loro composizione con il livello MPI.

6.5.1 Criterio di selezione dei parametri

Le campagne della prima fase producono, per ciascun parametro studiato, una serie di misure sui cinque valori

$$k \in \{3, 6, 8, 20, 32\}.$$

Poiché i parametri di configurazione vengono fissati a tempo di compilazione, non è possibile scegliere un valore diverso per ogni k . Occorre quindi individuare una configurazione che mantenga buone prestazioni sull'intero insieme di casi richiesti.

Indicando con $P(v, k)$ la prestazione ottenuta dalla configurazione v per un determinato valore di k , si definisce la **prestazione relativa**:

$$R(v, k) = \frac{P(v, k)}{\max_{v'} P(v', k)}. \quad (6.17)$$

Per ogni k , il valore migliore assume quindi $R = 1$, mentre gli altri valori esprimono la frazione di prestazione mantenuta rispetto al massimo osservato per quello stesso caso.

È utile esprimere la stessa informazione in termini di **perdita relativa**:

$$L(v, k) = 1 - R(v, k), \quad (6.18)$$

e riassumerla mediante due quantità:

$$L_{\text{media}}(v) = \frac{1}{5} \sum_{k \in \{3, 6, 8, 20, 32\}} L(v, k), \quad (6.19)$$

$$L_{\text{max}}(v) = \max_{k \in \{3, 6, 8, 20, 32\}} L(v, k). \quad (6.20)$$

La prima descrive la perdita media della configurazione rispetto al valore ottimale per ciascun k , mentre la seconda rappresenta il suo caso peggiore.

La selezione privilegia la robustezza rispetto al caso peggiore: le configurazioni vengono ordinate innanzitutto per $L_{\max}$ crescente e, a parità, per L_{media} crescente.

In questo modo viene preferita una configurazione che mantiene *prestazioni uniformi* sui diversi valori di k , evitando che un buon risultato medio nasconda una perdita marcata in uno dei casi richiesti dalla traccia.

La stessa procedura si applica anche quando una configurazione è descritta da più parametri e quindi v rappresenta direttamente la loro combinazione, come nel caso $v = (\text{BLOCK}, \text{TILE_GRANULARITY})$.

6.6 Fase 1 - Configurazione dei backend CUDA

La prima fase ha l'obiettivo di determinare una configurazione rappresentativa per ciascun backend CUDA prima di procedere ai confronti sulle caratteristiche del problema.

Tutte le campagne utilizzano le stesse condizioni di base:

$$M = N = 16\,384, \quad P = 1, \quad k \in \{3, 6, 8, 20, 32\},$$

in doppia precisione, con 5 invocazioni di warm-up e 20 repetition misurate.

La dimensione $16\,384 \times 16\,384$ fornisce un problema sufficientemente grande da esporre **un elevato grado di parallelismo** e da ridurre il peso relativo dei costi fissi, mantenendo al tempo stesso un ampio margine nella memoria della GPU per i buffer ausiliari e per le diverse configurazioni dei backend.

La fase considera separatamente i principali parametri che possono modificare il comportamento dei kernel: dimensione del blocco, layout di X , padding e granularità del tile in shared memory. Quando due parametri interagiscono direttamente, come nel caso della dimensione del blocco e della granularità del tile di `cuda_warp_smem`, essi vengono esplorati congiuntamente.

Le configurazioni selezionate vengono infine utilizzate per confrontare `cuda_naive`, `cuda_warp`, `cuda_warp_smem` e `cublas` nelle stesse condizioni sperimentali.

6.6.1 Campagna 1.1 - Dimensione del blocco e confronto dei backend

La prima campagna studia l'effetto della dimensione del blocco sui kernel CUDA custom. Vengono considerati:

$$\text{BLOCK} \in \{64, 128, 192, 256, 384, 512, 1024\},$$

tutti multipli della dimensione del warp e distribuiti su un intervallo che comprende configurazioni da due warp fino al massimo di 1024 thread per blocco supportato dall'architettura.

Lo sweep viene ripetuto sui cinque valori di k richiesti dalla traccia. In questo modo è possibile verificare quanto la scelta della geometria di lancio dipenda dalla larghezza del multivettore e selezionare un valore che mantenga prestazioni robuste sull'intero insieme dei casi, secondo il criterio definito in sezione 6.5.1.

La campagna comprende `cuda_naive` e `cuda_warp` (variante *column-major*). Nei primi due backend lo sweep agisce direttamente sulla geometria del lancio.

Per `cuda_warp_smem` la dimensione del blocco determina anche il numero di warp che condividono il medesimo tile di X e interagisce quindi con le successive scelte relative alla shared memory. Per questo motivo questa campagna non costituisce una configurazione iniziale per `cuda_warp_smem`, poiché il suo valore di `BLOCK` viene esaminato congiuntamente a `TILE_GRANULARITY` nella campagna dedicata al tiling.

Lo sweep svolge inoltre una seconda funzione. Una volta completate le successive campagne di configurazione, le misure dei backend vengono ricondotte alle rispettive configurazioni selezionate e confrontate sullo stesso problema di riferimento.

Il confronto coinvolge:

`cuda_naive` (variante *column-major*), `cuda_warp`, `cublas`.

Per i kernel custom vengono utilizzati i parametri selezionati durante la Fase 1, mentre cuBLAS costituisce il riferimento esterno e non richiede la scelta di una dimensione del blocco da parte del progetto.

Questo confronto non viene trattato come una campagna sperimentale separata: rappresenta l'esito conclusivo del processo di configurazione iniziato con lo sweep di `BLOCK`.

Il confronto permette quindi di valutare separatamente l'effetto delle principali scelte introdotte nel capitolo CUDA: il passaggio dalla mappatura thread-per-elemento alla mappatura warp-per-row, le due strategie adottate per migliorare gli accessi a X e la distanza rispetto alla routine GEMM fornita da cuBLAS. Le cause delle differenze osservate vengono discusse successivamente alla luce del modello prestazionale e delle caratteristiche dei singoli kernel.

6.6.2 Campagna 1.2 - Layout in memoria di X

La seconda campagna isola l'effetto del layout di X sul backend `cuda_warp`. Vengono confrontate le due varianti descritte in sezione 5.6:

$$X_LAYOUT \in \{\text{row-major}, \text{column-major}\}.$$

La mappatura del lavoro rimane invariata: un warp continua a essere responsabile di una riga di Y e le lane si dividono la dimensione j . Cambia soltanto la disposizione fisica del blocco locale di X .

Nel layout *row-major*, per una colonna c fissata, lane consecutive accedono a elementi appartenenti a righe successive di X e separati da uno stride pari a k elementi. Nel layout *column-major* gli stessi elementi

$$X[j_0][c], X[j_0 + 1][c], \dots, X[j_0 + 31][c]$$

sono invece memorizzati consecutivamente. La campagna permette quindi di misurare direttamente l'effetto del layout sul pattern di accesso alla global memory, mantenendo invariata la struttura del kernel.

Il confronto viene eseguito sui cinque valori di k richiesti dalla traccia e utilizzando per `cuda_warp` la dimensione del blocco selezionata nella Campagna 1.1. Tutti gli altri parametri sperimentali rimangono invariati.

La conversione dal layout row-major al layout per colonne viene eseguita prima delle repetition misurate. Il suo costo non contribuisce quindi al tempo del kernel, ma viene registrato separatamente tramite `t_prep_convert_X_s`.

Il confronto principale della campagna riguarda il tempo di esecuzione del kernel. Il tempo di conversione viene mantenuto come informazione aggiuntiva, necessaria per valutare il costo richiesto dalla variante column-major prima dell'esecuzione delle repetition.

Quando una delle due configurazioni coincide esattamente con un punto già misurato nella Campagna 1.1, viene riutilizzato lo stesso campione anziché ripetere l'esecuzione.

6.6.3 Campagna 1.3 - Granularità del tile

La quarta campagna riguarda esclusivamente `cuda_warp_smem` e studia il parametro `TILE_GRANULARITY`, utilizzato durante la scelta della dimensione del tile di X .

Il numero di righe caricate contemporaneamente in shared memory non è fissato nel sorgente, ma viene determinato a runtime in funzione del budget di shared memory disponibile per blocco. Il valore così ottenuto viene arrotondato verso il basso a un multiplo di `TILE_GRANULARITY`.

La campagna considera

$$\text{TILE_GRANULARITY} \in \{1, 8, 16, 32\}.$$

Una granularità piccola permette di utilizzare più precisamente il budget di shared memory disponibile, mentre una granularità pari alla dimensione del warp favorisce tile con un numero di righe multiplo di 32. Quest'ultima scelta evita passi parziali nel ciclo con cui le lane attraversano il tile, ma può lasciare inutilizzata una parte del budget disponibile.

La granularità non viene studiata indipendentemente dalla dimensione del blocco. In `cuda_warp_smem`, `BLOCK` determina il numero di warp appartenenti allo stesso blocco e contribuisce ai vincoli che determinano quanti blocchi possono risiedere contemporaneamente su uno SM. La pianificazione del tile utilizza proprio queste informazioni per stabilire il budget di shared memory assegnabile a ciascun blocco.

Di conseguenza, modificare `BLOCK` può modificare anche la dimensione del tile risultante. Per evitare di selezionare separatamente due parametri che possono influenzarsi, la campagna esplora congiuntamente

$$\text{BLOCK} \in \{64, 128, 192, 256, 384, 512, 1024\}$$

e

$$\text{TILE_GRANULARITY} \in \{1, 8, 16, 32\}.$$

Lo sweep viene ripetuto sui cinque valori di k , per un totale di $7 \times 4 \times 5 = 140$ configurazioni.

Il valore di `SMEM_PAD` viene invece mantenuto fisso alla configurazione selezionata nella Campagna 1.3, così da isolare l'interazione fra dimensione del blocco e granularità del tile.

In questo caso una configurazione v del criterio definito in sezione 6.5.1 è quindi una coppia

$$v = (\text{BLOCK}, \text{TILE_GRANULARITY}).$$

La selezione viene effettuata sulla coppia nel suo insieme, anziché scegliere prima la dimensione del blocco e successivamente la granularità.

6.6.4 Campagna 1.4 - Padding del tile in shared memory

La terza campagna riguarda esclusivamente `cuda_warp_smem` e studia l'effetto del padding applicato alle righe del tile di X in shared memory.

Vengono confrontate le due configurazioni:

$$\text{SMEM_PAD} \in \{0, 1\},$$

che producono rispettivamente:

$$\text{tile_row_stride} = k \quad \text{e} \quad \text{tile_row_stride} = k + 1.$$

La struttura del kernel, la dimensione del blocco e tutti gli altri parametri sperimentali vengono mantenuti invariati.

Il parametro modifica la distanza, in shared memory, fra gli elementi letti da lane consecutive durante il consumo del tile. In doppia precisione ciascun elemento occupa due parole da 32 bit, per cui la distribuzione degli accessi sui 32 bank dipende dallo stride

$$2(k + \text{SMEM_PAD}) \pmod{32}.$$

Il padding è stato introdotto per interrompere pattern periodici sfavorevoli, in particolare il caso $k = 32$, per il quale in assenza di padding lo stride della riga produce una mappatura patologica sui bank.

L'effetto del padding non è però indipendente dal valore di k . Modificando lo stride della riga cambia infatti anche la mappatura sui bank per i casi che non presentano il comportamento patologico di $k = 32$.

La campagna viene quindi eseguita su tutti i valori

$$k \in \{3, 6, 8, 20, 32\},$$

con l'obiettivo di verificare se il beneficio ottenuto nei casi sfavorevoli viene mantenuto senza introdurre penalizzazioni significative negli altri.

6.7 Fase 2 - Caratterizzazione dimensionale

La seconda fase verifica se il comportamento dei backend configurati nella Fase 1 rimane stabile quando cambiano la dimensione e la forma del problema.

La campagna risponde inoltre direttamente alla richiesta della traccia di ripetere le misure per valori crescenti di M e N e di considerare almeno tre rapporti fra le due dimensioni, includendo il caso quadrato $M = N$.

Vengono controllati due fattori:

$$S \in \{4096, 8192, 16384\} \quad (\text{scala nominale}), \quad (6.21)$$

$$\text{forma} \in \{M = N, N = 2M, M = 3N\} \quad (\text{rapporto fra le dimensioni}). \quad (6.22)$$

La scala S definisce la dimensione del problema di riferimento, corrispondente al caso quadrato $M = N = S$. Per confrontare forme differenti senza modificare sostanzialmente il numero di operazioni, le dimensioni dei casi rettangolari vengono scelte in modo tale che

$$MN \simeq S^2.$$

A parità di S e di k , le tre forme richiedono quindi una quantità di calcolo pressoché identica, ma presentano rapporti differenti tra le dimensioni della matrice. In questo modo è possibile studiare l'effetto della forma del problema senza confonderlo con una variazione significativa del numero complessivo di operazioni.

Tabella 6.1: Punti della caratterizzazione dimensionale. A parità di scala, il prodotto MN rimane entro l'1,2% del valore del caso quadrato.

Scala	$M = N$	$N = 2M$	$M = 3N$	MN nominale
$S = 4096$	4096×4096	2880×5760	7104×2368	$1,68 \cdot 10^7$
$S = 8192$	8192×8192	5760×11520	14208×4736	$6,71 \cdot 10^7$
$S = 16384$	16384×16384	11520×23040	28416×9472	$2,68 \cdot 10^8$

Per ogni combinazione di scala e forma vengono eseguiti i quattro backend CUDA nelle configurazioni selezionate durante la Fase 1 e sui cinque valori

$$k \in \{3, 6, 8, 20, 32\}.$$

Le misure vengono raccolte in doppia precisione e con un solo rank MPI.

Il disegno comprende quindi $3 \times 3 \times 4 \times 5 = 180$ configurazioni.

Le due variabili permettono di osservare fenomeni differenti:

- Variando S , ma mantenendo il rapporto $M = N$, cresce la dimensione complessiva del problema. La campagna permette quindi di verificare come il throughput evolva passando da problemi più piccoli a problemi che espongono una quantità maggiore di lavoro parallelo, e se le prestazioni tendano a stabilizzarsi alle taglie maggiori.
- Variando invece il rapporto $M : N$ a scala fissata, il numero complessivo di operazioni rimane pressoché costante, mentre cambia la geometria del problema. Cambiano, fra le altre cose, il numero di righe da elaborare, la lunghezza della dimensione di riduzione e le dimensioni relative dei buffer X e Y .

Il confronto fra le tre forme permette quindi di verificare quanto ciascun backend sia sensibile alla geometria della matrice, separando per quanto possibile questo effetto da quello dovuto alla sola quantità di lavoro.

6.8 Fase 3 - Effetto della precisione numerica

La terza fase studia l'effetto della precisione numerica sulle prestazioni dei backend CUDA. Il confronto viene effettuato fra FP64 e FP32 mantenendo invariate, per quanto possibile, tutte le altre caratteristiche dell'esecuzione.

La campagna utilizza il caso quadrato di dimensione maggiore,

$$M = N = 16\,384, \quad P = 1,$$

e considera i cinque valori

$$k \in \{3, 6, 8, 20, 32\}.$$

Vengono confrontati i quattro backend `cuda_naive`, `cuda_warp`, `cuda_warp_smem` e `cublas`.

Per i kernel custom vengono mantenuti gli stessi parametri strutturali selezionati nella Fase 1. La dimensione del blocco, il layout di X , il padding della shared memory e la granularità del tile non vengono quindi ritirati separatamente per FP32.

Questa scelta permette di isolare il più possibile l'effetto della precisione: fra le due build cambia il tipo `scalar_t`, mentre la struttura dell'algoritmo e la configurazione scelta per il kernel rimangono invariate.

Di conseguenza, la campagna non deve essere interpretata come una ricerca della migliore configurazione possibile per ciascuna precisione, ma come un confronto controllato fra le due rappresentazioni numeriche.

Le misure FP32 vengono confrontate con il punto FP64 corrispondente già raccolto nella Fase 2 per $M = N = 16\,384$.

Il riferimento viene riutilizzato perché dimensioni del problema, backend, configurazioni dei kernel, numero di processi, valori di k e protocollo di misura coincidono con quelli della presente campagna. Non è quindi necessario introdurre una diversa baseline FP64.

Poiché l'obiettivo è isolare l'effetto della precisione sul calcolo GPU, il confronto viene effettuato sul tempo del solo kernel. Per ciascun backend e valore di k si definisce:

$$S_{32/64}(k) = \frac{t_{\text{kernel}}^{\text{FP64}}(k)}{t_{\text{kernel}}^{\text{FP32}}(k)}. \quad (6.23)$$

Un valore maggiore di uno indica quindi una riduzione del tempo di kernel passando dalla doppia alla singola precisione.

La precisione modifica contemporaneamente più proprietà dell'esecuzione. Passando da FP64 a FP32 si dimezza la dimensione di ciascun elemento in memoria e cambia il throughput aritmetico disponibile sulla GPU. Possono inoltre cambiare l'utilizzo dei registri e, per i kernel che utilizzano shared memory, la quantità di memoria richiesta dal tile.

La campagna permette quindi di osservare come questi effetti complessivi si riflettano sul tempo di esecuzione e se il comportamento misurato sia coerente con il modello roofline discusso nel ??.

L'interpretazione delle differenze viene rimandata alla sezione dei risultati.

6.9 Fase 4 - Esecuzione distribuita MPI + CUDA

Le prime tre fasi caratterizzano i backend CUDA utilizzando un solo processo MPI. Questa scelta permette di attribuire le differenze osservate alle caratteristiche dei kernel senza introdurre contesa sull'acceleratore.

La quarta fase considera invece l'esecuzione distribuita completa. Il numero di processi MPI viene aumentato mantenendo invariato il problema globale, e ciascun rank esegue il prodotto sul proprio blocco locale di A secondo il solito protocollo:

$$\text{Bcast}(X) \longrightarrow \text{local_gemm} \longrightarrow \text{Reduce}(Y).$$

L'obiettivo non è caratterizzare nuovamente il kernel locale, ma osservare come comunicazioni MPI, decomposizione del problema ed esecuzione CUDA interagiscano nel tempo complessivo del prodotto distribuito.

Il nodo utilizzato per gli esperimenti dispone di una sola GPU. Tutti i rank CUDA selezionano quindi lo stesso device e, per $P > 1$, più processi con contesti CUDA distinti condividono lo stesso acceleratore.

La campagna non rappresenta pertanto uno scaling su più GPU: aumentare il numero di processi suddivide il problema globale in blocchi locali più piccoli e modifica il costo delle comunicazioni MPI, ma non aumenta le risorse di calcolo disponibili sul device. Al contrario, i rank devono contendersi l'unica GPU presente nel sistema.

Per questo motivo i risultati vengono interpretati come studio dell'interazione fra MPI e un acceleratore condiviso, e non come misura della scalabilità di un sistema multi-GPU.

Per ogni configurazione vengono conservate le stesse componenti temporali utilizzate nelle fasi precedenti:

$$t_{\text{Bcast}}, \quad t_{\text{kernel}}, \quad t_{\text{Reduce}}.$$

La loro evoluzione al crescere di P permette di distinguere due effetti. Da un lato cambia il costo della parte MPI, dall'altro ciascun processo esegue un prodotto locale più piccolo ma condivide la GPU con gli altri rank.

La metrica prestazionale principale rimane quella definita in sezione 6.3, aggregando per ogni repetition il tempo dei rank mediante il massimo e calcolando successivamente la media sulle repetition.

La condivisione dell'acceleratore rende inoltre utile una misura aggiuntiva della fase locale nel suo complesso. A questo scopo il programma può essere eseguito con l'opzione `-group-timing`.

In questa modalità i rank vengono sincronizzati immediatamente prima della chiamata a `local_gemm` e nuovamente al suo completamento. L'intervallo misurato rappresenta quindi il tempo necessario affinché l'intero gruppo di processi completi la fase locale a partire da un istante comune.

Indichiamo tale quantità con:

$$t_{\text{local}}^{\text{gruppo}}.$$

A differenza di t_{kernel} , che descrive l'intervallo misurato dal singolo rank mediante eventi CUDA, $t_{\text{local}}^{\text{gruppo}}$ osserva il completamento collettivo della fase locale e comprende anche i trasferimenti host-device e device-host effettuati dalle diverse chiamate a `local_gemm`.

Le esecuzioni con `-group-timing` vengono considerate misure diagnostiche separate. Le barriere aggiuntive modificano infatti la sincronizzazione fra i rank e non devono essere introdotte nelle esecuzioni utilizzate per il confronto prestazionale principale.

La campagna viene quindi letta su due livelli: le esecuzioni standard descrivono le prestazioni del prodotto distribuito, mentre le esecuzioni con group timing vengono utilizzate per caratterizzare il costo collettivo della fase CUDA quando più processi condividono lo stesso acceleratore.

La campagna utilizza il backend custom selezionato al termine delle fasi precedenti nella sua configurazione FP64 di riferimento. Il confronto con cuBLAS viene mantenuto come baseline esterna, mentre le varianti utilizzate esclusivamente durante il tuning non vengono nuovamente esplorate.

La fase non cerca quindi un valore di P che massimizzi artificialmente le prestazioni sulla GPU condivisa. L'obiettivo è quantificare il costo della composizione fra il livello MPI e il backend CUDA e mostrare come l'architettura distribuita del programma si comporti nelle condizioni hardware effettivamente disponibili.

Capitolo 7

Risultati

Il capitolo segue le quattro fasi definite in sezione 6.5: le prime tre fasi caratterizzano i backend CUDA con un solo processo MPI, mentre la quarta fase porta le configurazioni ottenute nell'applicazione distribuita completa, dove più rank condividono l'unica GPU del nodo.

7.1 Fase 1 - Configurazione dei backend CUDA

Tutte le misure di questa sezione sono prese a $M = N = 16\,384$, doppia precisione, $P = 1$.

7.1.1 Campagna 1.1 - Dimensione del blocco

La tabella 7.1 riporta, per ciascun kernel e ciascun valore di BLOCK, le due statistiche di perdita definite nella eq. (6.20): quanto quella dimensione di blocco perde, in media e nel caso peggiore, rispetto al miglior blocco *per lo stesso k*.

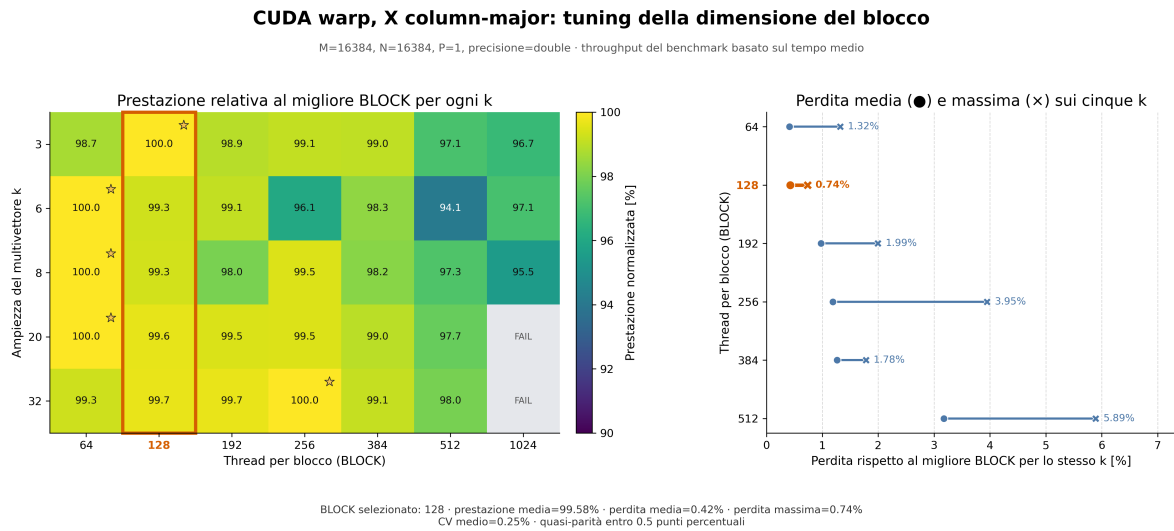
Tabella 7.1: Campagna 1.1: perdita rispetto al miglior BLOCK per lo stesso k , in percentuale. In grassetto il valore selezionato per ciascun kernel.

BLOCK	cuda_naive		cuda_warp	
	media	max	media	max
64	1,13	3,85	0,41	1,32
128	0,92	1,69	0,42	0,74
192	3,55	8,21	0,98	1,99
256	1,09	4,56	1,19	3,95
384	3,84	9,71	1,27	1,78
512	5,64	10,23	3,17	5,89
1024	10,86	23,29	<i>lancio fallito per $k = 20$ e $k = 32$</i>	

I valori selezionati sono quindi BLOCK = 128 per cuda_naive e cuda_warp, BLOCK = 256 per cuda_warp_smem. Le prestazioni assolute corrispondenti sono riportate in tabella 7.2.

Tabella 7.2: Campagna 1.1: gflops_kernel al BLOCK selezionato.

Backend	$k = 3$	$k = 6$	$k = 8$	$k = 20$	$k = 32$
cuda_naive (BLOCK=128)	149,3	308,7	337,3	345,6	350,3
cuda_warp (BLOCK=128)	300,3	351,4	350,3	351,4	352,5



(a) cuda_warp, X column-major.

Figura 7.1: Campagna 1.1: prestazione normalizzata per (BLOCK, k) a sinistra, perdita media e massima sui cinque k a destra. Le celle FAIL sono i lanci non riusciti a BLOCK= 1024.

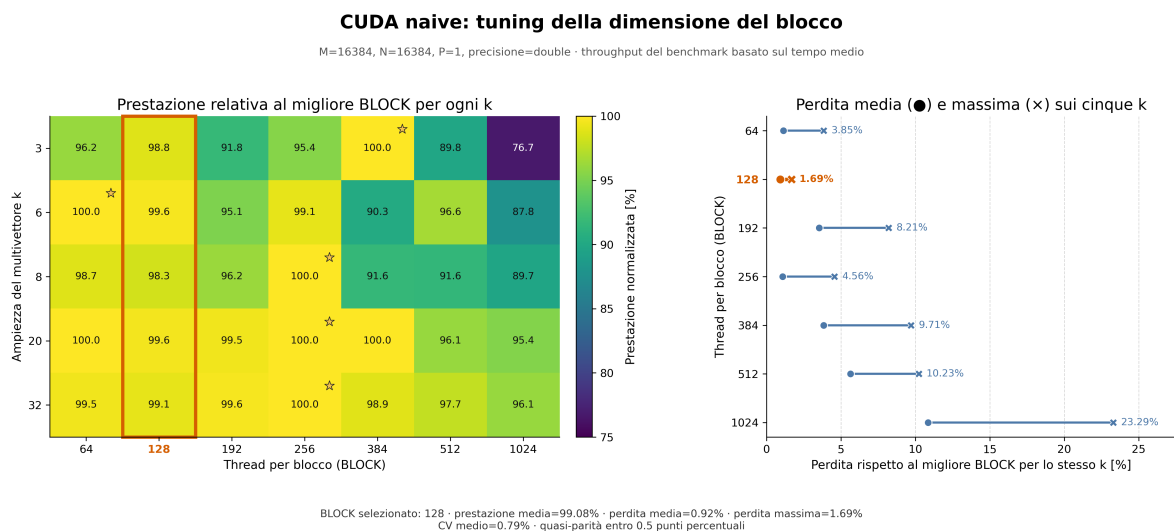


Figura 7.2: Campagna 1.1: stessa lettura per cuda_naive, l'unico kernel che completa anche a BLOCK= 1024.

Lettura

La sensibilità a BLOCK dipende dalla struttura del kernel.

cuda_warp è il backend meno sensibile nell'intervallo delle configurazioni valide. Fra 64 e 384 thread per blocco le prestazioni rimangono vicine all'ottimo e il valore selezionato, BLOCK=128, limita la perdita massima allo 0,74%. Il risultato è coerente con la mappatura warp-per-row: il lavoro associato a una riga di Y è già definito alla granularità del warp, per cui modificare la dimensione del blocco cambia principalmente quanti warp vengono raggruppati nello stesso blocco, senza modificare il lavoro assegnato a ciascuno di essi.

Non emerge una regola semplice basata sulla sola dimensione del blocco.

Per `cuda_naive`, ad esempio, le configurazioni `BLOCK=192` e `BLOCK=384` risultano peggiori di 128 e 256, ma lo stesso andamento non si ripete in modo uniforme sugli altri kernel. Analogamente, `BLOCK=512` produce per `cuda_warp` una perdita maggiore rispetto a blocchi più piccoli. Il numero di thread per blocco non è quindi sufficiente, da solo, a spiegare le differenze osservate. Alla geometria del lancio si aggiungono infatti il consumo di registri per thread.

I fallimenti a `BLOCK=1024` costituiscono un ulteriore limite della configurazione.

Con 1024 thread per blocco, `cuda_warp` non completa l'esecuzione per $k = 20$ e $k = 32$, mentre `cuda_naive` continua a essere eseguibile. La differenza è coerente con la diversa pressione sui registri: il kernel naive mantiene un solo accumulatore per thread, mentre nelle istanze specializzate dei due kernel warp ogni lane mantiene un vettore di k accumulatori.

Esito della campagna. Il criterio di robustezza porta a selezionare `BLOCK=128` per `cuda_naive` e per `cuda_warp`. In entrambi i casi la configurazione scelta rimane molto vicina al miglior valore specifico per ciascun k , evitando di ottimizzare il kernel per un solo punto della traccia.

7.1.2 Campagna 1.2 - Layout in memoria di X

Tabella 7.3: Campagna 1.2: `cuda_warp` con X row-major e column-major, `BLOCK=128`. L'ultima colonna è il costo della conversione di X , che è preprocessing e non entra in T .

k	row-major [GFLOPS]	column-major [GFLOPS]	speedup col/row	$t_{\text{kernel row}}$ [ms]	t_{convert} [ms]
3	297,3	300,3	1,01	5,4	0,26
6	352,0	351,4	1,00	9,2	0,57
8	182,6	350,3	1,92	23,5	0,88
20	174,1	351,4	2,02	61,7	2,43
32	92,2	352,5	3,82	186,4	3,32

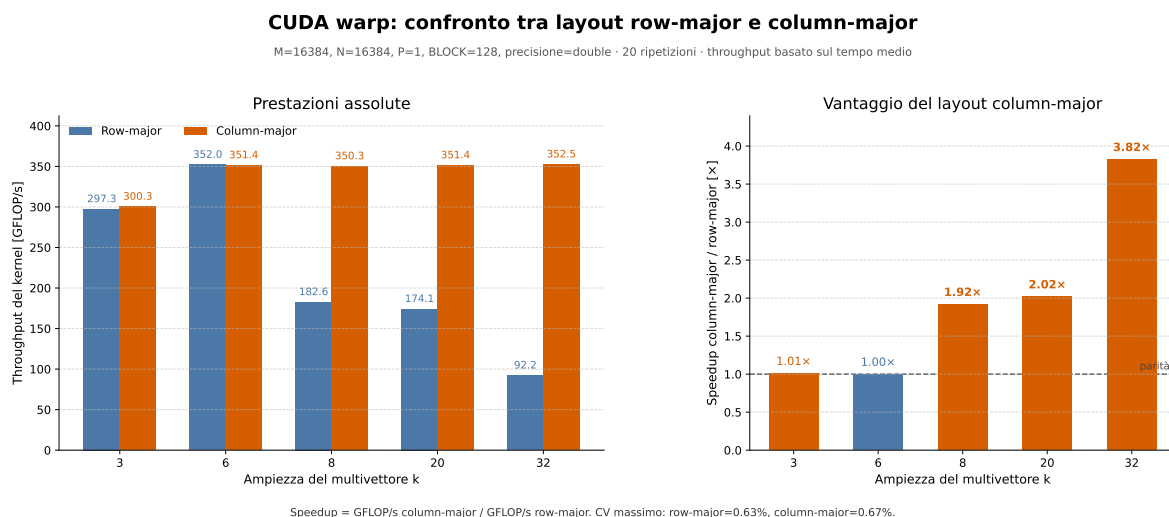


Figura 7.3: Campagna 1.2: prestazioni assolute (sinistra) e vantaggio del layout column-major (destra). CV massimo: 0,63% row-major, 0,67% column-major.

Lettura

L'effetto del layout emerge a partire da $k = 8$.

Per $k = 3$ e $k = 6$ le due varianti hanno prestazioni molto simili: lo speedup del layout column-major vale rispettivamente $1,01\times$ e $1,00\times$, con coefficienti di variazione inferiori allo 0,7%. In questa regione l'eventuale vantaggio è quindi dello stesso ordine della dispersione delle misure e non è sufficientemente marcato da attribuirgli un significato prestazionale.

A partire da $k = 8$ il comportamento cambia nettamente: il layout column-major raggiunge uno speedup di $1,92\times$, che sale a $2,02\times$ per $k = 20$ e a $3,82\times$ per $k = 32$.

Il vantaggio cresce quindi all'aumentare della larghezza del multivettore e rende il layout column-major la scelta naturale per `cuda_warp` nelle campagne successive.

L'andamento è coerente con il pattern di accesso previsto dal kernel.

In `cuda_warp` lane consecutive elaborano valori consecutivi dell'indice j . Per una colonna c fissata, con X row-major esse richiedono

$$X[j_0][c], X[j_0 + 1][c], \dots, X[j_0 + 31][c],$$

che nel buffer sono separati da uno stride di k elementi. All'aumentare di k gli indirizzi richiesti dalle lane dello stesso warp risultano quindi sempre più distanti, rendendo meno favorevole il raggruppamento degli accessi alla global memory.

Con X column-major gli stessi elementi sono invece *consecutivi* in memoria. Il kernel mantiene invariata la propria mappatura warp-per-row, ma le richieste delle lane possono essere servite mediante accessi molto più compatti. La crescita del vantaggio con k è quindi coerente con l'ipotesi che il layout row-major penalizzi progressivamente il percorso di lettura di X .

I risultati mostrano che la differenza fra i layout diventa rilevante proprio quando aumenta lo stride fra gli elementi letti simultaneamente dalle lane.

Il confronto chiarisce anche il ruolo della shared memory.

`cuda_warp_smem` mantiene X nel layout row-major, ma evita che il pattern strided del kernel `cuda_warp` venga ripetuto direttamente sulla global memory. Il tile di X viene prima caricato cooperativamente dai thread del blocco, utilizzando accessi globali contigui, e viene successivamente consumato dalla shared memory secondo la solita mappatura warp-per-row.

Le due varianti affrontano quindi lo stesso limite del percorso row-major con strategie differenti: `cuda_warp` modifica la rappresentazione fisica di X , mentre `cuda_warp_smem` mantiene il layout originario e modifica il percorso attraverso cui i dati raggiungono le lane. Il confronto conclusivo della Fase 1 permette di valutare quale delle due strategie risulti più conveniente nel regime studiato.

7.1.3 Campagna 1.3 - Granularità del tile

Tabella 7.4: Campagna 1.3: prestazione normalizzata di `cuda_warp_smem` per ogni coppia (`BLOCK`, `g`), in percentuale dell'ottimo per lo stesso `k`. Media sui cinque `k` e caso peggiore. In grassetto la coppia selezionata.

	$g = 1$		$g = 8$		$g = 16$		$g = 32$	
<code>BLOCK</code>	media	peggiore	media	peggiore	media	peggiore	media	peggiore
64	83,6	54,7	82,7	53,8	77,4	46,2	86,8	61,0
128	88,0	51,3	84,3	50,5	82,2	50,4	98,7	97,1
192	79,0	67,4	76,5	66,4	87,7	71,9	94,9	81,1
256	85,0	65,5	98,1	94,4	96,7	87,8	99,0	98,5
384	89,1	80,0	89,9	78,6	89,5	76,0	90,5	80,2
512	91,9	82,8	92,3	90,7	90,6	86,0	92,2	89,7

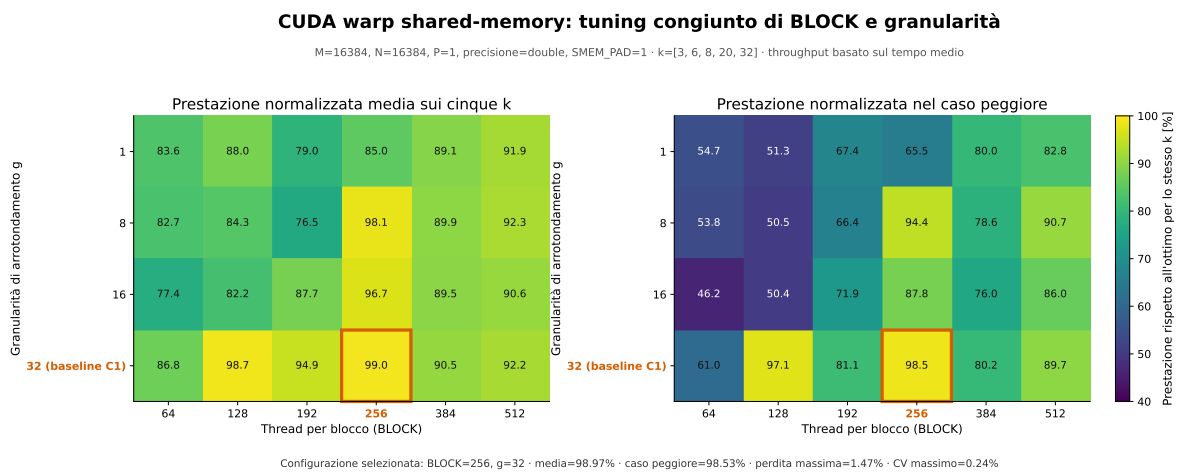


Figura 7.4: Campagna 1.4: tuning congiunto di `BLOCK` e granularità. A sinistra la prestazione normalizzata media sui cinque `k`, a destra il caso peggiore.

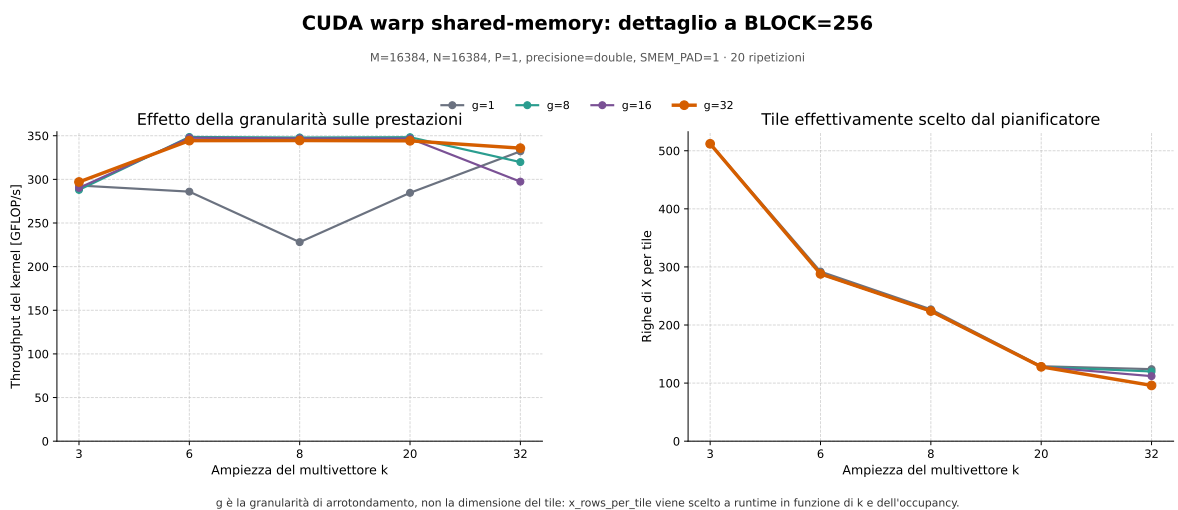


Figura 7.5: Campagna 1.4: dettaglio a `BLOCK`= 256. A destra il piano di tiling effettivamente scelto a runtime, che dipende da `k` e non da `g` se non attraverso l'arrotondamento.

Lettura

Lo sweep congiunto conferma che BLOCK e granularità non possono essere scelti indipendentemente.

La configurazione preferibile cambia infatti al variare di g . Con $g = 1$ il valore medio più alto si ottiene a BLOCK=512, mentre per $g = 8$, $g = 16$ e $g = 32$ il miglior valore medio si trova a BLOCK=256. Anche il comportamento nel caso peggiore cambia con la coppia considerata.

La dimensione del blocco influenza il numero di warp che cooperano nello stesso blocco e le risorse che determinano l'*occupancy*, mentre la granularità interviene successivamente sulla dimensione del tile scelta dal pianificatore. I due parametri agiscono quindi sullo stesso kernel attraverso meccanismi distinti, ma non indipendenti.

Fra le granularità considerate, $g = 32$ mostra il comportamento più robusto.

Essa fornisce la migliore prestazione media per cinque delle sei dimensioni di blocco considerate, con l'unica eccezione di BLOCK=512, dove $g = 8$ e $g = 32$ risultano sostanzialmente equivalenti. In particolare, la coppia

$$(\text{BLOCK}, g) = (256, 32)$$

mantiene in media il 99,0% della migliore prestazione ottenibile per ciascun k e non scende sotto il 98,5% nel caso peggiore.

Il risultato è coerente con il ruolo di TILE_GRANULARITY nel kernel. Il ciclo di consumo della shared memory assegna alla lane ℓ le righe

$$j = \ell, \ell + 32, \ell + 64, \dots$$

del tile. Quando il numero di righe è multiplo di 32, tutte le lane completano lo stesso numero di passi. Una dimensione non multipla di 32 può invece produrre un'ultima iterazione nella quale soltanto una parte delle lane ha ancora una riga da elaborare.

Questo non implica tuttavia che una granularità pari a 32 sia intrinsecamente ottimale. Arrotondare il tile a multipli più grandi può lasciare inutilizzata una parte del budget di shared memory e aumentare il numero complessivo di tile, e quindi delle sincronizzazioni. La campagna misura proprio il bilancio fra questi due effetti.

La granularità rifinisce un piano determinato principalmente dal budget di shared memory.

Come mostra fig. 7.5, le diverse granularità non producono piani radicalmente differenti. Il pianificatore determina prima quante righe sono compatibili con il budget disponibile e con l'obiettivo di occupancy, quindi arrotonda tale valore secondo g e verifica nuovamente il numero di blocchi residenti.

Per questo motivo TILE_GRANULARITY non determina da sola la dimensione del tile. Ne modifica piuttosto il valore finale attorno a una dimensione già imposta dalle risorse disponibili. L'effetto prestazionale può quindi essere ridotto quando i diversi arrotondamenti conducono a tile simili, oppure diventare più evidente quando cambiano il numero di passi necessari per attraversare X e, di conseguenza, il numero di sincronizzazioni eseguite dal blocco.

7.1.4 Campagna 1.4 - Padding del tile in shared memory

Tabella 7.5: Campagna 1.4: effetto di SMEM_PAD su cuda_warp_smem, BLOCK= 256. x_rows_per_tile è il numero di righe di X che il pianificatore fa entrare nel tile.

k	PAD=0 [GFLOPS]	PAD=1 [GFLOPS]	guadagno	x_rows PAD=0	x_rows PAD=1
3	292,8	296,7	+1,3%	672	512
6	315,8	351,8	+11,4%	320	288
8	327,5	352,4	+7,6%	256	224
20	353,9	352,2	-0,5%	128	128
32	177,1	343,7	+94,1%	128	96

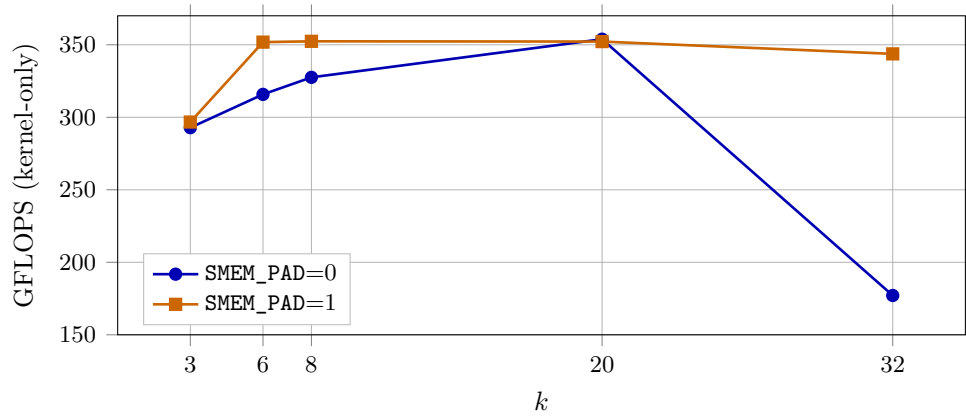


Figura 7.6: Campagna 1.3: confronto fra tile senza padding e con una colonna di padding. Il caso $k = 32$ rende visibile la patologia che negli altri punti è parzialmente nascosta dal resto del lavoro del kernel.

Lettura

Il caso $k = 32$ conferma il comportamento patologico previsto per la mappatura sui bank.

Senza padding, la distanza fra due righe consecutive del tile è pari a $k = 32$ elementi. In doppia precisione ciascun elemento occupa due parole da 32 bit, quindi lo spostamento fra gli indirizzi iniziali letti da lane consecutive vale

$$2 \cdot 32 = 64 \equiv 0 \pmod{32}.$$

Gli accessi ricadono quindi periodicamente sugli stessi bank, producendo un forte conflitto durante la lettura del tile dalla shared memory.

Con SMEM_PAD=1 lo stride diventa invece 33 elementi e lo spostamento corrispondente vale

$$2 \cdot 33 = 66 \equiv 2 \pmod{32},$$

rompendo la periodicità del caso precedente. Il risultato sperimentale è coerente con questa previsione: a $k = 32$ il throughput passa da 177,1 a 343,7 GFLOPS, con un incremento del 94,1%.

Il guadagno complessivo del kernel non deve però essere interpretato come misura diretta della molteplicità del bank conflict. Il conflitto interessa infatti soltanto la fase in cui X viene letto dalla shared memory, mentre il tempo totale comprende anche caricamento del tile, accessi ad A , operazioni aritmetiche, riduzione e sincronizzazioni.

Negli altri valori di k l'effetto del padding è meno regolare.

Il guadagno vale 11,4% a $k = 6$ e 7,6% a $k = 8$, mentre a $k = 20$ le due configurazioni differiscono soltanto dello 0,5% a favore della variante senza padding. A $k = 3$ il vantaggio è invece limitato all'1,3%.

Questi risultati mostrano che la semplice regola modulare è utile per individuare configurazioni chiaramente patologiche, come $k = 32$, ma non costituisce da sola un modello quantitativo delle prestazioni. Il tempo complessivo dipende anche dalla dimensione effettiva del tile, dal numero di blocchi residenti e dalla parte del kernel non coinvolta negli accessi alla shared memory.

Il padding migliora il caso peggiore al costo di una minore capacità utile del tile.

Aggiungere una colonna modifica la dimensione di ogni riga da k a $k + 1$ elementi. A parità di budget di shared memory possono quindi essere mantenute meno righe di X nello stesso tile. Ad esempio, il pianificatore passa da 672 a 512 righe per $k = 3$ e da 128 a 96 per $k = 32$.

Le righe che non entrano nel tile corrente non vengono escluse dal calcolo, ma vengono elaborate nei tile successivi. Il costo potenziale del padding è quindi una riduzione della quantità di X trattata in ciascuna iterazione del tiling e, in alcuni casi, un aumento del numero complessivo di tile e delle relative sincronizzazioni.

Nonostante questo costo, il peggioramento massimo osservato con `SMEM_PAD=1` è soltanto dello 0,5%, mentre nel caso patologico $k = 32$ il vantaggio è vicino a un fattore due. La scelta del padding è quindi fortemente favorevole secondo il criterio di robustezza adottato nella Fase 1.

Esito della campagna.

La configurazione selezionata è quindi:

$$\text{SMEM_PAD} = 1.$$

Ripercussioni sul tuning congiunto.

Combinando questo risultato con la scelta `TILE_GRANULARITY=32` della Campagna 1.3, la configurazione definitiva di `cuda_warp_smem` diventa

$$(\text{BLOCK}, \text{SMEM_PAD}, g) = (256, 1, 32).$$

Nello sweep congiunto la coppia (256, 32) presenta una perdita massima dell'1,5% rispetto alla migliore configurazione specifica per ciascun k . La scelta soddisfa quindi il criterio di robustezza adottato nella Fase 1 e definisce definitivamente il valore di `BLOCK`.

7.1.5 Esito della Fase 1 - Configurazioni consolidate

Le quattro campagne precedenti determinano le configurazioni utilizzate nelle fasi successive. I valori selezionati sono raccolti in tabella 7.6.

Tabella 7.6: Configurazione selezionata per ciascun backend CUDA.

Backend	BLOCK	Layout di X	Altro
cuda_naive	128	row	-
cuda_warp	128	column	-
cuda_warp_smem	256	row	SMEM_PAD=1, $g = 32$
cublas	-	row	riferimento esterno

Per `cuda_naive` e `cuda_warp` la dimensione del blocco deriva dalla Campagna 1.1. Per `cuda_warp` la Campagna 1.2 seleziona inoltre il layout column-major di X . Le Campagne 1.3 e 1.4 completano invece la configurazione di `cuda_warp_smem`, selezionando rispettivamente il padding e la coppia (BLOCK, TILE_GRANULARITY).

Da questo punto in avanti i parametri dei backend rimangono invariati. Le fasi successive modificano quindi le caratteristiche del problema o dell'aritmetica, senza introdurre un nuovo tuning.

7.2 Fase 2 - Caratterizzazione dimensionale

La Fase 1 ha fissato, per ciascun backend CUDA, una configurazione definitiva. Tutte quelle misure sono però state prese su un unico problema, $M = N = 16\,384$. Resta quindi aperta una domanda: quei risultati valgono solo per quella taglia, oppure descrivono un comportamento più generale?

La Fase 2 risponde variando le dimensioni del problema e separando due effetti che nella pratica si presentano sempre insieme:

- la **scala**, cioè quanto è grande il problema;
- la **forma**, cioè il rapporto fra il numero di righe M e il numero di colonne N , a parità di lavoro complessivo.

Le due domande vengono affrontate una alla volta. Prima si tiene la matrice quadrata e si fa crescere la taglia; poi si fissa la taglia e si cambia la forma lasciando il lavoro quasi invariato.

7.2.1 Disegno della campagna

La campagna considera tre scale nominali

$$S \in \{4096, 8192, 16\,384\}$$

e, per ciascuna scala, tre geometrie: il caso quadrato $M = N$, il caso “largo” $N = 2M$ e il caso “alto” $M = 3N$. I nove punti sono

	$M = N$	$N = 2M$	$M = 3N$
$S = 4096$	4096×4096	2880×5760	7104×2368
$S = 8192$	8192×8192	$5760 \times 11\,520$	$14\,208 \times 4736$
$S = 16\,384$	$16\,384 \times 16\,384$	$11\,520 \times 23\,040$	$28\,416 \times 9472$

Le dimensioni rettangolari non sono scelte a caso: sono state costruite in modo che il prodotto MN resti quasi uguale a quello del caso quadrato della stessa scala. Rispetto a S^2 , la forma $N = 2M$ ha l'1,12% di elementi in meno e la forma $M = 3N$ lo 0,27% in più.

Questo dettaglio è ciò che rende interpretabile il confronto. Poiché il numero di operazioni è $2MNk$, tenere MN costante significa tenere costante il lavoro. Se due forme della stessa scala

danno throughput diversi, la differenza non può essere attribuita al fatto che una delle due “fa più conti”: è una reazione del kernel alla geometria.

Tutte le misure sono in doppia precisione, con $P = 1$, 5 warm-up e 20 repetition misurate, e usano per ogni backend i parametri selezionati nella Fase 1 (sezione 7.1.5). La grandezza riportata è sempre `gflops_kernel`, cioè il throughput del solo kernel, al netto dei trasferimenti H2D e D2H. Il prodotto completo dei fattori dà $3 \text{ scale} \times 3 \text{ forme} \times 4 \text{ backend} \times 5 \text{ valori di } k$, cioè 180 configurazioni misurate.

7.2.2 Effetto della scala

Per isolare la scala si guardano soltanto le configurazioni quadrate, $M = N = S$, facendo crescere S di un fattore 2 per volta, cioè facendo crescere il lavoro di un fattore 4 per volta.

Tabella 7.7: Fase 2: `gflops_kernel` dei quattro backend sul caso quadrato $M = N = S$, per tutti i valori di k misurati.

Scala	Backend	$k = 3$	$k = 6$	$k = 8$	$k = 20$	$k = 32$
$S = 4096$	<code>cuda_naive</code>	162,3	252,0	253,2	277,2	287,7
	<code>cuda_warp</code>	269,6	275,7	277,4	279,8	281,5
	<code>cuda_warp_smem</code>	253,5	265,8	267,5	273,5	274,5
	<code>cublas</code>	16,5	34,6	46,1	115,2	184,3
$S = 8192$	<code>cuda_naive</code>	187,3	309,7	342,5	348,7	354,5
	<code>cuda_warp</code>	297,5	348,4	347,8	347,5	354,1
	<code>cuda_warp_smem</code>	291,5	338,3	337,6	338,6	330,7
	<code>cublas</code>	15,4	30,8	41,1	102,7	164,0
$S = 16384$	<code>cuda_naive</code>	151,5	307,1	342,0	345,5	349,7
	<code>cuda_warp</code>	302,0	354,3	353,2	354,6	354,9
	<code>cuda_warp_smem</code>	296,8	349,8	349,6	349,5	341,0
	<code>cublas</code>	15,3	30,7	40,8	102,0	163,2

Una prima lettura della tabella 7.7 mostra due comportamenti opposti. I tre kernel scritti nel progetto migliorano al crescere della taglia; `cublas` peggiora. Inoltre, per i kernel custom quasi tutto il guadagno si concentra nel primo raddoppio, da $S = 4096$ a $S = 8192$: il passaggio successivo aggiunge poco.

`cuda_warp`

Il caso più netto è `cuda_warp`. Alla scala più piccola il throughput è praticamente insensibile a k : resta fra 269,6 e 281,5 GFLOPS su tutto l'intervallo, cioè si muove di poco più di 10 GFLOPS mentre il lavoro cresce di dieci volte.

Alle due scale maggiori il comportamento cambia forma. Il valore a $k = 3$ resta isolato in basso (297,5 e 302,0 GFLOPS), mentre da $k = 6$ in poi il kernel si appiattisce su un plateau: 347,5–354,1 GFLOPS a $S = 8192$ e 353,2–354,9 GFLOPS a $S = 16384$.

Il confronto fra la prima e l'ultima scala si riassume in due numeri:

- per $k \geq 6$ il guadagno sta fra 1,26 e 1,29 $\times$;
- a $k = 3$ è solo di circa 1,12 $\times$.

Il punto importante non è però l'entità del guadagno, ma il fatto che a $S = 4096$ il plateau *non viene mai raggiunto*, per nessun valore di k . A $S = 4096$ e $k = 32$ il kernel fa 281,5 GFLOPS, contro i 354,9 dello stesso kernel e dello stesso k a $S = 16384$: la differenza non dipende dalla larghezza del multivettore, ma dalla taglia del problema.

Una taglia più piccola significa meno lavoro per singola invocazione e quindi meno righe di Y da produrre, cioè meno warp indipendenti disponibili. Le sole misure temporali non permettono però di attribuire il deficit a un meccanismo preciso (occupancy, latenza, comportamento delle cache o numero di blocchi residenti). L'affermazione che i dati sostengono direttamente è più semplice e più utile: `cuda_warp` raggiunge il proprio regime di plateau soltanto quando il problema è abbastanza grande, e $S = 4096$ è al di sotto di quella soglia.

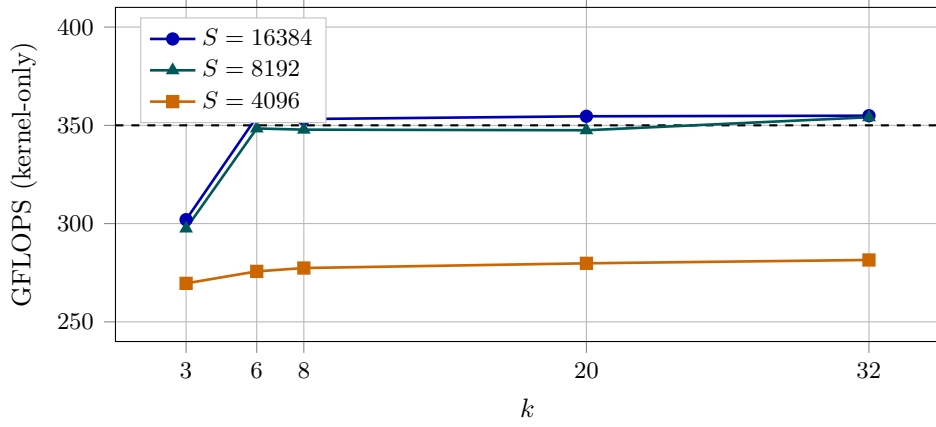


Figura 7.7: Fase 2: `cuda_warp` sul caso quadrato alle tre scale, dai valori della tabella 7.7. La linea tratteggiata indica il tetto FP64 nominale di 350 GFLOP/s utilizzato nel modello.

La figura rende visibile la differenza qualitativa: a $S = 4096$ la curva è piatta e lontana dal tetto, alle due scale maggiori sale subito e poi si appoggia sul tetto.

`cuda_warp_smem`

La variante con shared memory segue lo stesso andamento generale, ma in modo più graduale e senza mai superare `cuda_warp` sul caso quadrato.

A differenza degli altri due kernel custom, qui la crescita con la scala è monotona per ogni k : a $k = 3$ i tre valori sono 253,5, 291,5 e 296,8 GFLOPS; a $k = 32$ sono 274,5, 330,7 e 341,0 GFLOPS. Il throughput continua quindi ad aumentare anche nel passaggio da $S = 8192$ a $S = 16384$, dove `cuda_warp` si era già fermato.

Lo scarto rispetto a `cuda_warp` resta comunque sempre a favore di quest'ultimo. Alle due scale maggiori è piccolo, entro il 3% per $k \leq 20$; a $S = 4096$ è un po' più ampio e vale il 6,3% a $k = 3$ (269,6 contro 253,5). Fa eccezione $k = 32$, dove il divario si allarga anche sui problemi grandi (330,7 contro 354,1 a $S = 8192$ e 341,0 contro 354,9 a $S = 16384$).

Si nota anche, alle due scale maggiori, un piccolo calo passando da $k = 20$ a $k = 32$ ($338,6 \rightarrow 330,7$ e $349,5 \rightarrow 341,0$), che sulle altre due varianti non compare. È l'unico punto della campagna in cui lo staging esplicito di X in shared memory sembra costare più di quanto rende in FP64.

`cuda_naive`

Il kernel di riferimento interno ha un andamento meno regolare, e il motivo è concentrato tutto a $k = 3$.

Per $k \geq 8$ `cuda_naive` cresce con la scala e arriva dove arrivano gli altri: 342,0–349,7 GFLOPS a $S = 16384$, cioè sostanzialmente il valore di `cuda_warp`. A $k = 3$, invece, la successione è

$$162,3 \rightarrow 187,3 \rightarrow 151,5 \quad \text{GFLOPS},$$

cioè non monotona: il valore migliore si osserva alla scala intermedia e quello peggiore alla scala massima.

Il confronto fra i due kernel a $k = 3$ è quindi il risultato più stabile della tabella, e non si attenua crescendo: `cuda_warp` vale $1,66\times$ a $S = 4096$ e arriva a $1,99\times$ a $S = 16\,384$. Il vantaggio della mappatura a warp non è un throughput di picco più alto, ma il fatto di raggiungerlo già con multivettori stretti.

cublas

Il riferimento esterno si comporta in modo qualitativamente diverso da tutti i kernel custom, e la tabella lo mostra senza bisogno di altre misure.

Si prenda la riga $S = 16\,384$ e si parta da $k = 3$, dove il valore è $15,3$ GFLOPS. Moltiplicando per $k/3$ si ottiene $30,6$, $40,8$, $102,0$ e $163,2$, contro i $30,7$, $40,8$, $102,0$ e $163,2$ effettivamente misurati. Il throughput è quindi *proporzionale* a k , e poiché

$$\text{GFLOPS} = \frac{2MNk}{t_{\text{kernel}}},$$

questo equivale a dire che t_{kernel} non dipende da k . La stessa verifica riesce a $S = 8192$ e, da $k = 6$ in poi, anche a $S = 4096$.

Il costo della chiamata è dunque determinato dallo streaming di A , non dal lavoro effettivamente richiesto: con $k \leq 32$ una GEMM general-purpose paga quasi lo stesso tempo che pagherebbe per un multivettore molto più largo.

Sulla scala, cuBLAS perde invece throughput: a ogni k il valore a $S = 8192$ è circa l'89% di quello a $S = 4096$, e a $S = 16\,384$ resta lì. Il divario rispetto a `cuda_warp` sul problema più grande va da $19,7\times$ a $k = 3$ a $2,2\times$ a $k = 32$: si riduce al crescere di k , come previsto, ma non si annulla nell'intervallo richiesto dalla traccia.

7.2.3 Effetto della forma

La seconda domanda riguarda la geometria. Per ogni scala si confrontano le tre forme $M = N$, $N = 2M$ e $M = 3N$, che come detto hanno lo stesso MN entro l'1,2%.

Il confronto va letto così: a parità di scala e di k , il numero di operazioni è lo stesso in tutte e tre le righe di una tabella, quindi ogni differenza verticale è sensibilità alla forma. Un modo compatto per riassumere ciascun blocco è lo *scarto* fra la forma migliore e la peggiore, espresso in percentuale della peggiore.

cuda_warp**Tabella 7.8:** Fase 2: `gflops_kernel` di `cuda_warp` sulle tre forme e sulle tre scale.

Scala	Forma	$k = 3$	$k = 6$	$k = 8$	$k = 20$	$k = 32$
$S = 4096$	$M = N$	269,6	275,7	277,4	279,8	281,5
	$N = 2M$	280,8	308,9	310,4	289,0	292,0
	$M = 3N$	285,7	330,0	330,5	278,0	281,4
$S = 8192$	$M = N$	297,5	348,4	347,8	347,5	354,1
	$N = 2M$	296,6	353,5	351,5	354,7	357,5
	$M = 3N$	297,8	343,1	342,7	345,9	347,6
$S = 16384$	$M = N$	302,0	354,3	353,2	354,6	354,9
	$N = 2M$	300,0	356,0	355,8	357,4	356,7
	$M = 3N$	301,7	349,6	349,8	351,9	352,4

Alla scala più piccola la forma conta molto. A $S = 4096$ e $k = 6$ il throughput passa da 275,7 GFLOPS nel caso quadrato a 308,9 per $N = 2M$ e 330,0 per $M = 3N$; a $k = 8$ l'andamento è quasi identico ($277,4 \rightarrow 310,4 \rightarrow 330,5$). In entrambi i casi lo scarto fra la forma migliore e la peggiore è vicino al 20%.

Non esiste però una forma sempre vincente, nemmeno a scala fissa. Sulla stessa riga di scala, a $k = 20$ la migliore è $N = 2M$ (289,0 contro 279,8 e 278,0) e a $k = 32$ vale lo stesso (292,0 contro 281,5 e 281,4). La forma $M = 3N$, che era la migliore ai k stretti, diventa la peggiore ai k larghi. L'effetto della geometria dipende quindi anche da k .

Crescendo di scala la dispersione crolla. A $S = 8192$ lo scarto massimo su tutti i k è del 3,1%, a $S = 16384$ dell'1,8%: a $k = 6$, per esempio, i tre valori sono 354,3, 356,0 e 349,6 GFLOPS.

Questo comportamento si lega alla struttura del kernel. `cuda_warp` assegna un warp a ogni riga di Y , quindi cambiare il rapporto $M:N$ a prodotto costante modifica due cose insieme: quante righe indipendenti esistono, e quanto è lunga la riduzione lungo N che ciascun warp deve percorrere. Nel caso $M = 3N$ ci sono più righe ma riduzioni più corte; nel caso $N = 2M$ avviene il contrario.

La campagna non permette di separare quantitativamente i due effetti, e il vantaggio di $M = 3N$ a $S = 4096$ non va quindi letto come prova che “più righe è meglio”. Il risultato solido è un altro: quando il problema è abbastanza grande da fornire comunque parallelismo in abbondanza, la forma smette praticamente di contare.

cuda_naive**Tabella 7.9:** Fase 2: `gflops_kernel` di `cuda_naive` sulle tre forme e sulle tre scale.

Scala	Forma	$k = 3$	$k = 6$	$k = 8$	$k = 20$	$k = 32$
$S = 4096$	$M = N$	162,3	252,0	253,2	277,2	287,7
	$N = 2M$	149,9	274,2	329,1	275,5	300,7
	$M = 3N$	165,9	297,7	324,5	283,5	295,8
$S = 8192$	$M = N$	187,3	309,7	342,5	348,7	354,5
	$N = 2M$	160,3	284,8	326,0	345,6	356,5
	$M = 3N$	140,1	296,0	341,4	346,8	354,4
$S = 16384$	$M = N$	151,5	307,1	342,0	345,5	349,7
	$N = 2M$	134,0	277,4	349,9	342,0	347,6
	$M = 3N$	132,6	289,2	348,2	342,9	346,1

`cuda_naive` è il backend più sensibile alla forma, e soprattutto è l'unico in cui la sensibilità *non* sparisce crescendo di scala: cambia solo il valore di k in cui si manifesta.

A $S = 4096$ il massimo scarto è del 30,0% ed è a $k = 8$ (253,2 nel caso quadrato contro 329,1 per $N = 2M$): le forme rettangolari sono molto migliori.

A $S = 8192$ e $S = 16384$ il segno si rovescia e il fenomeno si sposta a $k = 3$. Alla scala intermedia lo scarto è del 33,7%, il più grande dell'intera campagna (187,3 per il quadrato contro 140,1 per $M = 3N$); alla scala massima è ancora del 14,2% (151,5 contro 132,6). Adesso è il caso quadrato a essere nettamente il migliore.

Per $k \geq 20$, invece, le tre forme si avvicinano molto: lo scarto è del 4,5% a $S = 4096$ e scende sotto l'1% alle due scale maggiori.

La lettura complessiva è coerente con quanto visto sulla scala: il punto debole di `cuda_naive` sono i multivettori stretti, e proprio lì il kernel resta esposto alla geometria del problema anche quando il problema è grande.

`cuda_warp_smem`

Tabella 7.10: Fase 2: `gflops_kernel` di `cuda_warp_smem` sulle tre forme e sulle tre scale.

Scala	Forma	$k = 3$	$k = 6$	$k = 8$	$k = 20$	$k = 32$
$S = 4096$	$M = N$	253,5	265,8	267,5	273,5	274,5
	$N = 2M$	272,5	316,9	318,6	268,5	262,0
	$M = 3N$	282,3	312,3	319,2	268,5	263,0
$S = 8192$	$M = N$	291,5	338,3	337,6	338,6	330,7
	$N = 2M$	291,4	347,9	348,3	349,6	342,4
	$M = 3N$	293,6	338,0	338,7	341,5	333,8
$S = 16384$	$M = N$	296,8	349,8	349,6	349,5	341,0
	$N = 2M$	289,7	349,7	349,6	350,3	341,6
	$M = 3N$	295,8	346,9	345,8	347,3	338,8

La variante con shared memory riproduce la convergenza già vista per `cuda_warp`: scarto massimo del 19,3% a $S = 4096$, del 3,5% a $S = 8192$ e del 2,4% a $S = 16384$.

Alla scala piccola, però, l'effetto cambia segno con k in modo molto marcato. A $k = 6$ e $k = 8$ le forme rettangolari vincono nettamente ($267,5 \rightarrow 318,6$ e $319,2$ a $k = 8$), mentre a $k = 20$ e $k = 32$ perdono e il caso quadrato diventa il migliore (274,5 contro 262,0 e 263,0 a $k = 32$).

Alla scala intermedia emerge invece una regolarità pulita: per $k \geq 6$ la forma $N = 2M$ è la migliore a tutti i valori di k , con un vantaggio di circa il 3% (a $k = 3$ le tre forme sono già indistinguibili, entro lo 0,7%). Alla scala massima anche questo vantaggio si assottiglia e le tre forme diventano intercambiabili.

Va sottolineato un punto che riguarda l'interpretazione. La configurazione del backend resta identica lungo tutta la campagna: a parità di k , il numero di blocchi residenti per SM e il numero di righe di X caricate per tile non cambiano al variare del rapporto $M:N$. Le differenze osservate fra le forme non sono quindi riconducibili a una diversa scelta del piano di tiling.

cublas**Tabella 7.11:** Fase 2: `gflops_kernel` di cublas sulle tre forme e sulle tre scale.

Scala	Forma	$k = 3$	$k = 6$	$k = 8$	$k = 20$	$k = 32$
$S = 4096$	$M = N$	16,5	34,6	46,1	115,2	184,3
	$N = 2M$	13,6	32,5	43,4	108,2	173,4
	$M = 3N$	16,0	32,0	42,7	106,6	170,8
$S = 8192$	$M = N$	15,4	30,8	41,1	102,7	164,0
	$N = 2M$	16,3	32,5	43,4	108,2	173,5
	$M = 3N$	15,9	31,9	42,5	106,1	169,6
$S = 16384$	$M = N$	15,3	30,7	40,8	102,0	163,2
	$N = 2M$	16,0	32,1	42,8	106,9	170,9
	$M = 3N$	15,8	31,6	42,1	105,3	168,5

cuBLAS è l'unico backend in cui la sensibilità alla forma *non* si riduce crescendo di scala. Lo scarto resta intorno all'8% a $S = 4096$ (per $k \geq 6$), al 5,7% a $S = 8192$ e al 4,9% a $S = 16384$.

Cambia però quale forma vince. A $S = 4096$ il caso quadrato è il migliore a tutti i k ; alle due scale maggiori diventa il *peggiore*, e la forma $N = 2M$ passa davanti (170,9 contro 163,2 GFLOPS a $k = 32$, $S = 16384$). Vale la pena notare che $N = 2M$ è anche la forma con l'1,12% di elementi in meno: il vantaggio osservato è quattro volte più grande di quella differenza, quindi non è un artefatto del disegno.

Una parte della spiegazione è nel modo in cui il backend chiama la libreria. Il prodotto row-major viene realizzato reinterpretando i buffer secondo la convenzione column-major, e per $P = 1$ la GEMM riceve dimensioni

$$m_{\text{GEMM}} = k, \quad n_{\text{GEMM}} = M, \quad K_{\text{GEMM}} = N.$$

Cambiare il rapporto $M:N$ cambia quindi direttamente la geometria vista dalla routine. I soli tempi non dicono però quale kernel interno o quale tiling venga selezionato: una spiegazione microarchitetturale richiederebbe una profilazione dedicata.

Confronto fra i backend

Mettendo insieme le quattro tabelle, la robustezza alla forma si riassume nello scarto massimo osservato su tutti i k di una stessa scala:

Backend	$S = 4096$	$S = 8192$	$S = 16384$
<code>cuda_naive</code>	30,0%	33,7%	14,2%
<code>cuda_warp</code>	19,7%	3,1%	1,8%
<code>cuda_warp_smem</code>	19,3%	3,5%	2,4%
<code>cublas</code>	21,4%	5,7%	4,9%

I valori sono calcolati dalle tabelle precedenti come $(\max - \min) / \min$ fra le tre forme, prendendo poi il caso peggiore sui cinque k .

Alla scala più piccola tutti e quattro i backend sono sensibili alla geometria in misura confrontabile, fra il 19% e il 30%. È crescendo che i comportamenti si separano: i due kernel a mappatura warp diventano praticamente indifferenti alla forma, cuBLAS si stabilizza intorno al 5% e `cuda_naive` resta esposto per via del solo $k = 3$.

7.2.4 Sintesi della Fase 2

La Fase 2 porta tre conclusioni.

La scala conta, e conta una volta sola. Per i kernel custom il salto prestazionale avviene fra $S = 4096$ e $S = 8192$; oltre, il guadagno è marginale. `cuda_warp` guadagna fra 1,26 e $1,29\times$ da $S = 4096$ a $S = 16384$ per $k \geq 6$, ma quasi tutto nel primo raddoppio. Il problema a $S = 4096$ appartiene a un regime diverso: il plateau non viene raggiunto per nessun k .

La forma conta soprattutto quando la scala è insufficiente. Le differenze vicine al 20% osservate a $S = 4096$ si riducono sotto il 2% per `cuda_warp` a $S = 16384$. Non si tratta però di una regola valida per tutti i backend: `cuda_naive` conserva un 14,2% di scarto a $k = 3$ anche alla scala massima, e cuBLAS resta intorno al 5% a ogni scala. Scala, forma e larghezza del multivettore vanno quindi considerate insieme, non come fattori indipendenti.

Il comportamento di cuBLAS è strutturalmente diverso. Il throughput proporzionale a k mostra che il tempo di kernel della libreria non dipende dal lavoro richiesto: per $k \leq 32$ una GEMM general-purpose non ammortizza il proprio percorso. Questo, e non un difetto di ottimizzazione, è ciò che i kernel custom sfruttano.

Su questa base la Fase 3 mantiene il problema

$$M = N = 16384.$$

La scelta non implica che la geometria quadrata massima sia ottimale per ogni backend. Fornisce però un punto in cui `cuda_warp` è nel proprio plateau (circa 355 GFLOPS per $k \geq 6$, cioè poco oltre il tetto FP64 nominale di 350 GFLOP/s) ed è al tempo stesso il più insensibile alla forma. Fissare quella geometria permette alla Fase 3 di isolare l'effetto della precisione numerica senza cambiare contemporaneamente dimensione e forma del problema. Nella Fase 4 la scala torna invece a essere un fattore sperimentale, per osservare come la composizione MPI + CUDA dipenda dalla dimensione del problema globale. dipenda anche dalla dimensione del problema globale.

7.3 Fase 3 - Effetto della precisione numerica

La terza fase studia l'effetto della precisione numerica sulle prestazioni dei quattro backend GPU.

Il problema viene mantenuto fisso a

$$M = N = 16384, \quad P = 1,$$

utilizzando per ciascun backend le configurazioni consolidate nelle fasi precedenti. Anche l'insieme dei valori di k rimane invariato:

$$k \in \{3, 6, 8, 20, 32\}.$$

Il confronto viene eseguito fra doppia precisione (`double`, FP64) e singola precisione (`float`, FP32). I risultati FP64 corrispondono al punto quadrato $S = 16384$ della Fase 2, mentre le misure FP32 sono state ottenute con lo stesso protocollo sperimentale.

Per quantificare il beneficio del cambio di precisione si utilizza lo speedup

$$S_{32/64} = \frac{t_{\text{kernel}}^{\text{FP64}}}{t_{\text{kernel}}^{\text{FP32}}}, \quad (7.1)$$

calcolato sui tempi del solo kernel. Un valore maggiore di uno indica quindi una riduzione del tempo di esecuzione passando da FP64 a FP32.

7.3.1 Confronto tra FP64 e FP32

La tabella 7.12 riporta il throughput ottenuto nelle due precisioni e lo speedup corrispondente.

Tabella 7.12: Fase 3: `gflops_kernel` nelle due precisioni e speedup $S_{32/64}$, con $M = N = 16384$ e $P = 1$.

Backend	$k = 3$	$k = 6$	$k = 8$	$k = 20$	$k = 32$
<i>GFLOPS in FP32</i>					
<code>cuda_naive</code>	198,3	386,8	522,0	791,6	1107,4
<code>cuda_warp</code>	604,4	1169,8	1414,3	1489,3	1945,9
<code>cuda_warp_smem</code>	601,2	1164,7	1457,2	1726,9	1473,0
<code>cublas</code>	426,7	861,1	1141,0	2962,3	4791,9
<i>GFLOPS in FP64</i>					
<code>cuda_naive</code>	151,5	307,1	342,0	345,5	349,7
<code>cuda_warp</code>	302,0	354,3	353,2	354,6	354,9
<code>cuda_warp_smem</code>	296,8	349,8	349,6	349,5	341,0
<code>cublas</code>	15,3	30,7	40,8	102,0	163,2
$S_{32/64} = t_{\text{kernel}}^{\text{FP64}}/t_{\text{kernel}}^{\text{FP32}}$					
<code>cuda_naive</code>	1,31	1,26	1,53	2,29	3,17
<code>cuda_warp</code>	2,00	3,30	4,00	4,20	5,48
<code>cuda_warp_smem</code>	2,03	3,33	4,17	4,94	4,32
<code>cublas</code>	27,89	28,08	27,97	29,04	29,37

Il primo risultato della campagna è che il beneficio del passaggio a FP32 non è costante. Lo speedup dipende sia dal backend sia dalla larghezza k del multivettore.

Per `cuda_naive` il guadagno rimane limitato per i valori più piccoli di k : vale $1,31 \times$ a $k = 3$ e $1,26 \times$ a $k = 6$. Aumentando k , il vantaggio diventa progressivamente più significativo e raggiunge $3,17 \times$ a $k = 32$.

Il comportamento di `cuda_warp` è più marcato. Lo speedup parte da esattamente $2,00 \times$ a $k = 3$, cresce a $3,30 \times$ a $k = 6$ e $4,00 \times$ a $k = 8$, fino a raggiungere $5,48 \times$ a $k = 32$.

Il passaggio dalla doppia alla singola precisione non può quindi essere descritto, per i kernel custom, mediante un unico fattore moltiplicativo. L'effetto dipende dal modo in cui ciascun kernel utilizza le risorse della GPU e varia sensibilmente con k .

7.3.2 Comportamento dei kernel custom

Il confronto fra i tre kernel sviluppati nel progetto mostra che il cambio di precisione modifica anche i rapporti prestazionali osservati in FP64.

In doppia precisione, per $k \geq 6$, `cuda_warp` e `cuda_warp_smem` risultavano molto vicini, con throughput nell'intorno dei 350 GFLOPS. In FP32 il comportamento si differenzia maggiormente.

Per $k = 3, 6, 8$ i due kernel warp rimangono praticamente indistinguibili, di fatto la differenza rimane contenuta.

Il comportamento cambia invece a $k = 20$. In questo caso `cuda_warp_smem` raggiunge

1726,9 GFLOPS,

contro

1489,3 GFLOPS

di `cuda_warp`, con un vantaggio di circa il 16%.

Il risultato è rilevante perché in FP64 lo staging di X in shared memory non aveva prodotto un vantaggio prestazionale rispetto alla variante `cuda_warp`. Il cambio di precisione modifica quindi il regime nel quale viene eseguito il kernel e rende conveniente, almeno per alcune configurazioni, la strategia basata sulla shared memory.

Il vantaggio non è tuttavia uniforme. A $k = 32$ il comportamento si ribalta:

`cuda_warp` = 1945,9 GFLOPS,

mentre

`cuda_warp_smem` = 1473,0 GFLOPS.

La variante shared risulta quindi circa il 24% più lenta di `cuda_warp`.

7.3.3 Piano di esecuzione di `cuda_warp_smem`

Per interpretare il cambiamento osservato nella variante shared è utile considerare il piano di tiling scelto a runtime nelle due precisioni.

Tabella 7.13: Fase 3: piano di tiling scelto a runtime da `cuda_warp_smem` nelle due precisioni.

k	x_rows_per_tile		blocks_per_sm	
	FP64	FP32	FP64	FP32
3	512	1344	4	3
6	288	576	4	4
8	224	448	4	4
20	128	192	3	4
32	96	160	2	3

Il passaggio a FP32 consente generalmente di utilizzare tile più grandi, poiché ciascun elemento di X occupa metà dello spazio rispetto alla rappresentazione FP64.

Il caso $k = 20$ è particolarmente interessante. In FP32 il planner seleziona 192 righe per tile e consente la residenza di 4 blocchi per SM, contro 128 righe e 3 blocchi per SM in FP64. Questa configurazione coincide con il punto nel quale `cuda_warp_smem` mostra il vantaggio maggiore rispetto a `cuda_warp`.

A $k = 32$ il piano FP32 utilizza invece 160 righe per tile e 3 blocchi per SM. Rispetto al punto $k = 20$ diminuisce quindi il numero di blocchi residenti per SM, contemporaneamente all'aumento della quantità di stato richiesta per elaborare un multivettore più largo.

Questa correlazione è compatibile con il calo di throughput osservato a $k = 32$.

7.3.4 Comportamento di cuBLAS

Il backend `cublas` presenta un comportamento nettamente diverso dai kernel custom.

In FP64 il throughput rimane molto basso su tutto l'intervallo considerato: passa da 15,3 GFLOPS a $k = 3$ a 163,2 GFLOPS a $k = 32$.

In FP32, invece, i valori diventano

426,7, 861,1, 1141,0, 2962,3, 4791,9 GFLOPS.

Lo speedup FP32/FP64 rimane inoltre quasi costante, compreso fra circa $28\times$ e $29\times$ per tutti i valori di k .

Questo valore non deve essere interpretato come un generico beneficio della singola precisione pari a circa $28\times$. Esso descrive il rapporto prestazionale osservato, sul problema considerato, fra le esecuzioni FP32 e FP64 di `cublas`. Il fatto che il rapporto rimanga quasi costante mostra inoltre che la forte differenza fra le due precisioni si mantiene su tutto l'intervallo di k analizzato.

Il confronto con i kernel custom cambia anch'esso con la precisione. In FP64 le implementazioni sviluppate nel progetto risultano sensibilmente più veloci di `cublas` per tutti i valori di k considerati. In FP32, invece, `cublas` rimane più lento per i valori minori di k , ma supera i kernel custom per $k = 20$ e $k = 32$.

A $k = 20$, `cublas` raggiunge

2962,3 GFLOPS,

contro 1726,9 GFLOPS del migliore kernel custom nello stesso punto. A $k = 32$ il divario aumenta ulteriormente:

4791,9 GFLOPS

per `cublas`, contro

1945,9 GFLOPS

di `cuda_warp`.

Il passaggio alla singola precisione modifica quindi non soltanto il throughput assoluto dei backend, ma anche il loro ordinamento relativo per i valori maggiori di k .

7.3.5 Sintesi

La campagna mostra che l'effetto della precisione numerica dipende fortemente dall'implementazione e dalla larghezza del multivettore.

Per i kernel custom lo speedup FP32/FP64 tende generalmente ad aumentare con k , ma con comportamenti differenti. `cuda_naive` presenta il guadagno più contenuto, mentre `cuda_warp` raggiunge $5,48\times$ a $k = 32$.

La variante `cuda_warp_smem` evidenzia inoltre che il cambio di precisione può modificare la convenienza delle scelte implementative: lo staging di X in shared memory produce un vantaggio rispetto a `cuda_warp` a $k = 20$, mentre a $k = 32$ la variante `cuda_warp` torna a essere più veloce.

Il risultato più marcato riguarda infine cuBLAS. La libreria passa da prestazioni sensibilmente inferiori ai kernel custom in FP64 a prestazioni superiori in FP32 per i valori maggiori di k .

Nel complesso, i risultati mostrano che la scelta fra FP32 e FP64 non produce un semplice fattore di accelerazione uniforme. La precisione modifica in modo significativo il comportamento relativo delle diverse implementazioni e può cambiare quale strategia risulti più prestante nel regime sperimentale considerato.

7.4 Fase 4 - Esecuzione distribuita MPI + CUDA

La quarta fase raccoglie 600 configurazioni in doppia precisione, ottenute da 3 scale, 4 backend, 5 valori di k e tutte le griglie ammissibili per $P \in \{1, 2, 4, 8\}$.

Il punto di partenza è un vincolo dell'hardware, non una scelta: il nodo ha una sola GPU, e tutti i rank la usano. Aumentare P non aggiunge quindi capacità di calcolo. Suddivide soltanto il problema in chiamate locali più piccole e introduce contesa fra contesti CUDA, che senza MPS si alternano sulla scheda.

Per questo motivo tutte le esecuzioni includono il *group timing*, che misura il tempo necessario affinché *tutti* i rank completino la fase locale. La metrica usata in questa fase è di conseguenza `gflops_local_group`: è l'unica che descrive il lavoro effettivamente sostenuto dal device, come la prima sottosezione mostra con i numeri.

L'analisi procede in tre passi:

- Si verifica prima quale metrica sia lecito usare;
- si studia l'effetto del numero di rank P ;
- infine quello della forma della griglia $P_r \times P_c$ a parità di P .

Un'ultima sottosezione confronta i quattro backend separando i costi che la composizione aggiunge al kernel.

7.4.1 Il tempo collettivo come metrica

La tabella 7.14 mostra perché il tempo associato a un singolo rank non è sufficiente quando più contesti condividono l'acceleratore. Il rapporto confronta il throughput derivato dalla metrica ufficiale con quello derivato dalla finestra collettiva. Il coefficiente di variazione (CV) è calcolato sulle 20 repetition del tempo di gruppo.

Tabella 7.14: Fase 4: confronto fra throughput derivato dai tempi rank-local e throughput della fase locale collettiva per casi rappresentativi di `cuda_warp`.

S	P	griglia	k	gflops	gflops_local_group	rapporto	CV gruppo
4096	8	2×4	3	919,2	78,4	11,73	0,38%
8192	8	2×4	3	1025,6	172,4	5,95	0,26%
16384	8	1×8	3	465,9	250,5	1,86	0,11%
16384	8	2×4	32	296,1	298,5	0,99	0,44%

Lettura

Un tempo di kernel associato a un rank non diventa un tempo globale quando più contesti condividono la GPU. A $S = 4096$, $P = 8$ e $k = 3$ la metrica rank-local produce 919,2 GFLOPS, mentre la finestra collettiva ne misura 78,4. Il primo valore supera di oltre undici volte il secondo e non può essere interpretato come throughput sostenuto dal device. Il singolo rank osserva soltanto il proprio tratto di esecuzione; il gruppo deve attendere il completamento di tutti i contesti.

La discrepanza non è rumore statistico. Nel caso più estremo il CV del tempo di gruppo è appena lo 0,38%. Sulle 600 configurazioni il CV mediano vale 0,30% e il novantesimo percentile 1,63%. Il fenomeno è quindi riproducibile e descrive una differenza semantica fra le metriche, non una fluttuazione.

Le due letture convergono quando ogni invocazione è abbastanza lunga. A $S = 16384$ e $k = 32$ i due throughput coincidono entro l'1%. Questo non rende la misura rank-local collettiva: indica soltanto che in quel regime la schedulazione dei contesti non crea una finestra invisibile al rank più lento.

Tutte le tabelle che seguono riportano perciò `gflops_local_group`. Dove compare anche la metrica ufficiale, è per mostrarne la differenza, non per sostituirla.

7.4.2 Effetto del numero di rank P

Il numero di rank e la forma della griglia sono due fattori distinti, e vanno separati. Per isolare P si usa una sola famiglia di griglie, scelta in modo che ogni raddoppio di P raddoppi una sola delle due dimensioni: 1×1 , 2×1 , 2×2 , 4×2 .

La tabella 7.15 riporta il throughput collettivo dei quattro backend sulle tre scale, ai due estremi dell'intervallo di k .

Tabella 7.15: Fase 4: throughput della fase locale collettiva (`gflops_local_group`), in GFLOPS, per i quattro backend al variare della scala e del numero di rank. Griglie della famiglia coerente 1×1 , 2×1 , 2×2 , 4×2 .

Backend	P	$S = 4096$		$S = 8192$		$S = 16384$	
		$k = 3$	$k = 32$	$k = 3$	$k = 32$	$k = 3$	$k = 32$
cuda_naive	1	145,1	250,7	175,7	327,1	146,6	331,3
	2	97,6	270,0	174,1	290,1	166,7	301,9
	4	93,8	288,1	161,8	296,3	166,4	302,3
	8	56,0	240,3	114,8	305,0	176,4	293,1
cuda_warp	1	235,2	253,6	273,5	326,1	286,9	335,5
	2	160,8	275,9	243,3	292,5	247,9	307,3
	4	118,1	270,8	217,0	291,1	273,0	305,5
	8	78,2	227,5	166,7	295,5	251,9	295,5
cuda_warp_smem	1	232,8	236,3	267,8	304,1	277,3	324,4
	2	156,7	245,8	239,7	278,3	251,0	289,4
	4	114,5	240,5	213,9	279,7	253,2	285,0
	8	75,8	214,9	164,8	262,8	249,1	281,6
cublas	1	16,8	166,0	15,1	156,3	15,0	157,9
	2	13,2	134,5	15,7	163,0	13,9	146,5
	4	13,2	137,7	15,8	164,0	13,9	146,3
	8	14,4	148,6	13,2	137,0	15,6	163,5

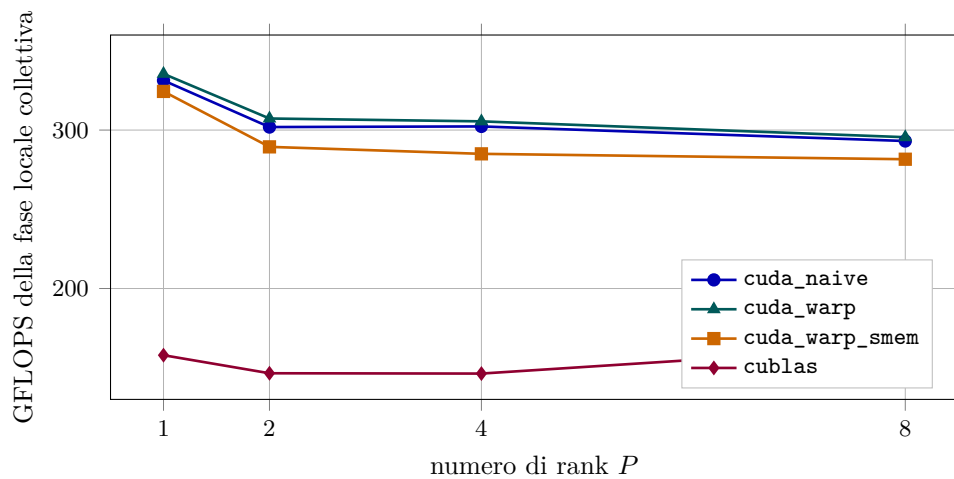


Figura 7.8: Fase 4: throughput collettivo a $S = 16384$ e $k = 32$ per i quattro backend, dalla colonna corrispondente della tabella 7.15.

Lettura

Non compare strong scaling, e non potrebbe comparire. Il lavoro globale è invariato e la GPU è una sola: un incremento proporzionale a P sarebbe impossibile. A $S = 16384$ e $k = 32$ tutti e tre i kernel custom perdono invece qualcosa passando da uno a otto rank: `cuda_warp` scende da 335,5 a 295,5 GFLOPS (−12%), `cuda_naive` da 331,3 a 293,1 (−12%) e `cuda_warp_smem` da 324,4 a 281,6 (−13%). La perdita quantifica il costo della composizione: alternanza fra contesti CUDA, blocchi locali più piccoli e trasferimenti concorrenti.

Quello che conta non è P , ma quanto lavoro resta a ciascun rank. Lo stesso $P = 8$ produce effetti completamente diversi a seconda della scala. A $S = 4096$ e $k = 3$ il crollo è verticale: `cuda_warp` passa da 235,2 a 78,2 GFLOPS (−67%), `cuda_warp_smem` da 232,8 a 75,8 e `cuda_naive` da 145,1 a 56,0. Alla scala massima, con lo stesso k e lo stesso P , `cuda_warp`

perde invece solo il 12% (da 286,9 a 251,9). Nel primo caso ogni rank riceve un ottavo di un problema già piccolo, e i costi fissi diventano dominanti.

In alcune configurazioni dividere il problema conviene davvero. Il caso più netto è `cuda_naive` a $S = 16384$ e $k = 3$, che *sale* da 146,6 a 176,4 GFLOPS con otto rank, un guadagno del 20%. Lo stesso backend a $S = 4096$ e $k = 32$ passa da 250,7 a 288,1 GFLOPS con quattro rank (+15%), e cuBLAS a $S = 16384$ e $k = 32$ da 157,9 a 163,5 (+4%).

Sono esattamente le configurazioni che nelle fasi precedenti risultavano le meno efficienti a rank singolo: `cuda_naive` con multivettore stretto e cuBLAS in qualunque regime. Dove un solo rank non riesce a tenere occupata la GPU, più contesti che si alternano la riempiono meglio. Le misure temporali raccolte non permettono però di attribuire il guadagno a un meccanismo specifico, e il confronto resta descrittivo.

cuBLAS non segue alcun andamento regolare in P . I suoi valori a $k = 32$ oscillano fra 134,5 e 166,0 GFLOPS su tutte le scale, senza direzione definita e senza dipendenza chiara da P . La sottosezione successiva mostra il motivo: per cuBLAS pesa molto più la forma del blocco locale che il numero di rank.

7.4.3 Effetto della griglia MPI

A parità di P la griglia $P_r \times P_c$ decide due cose contemporaneamente.

La prima è quale collettiva sostiene il costo maggiore. Il broadcast di X avviene lungo le colonne della griglia e il suo costo cresce con P_r , mentre la reduce dei contributi parziali di Y avviene lungo le righe e cresce con P_c . Le due griglie monodimensionali sono quindi i due estremi: 1×8 azzerà il broadcast e paga tutta la reduce, 8×1 fa l'opposto.

La seconda è la forma del blocco locale, $m_{\text{loc}} = M/P_r$ e $n_{\text{loc}} = N/P_c$. Cambiare griglia cambia quindi anche la geometria del problema che ciascun kernel riceve, esattamente il fattore studiato nella Fase 2.

La tabella 7.16 scompone il tempo dei quattro backend sul punto di riferimento $S = 16384$, $P = 8$, $k = 32$.

Tabella 7.16: Fase 4: scomposizione temporale dei quattro backend per le quattro griglie a $S = 16384$, $P = 8$, $k = 32$. Tempi in millisecondi.

Backend	$P_r \times P_c$	t_{bcast}	t_{kernel}	t_{reduce}	T	$t_{\text{gruppo}}^{\text{local}}$	GFLOPS	GFLOPS gruppo
<code>cuda_naive</code>	1×8	0,000	55,010	8,875	63,250	57,906	271,6	296,7
	2×4	0,374	54,995	2,220	57,385	56,776	299,4	302,6
	4×2	1,506	56,318	0,581	57,786	58,617	297,3	293,1
	8×1	1,408	55,719	0,218	57,322	59,487	299,7	288,8
<code>cuda_warp</code>	1×8	0,000	58,763	8,635	66,000	60,481	260,3	284,1
	2×4	0,342	55,736	2,085	58,022	57,555	296,1	298,5
	4×2	1,586	55,881	0,604	57,453	58,145	299,0	295,5
	8×1	3,227	55,049	0,211	58,443	58,496	294,0	293,7
<code>cuda_warp_smem</code>	1×8	0,000	61,158	8,652	67,886	62,850	253,1	273,3
	2×4	0,379	60,132	2,201	61,918	61,974	277,5	277,2
	4×2	1,356	58,684	0,593	59,954	61,000	286,6	281,6
	8×1	1,543	63,288	0,206	65,012	66,868	264,3	256,9
<code>cublas</code>	1×8	0,001	115,798	8,672	121,719	117,440	141,1	146,3
	2×4	0,406	115,790	2,319	118,181	117,521	145,4	146,2
	4×2	1,401	102,778	0,595	104,035	105,093	165,1	163,5
	8×1	1,499	121,909	0,240	123,607	125,604	139,0	136,8

Lettura

Il costo si sposta da una collettiva all'altra, come previsto. Per `cuda_warp` il broadcast cresce da 0,000 a 3,227 ms passando da 1×8 a 8×1 , mentre la reduce scende da 8,635 a 0,211 ms. Gli altri tre backend mostrano lo stesso ribaltamento sulle stesse griglie: è una proprietà della decomposizione, non del kernel.

Le due collettive non costano però lo stesso. La reduce di 1×8 vale 8,635 ms per `cuda_warp`, il broadcast di 8×1 solo 3,227 ms. Il motivo è nel contenuto dei messaggi: la reduce combina blocchi di Y e richiede un'operazione aritmetica su ogni elemento, il broadcast replica soltanto X . Le griglie bilanciate 2×4 e 4×2 evitano entrambi gli estremi e restano sotto i 2,5 ms complessivi.

Per i kernel custom la fase CUDA è quasi indifferente alla griglia, per cuBLAS no. Il tempo di kernel di `cuda_naive` varia solo del 2% fra le quattro griglie e quello di `cuda_warp` del 7%. Per cuBLAS invece si passa da 102,778 ms sulla griglia 4×2 a 121,909 ms su 8×1 , cioè il 19% in più. La libreria è sensibile alla forma del blocco locale, che qui va da 2048×16384 (8×1) a 4096×8192 (4×2): è lo stesso effetto osservato nella Fase 2, che ricompare dentro la decomposizione MPI.

La metrica ufficiale e quella di gruppo divergono soprattutto su 1×8 . Per `cuda_warp` valgono 260,3 e 284,1 GFLOPS, uno scarto del 9%; sulle altre tre griglie coincidono entro l'1%. La griglia che concentra il costo sulla reduce è anche quella che rende il tempo del singolo rank meno rappresentativo del tempo del gruppo.

La scomposizione completa non viene ripetuta per ogni k : sarebbero ottanta righe in cui la colonna del kernel domina tutte le altre e si muove poco. È più informativo separare le due componenti che con k cambiano davvero. La tabella 7.17 riporta il costo complessivo delle due collettive, e la tabella 7.18 la sua conseguenza sul throughput.

Tabella 7.17: Fase 4: costo complessivo delle due collettive, $t_{\text{bcast}} + t_{\text{reduce}}$, in millisecondi, a $S = 16384$ e $P = 8$, per ogni griglia e ogni k .

Backend	$P_r \times P_c$	$k = 3$	$k = 6$	$k = 8$	$k = 20$	$k = 32$
<code>cuda_naive</code>	1×8	0,392	0,916	1,278	6,558	8,876
	2×4	0,149	0,378	0,438	1,616	2,594
	4×2	0,106	0,231	0,293	1,271	2,087
	8×1	0,218	0,440	0,601	1,132	1,625
<code>cuda_warp</code>	1×8	0,373	0,867	1,317	5,937	8,636
	2×4	0,146	0,386	0,429	1,657	2,428
	4×2	0,101	0,209	0,302	1,282	2,190
	8×1	0,219	0,438	0,604	1,966	3,438
<code>cuda_warp_smem</code>	1×8	0,392	0,889	1,265	6,671	8,653
	2×4	0,145	0,378	0,439	1,702	2,580
	4×2	0,108	0,219	0,291	1,261	1,949
	8×1	0,232	0,448	0,602	1,897	1,749
<code>cublas</code>	1×8	0,439	0,932	1,286	6,123	8,673
	2×4	0,173	0,405	0,493	1,621	2,725
	4×2	0,122	0,238	0,325	1,351	1,997
	8×1	0,239	0,497	0,658	1,975	1,739

Tabella 7.18: Fase 4: throughput collettivo (`gflops_local_group`) a $S = 16\,384$ e $P = 8$, per ogni griglia e ogni k .

Backend	$P_r \times P_c$	$k = 3$	$k = 6$	$k = 8$	$k = 20$	$k = 32$
cuda_naive	1×8	177,9	286,8	307,7	303,9	296,7
	2×4	168,5	279,6	309,8	305,3	302,6
	4×2	176,4	272,9	281,1	285,9	293,1
	8×1	140,5	251,5	279,9	277,1	288,8
cuda_warp	1×8	250,5	291,0	295,4	280,0	284,1
	2×4	254,3	305,7	310,6	303,5	298,5
	4×2	251,9	304,6	309,8	302,1	295,5
	8×1	239,8	303,0	307,4	290,7	293,7
cuda_warp_smem	1×8	249,4	288,2	292,7	276,2	273,3
	2×4	249,2	293,2	297,9	285,2	277,2
	4×2	249,1	297,3	303,6	296,8	281,6
	8×1	233,6	272,1	276,8	255,9	256,9
cublas	1×8	13,9	27,7	36,9	91,8	146,3
	2×4	13,9	27,7	36,9	91,8	146,2
	4×2	15,6	31,2	41,6	103,1	163,5
	8×1	13,1	26,2	34,8	86,3	136,8

Lettura

Sotto $k = 8$ la griglia quasi non conta. A $k = 3$ il costo complessivo delle collettive resta sotto il mezzo millisecondo per ogni backend e per ogni griglia. I messaggi sono troppo piccoli perché la loro distribuzione faccia differenza, e infatti nella tabella 7.18 le quattro griglie stanno entro pochi punti percentuali per `cuda_warp` (fra 239,8 e 254,3 GFLOPS).

Il costo delle collettive cresce più che proporzionalmente in k . Per `cuda_warp` sulla griglia 1×8 si passa da 0,373 ms a $k = 3$ a 8,635 ms a $k = 32$: k cresce di un fattore 10,7 e il tempo di un fattore 23. Le griglie bilanciate contengono molto meglio questa crescita, restando fra 2,190 e 2,427 ms a $k = 32$, cioè quasi quattro volte meno di 1×8 .

Ogni backend ha una griglia preferita, e quasi sempre è la stessa per tutti i k . `cuda_warp` preferisce 2×4 su tutti e cinque i valori di k ; `cuda_warp_smem` e `cuBLAS` preferiscono 4×2 per $k \geq 6$; `cuda_naive` è l'unico che cambia, da 1×8 ai k stretti a 2×4 da $k = 8$ in poi.

Il guadagno ottenibile dalla sola scelta della griglia è maggiore per cuBLAS che per i kernel custom. A $k = 32$ la griglia migliore vale il 5% in più della peggiore per `cuda_warp` (298,5 contro 284,1) e il 10% per `cuda_warp_smem` (281,6 contro 256,9), ma il 20% per `cuBLAS` (163,5 contro 136,8). Per i kernel custom la griglia è una scelta di comunicazione; per `cuBLAS` è anche, e soprattutto, una scelta di geometria del blocco locale.

7.4.4 Confronto fra backend nella composizione MPI + CUDA

Le due sottosezioni precedenti hanno misurato quanto costa la composizione. Resta da capire *dove* finisce quel costo, e se colpisce i quattro backend allo stesso modo.

Il confronto si costruisce su tre livelli annidati, tutti misurati sullo stesso punto $S = 16384$ e ricavati dalla stessa campagna:

1. il **kernel** da solo, a rank singolo (`gflops_kernel`);
2. l'intera **fase locale** a rank singolo, che aggiunge al kernel il trasferimento H2D di X e il D2H di Y ;

3. l'intera fase locale con **otto rank** sulla griglia 4×2 , che aggiunge la condivisione della GPU fra otto contesti.

La differenza fra il primo e il secondo livello è il costo dei trasferimenti; quella fra il secondo e il terzo è il costo della condivisione. Entrambe si leggono direttamente nella tabella 7.19.

Tabella 7.19: Fase 4: throughput dei tre livelli di composizione a $S = 16\,384$, in GFLOPS. *Kernel* è `gflops_kernel` a $P = 1$; *locale* è `gflops_local_group`, cioè l'intera fase locale (trasferimenti inclusi), a $P = 1$ e a $P = 8$ sulla griglia 4×2 .

Backend	Livello	$k = 3$	$k = 6$	$k = 8$	$k = 20$	$k = 32$
cuda_naive	kernel, $P = 1$	149,6	290,9	336,5	340,5	343,1
	locale, $P = 1$	146,6	280,3	321,9	329,0	331,3
	locale, $P = 8$	176,4	272,9	281,1	285,9	293,1
cuda_warp	kernel, $P = 1$	298,5	315,3	341,5	348,7	349,8
	locale, $P = 1$	286,9	301,5	326,6	336,5	335,5
	locale, $P = 8$	251,9	304,6	309,8	302,1	295,5
cuda_warp_smem	kernel, $P = 1$	288,1	343,2	326,5	344,1	335,7
	locale, $P = 1$	277,3	328,5	312,9	332,3	324,4
	locale, $P = 8$	249,1	297,3	303,6	296,8	281,6
cublas	kernel, $P = 1$	15,1	30,2	40,2	100,4	160,6
	locale, $P = 1$	15,0	30,1	39,9	99,4	157,9
	locale, $P = 8$	15,6	31,2	41,6	103,1	163,5

Lettura

I trasferimenti costano circa il 4% ai kernel custom e quasi nulla a cuBLAS. A $k = 32$ il passaggio dal kernel alla fase locale completa costa il 4,1% a `cuda_warp` (da 349,8 a 335,5 GFLOPS), il 3,4% a `cuda_naive` e a `cuda_warp_smem`, ma solo l'1,7% a cuBLAS. Il costo assoluto dei trasferimenti è lo stesso per tutti: cambia il denominatore, perché il kernel di cuBLAS dura circa tre volte di più. Un kernel lento nasconde i trasferimenti, e questo non è un merito.

La condivisione della GPU costa circa il 12%, ma solo dove la GPU era già satura. A $k = 32$ il terzo livello perde l'11,9% per `cuda_warp`, l'11,5% per `cuda_naive` e il 13,2% per `cuda_warp_smem`. A $k = 3$, invece, il segno cambia per i backend che a rank singolo lasciavano la scheda sottoutilizzata: `cuda_naive` guadagna il 20% e cuBLAS il 4%.

La gerarchia fra backend sopravvive alla composizione, ma si comprime. A $k = 3$ il vantaggio di `cuda_warp` su `cuda_naive` vale $1,96\times$ a rank singolo (286,9 contro 146,6) e si riduce a $1,43\times$ con otto rank (251,9 contro 176,4). Da $k = 8$ in poi i tre kernel custom stanno entro il 10% l'uno dall'altro a otto rank. La condivisione della GPU avvantaggia chi partiva peggio, ma non ribalta l'ordine.

cuBLAS resta indietro a tutti i livelli. Con otto rank vale $1,8\times$ meno di `cuda_warp` a $k = 32$ (163,5 contro 295,5) e $16\times$ meno a $k = 3$ (15,6 contro 251,9). La composizione MPI non corregge il percorso GEMM sfavorevole per un multivettore stretto: lo eredita.

7.4.5 Sintesi della Fase 4

La Fase 4 si riassume in quattro affermazioni, tutte sostenute dalle tabelle precedenti.

La metrica va scelta prima dei numeri. Con più contesti sulla stessa GPU il tempo del singolo rank può sovrastimare il throughput di oltre undici volte. Tutta l'analisi della fase poggia sul tempo collettivo, e il CV mediano dello 0,30% mostra che quella misura è stabile.

Più rank non significano più prestazioni, ma nemmeno sempre meno. Sul problema grande la composizione costa il 12% circa ai kernel custom. Sul problema piccolo con multi-vettore stretto costa fino al 67%. Dove invece un solo rank non saturava la GPU, otto rank migliorano il risultato fino al 20%. Il fattore che decide non è P , ma il lavoro che resta a ogni rank.

La griglia è una scelta di comunicazione, tranne per cuBLAS. Le griglie bilanciate 2×4 e 4×2 evitano i due estremi monodimensionali e contengono il costo delle collettive entro 2,5 ms a $k = 32$, contro gli 8,6 ms di 1×8 . Per i kernel custom il guadagno ottenibile scegliendo bene la griglia vale il 5–10%; per cuBLAS vale il 20%, perché la libreria reagisce anche alla forma del blocco locale.

Il quadro dei backend regge. `cuda_warp` resta il riferimento a ogni livello di composizione e a ogni k . La condivisione della GPU riduce le distanze, soprattutto ai k stretti dove `cuda_naive` recupera, ma non cambia l'ordine emerso nelle fasi a rank singolo.

7.5 Il costo dei dati: trasferimenti e preprocessing

Le voci di questa sezione non richiedono esecuzioni aggiuntive: ogni campagna le scrive in coda al proprio CSV. I valori riportati sono quelli di `cuda_warp` nel confronto conclusivo della Fase 1 ($M = N = 16\,384$, FP64, $P = 1$), dove A occupa 2 GiB e il blocco di X trasferito a ogni invocazione va da 0,38 MiB a $k = 3$ a 4 MiB a $k = 32$.

Tabella 7.20: Scomposizione della fase locale su GPU secondo la eq. (5.1), in percentuale di t_{local} .

k	$t_{\text{H2D } X}$	t_{kernel}	$t_{\text{D2H } Y}$	t_{launch}	t_{local} [ms]
3	2,17%	96,05%	1,62%	0,168%	5,55
6	2,45%	95,66%	1,79%	0,101%	9,42
8	2,48%	95,68%	1,76%	0,080%	12,60
20	1,99%	96,41%	1,57%	0,035%	31,14
32	2,07%	96,33%	1,58%	0,023%	49,78

Tabella 7.21: Preprocessing e banda dei trasferimenti, `cuda_warp`, $M = N = 16\,384$, FP64. Le voci di setup sono pagate una volta sola.

Voce	Valore	Nota
t_{setup} totale	0,617 s	<code>local_gemm_create</code>
creazione del contesto CUDA	0,139 s	costo fisso, indipendente dalla taglia
<code>cudaMalloc</code> in VRAM	0,26 ms	trascurabile
H2D di A	0,475 s	il trasferimento tolto dal cammino misurato
<code>bw_h2d_A_gbs</code>	4,53 GB/s	2 GiB in un'unica copia
<code>bw_h2d_X_gbs</code>	da 3,27 GB/s a 4,23 GB/s	messaggi da da 0,38 MiB a 4 MiB
<code>bw_d2h_Y_gbs</code>	da 4,38 GB/s a 5,36 GB/s	idem, in uscita
t_{prep} generazione di A	0,68 s	esclusa da T
t_{prep} distribuzione di A	0 s	con <code>-a-mode local</code>
t_{prep} conversione di X	da 0,26 ms a 3,32 ms	solo con X column-major
t_{prep} totale	1,30 s	somma locale, poi massimo fra i rank
t_{io} scrittura del CSV	0,19 ms	unico I/O del driver

Lettura

I trasferimenti per invocazione pesano circa il 4%, e il peso non dipende da k . H2D di X e D2H di Y sommano fra il 3,6% e il 4,2% di t_{local} su tutto l'intervallo. Il risultato non

era scontato: i due trasferimenti scalano con $n_{\text{loc}}k$ e $m_{\text{loc}}k$, cioè linearmente in k , ma anche il kernel scala linearmente in k nel regime compute-bound, e il rapporto resta costante. È la conferma che a 16384×16384 il problema è abbastanza grande da rendere il PCIe un contorno.

L’overhead di lancio si ammortizza, come deve. t_{launch} vale da $9\mu\text{s}$ a $12\mu\text{s}$ in assoluto, cioè un valore che non dipende da k . In percentuale scende quindi dallo 0,168% a $k = 3$ allo 0,023% a $k = 32$. Le due voci si muovono in direzioni opposte proprio perché una scala con il lavoro e l’altra no, ed è ciò che la scomposizione permette di verificare invece di asserire.

La banda misurata è un quarto del bus, e questo è un risultato azionabile. La copia di A raggiunge $4,53\text{ GB/s}$ su un PCIe 3.0 $\times 16$ il cui picco teorico è circa $15,75\text{ GB/s}$. Il valore è coerente con l’uso di memoria host *pageable*: il driver deve passare per un buffer di staging pinnato interno, dimezzando o peggio la banda utile. Allocare i buffer host con `cudaHostAlloc` è quindi l’intervento che ridurrebbe i $0,475\text{ s}$ di setup, e non richiede di toccare né i kernel né `mpi_matmul`.

Il ciclo di vita a tre fasi si ripaga in una decina di invocazioni. Il trasferimento iniziale di A costa $0,475\text{ s}$, contro i 48 ms di un kernel a $k = 32$. Copiare A a ogni invocazione, come imporrebbe un’interfaccia senza `create` e `destroy`, moltiplicherebbe per undici il tempo di ogni prodotto. Il ciclo di vita in tre fasi è quindi la condizione perché la misura riguardi la GPU e non il bus.

In funzione dei lavori futuri. I due trasferimenti per invocazione sommano, a $k = 32$, $1,82\text{ ms}$ su una fase locale di $49,78\text{ ms}$. È il tetto esatto del guadagno ottenibile da un MPI CUDA-aware, che eliminerebbe H2D di X e D2H di Y passando puntatori device direttamente alle collettive: meno del 4% sulla fase locale a rank singolo. Il beneficio vero di quell’intervento va quindi cercato nella Fase 4, dove i trasferimenti di più rank competono sullo stesso bus. Lo scarto fra throughput kernel-only e throughput collettivo mostra che la contesa fra contesti pesa più della singola coppia di copie.

7.6 Sintesi

Tabella 7.22: Punti rappresentativi a $M = N = 16384$ e $k = 32$. Per MPI + CUDA si usa il throughput della fase locale collettiva.

Regime	Backend migliore	GFLOPS	% tetto	Riferimento
GPU FP64, kernel-only	<code>cuda_warp</code>	354,9	101,4	tabella 7.7
GPU FP32, kernel-only	<code>cublas</code>	4791,9	66,9	??
MPI + CUDA FP64, $P = 1$	<code>cuda_warp</code>	335,5	95,9	tabella 7.19
MPI + CUDA FP64, $P = 8$	<code>cuda_naive</code> , 2×4	302,6	86,5	sezione 7.4.4
CPU FP64, $P = 1$	<code>scheme_a_jblock</code>	16,11	-	??
CPU FP64, $P = 8$	<code>scheme_a_jblock</code> , 4×2	104,15	-	??

Le quattro conclusioni che il capitolo sostiene, ciascuna appoggiata su più di una campagna:

- 1. Il modello roofline ha previsto correttamente il regime in entrambe le precisioni.** In FP64 il plateau al tetto per $k \geq 6$ nel confronto conclusivo della Fase 1, la sua indipendenza da taglia e forma alle scale sufficienti (Fase 2) e la crescita di $S_{32/64}$ con k (Fase 3) sono tre verifiche indipendenti della stessa previsione. Il modello sovrastima però il tetto FP32 ai k grandi, e la Fase 3 individua la ragione nel traffico su X che il roofline a un livello non contempla.
- 2. Sulla GPU condivisa il tempo collettivo è la metrica indispensabile.** I throughput rank-local possono superare di oltre un ordine di grandezza la capacità sostenuta dal device.

Con il group timing, il passaggio da uno a otto rank costa circa il 12% a $S = 16384$ e $k = 32$, mentre penalizza molto di più i problemi piccoli.

3. **Sul backend CPU la griglia può cambiare il regime di cache.** Con `scheme_a`, suddividere X lungo P_c produce speedup superlineari perché riduce il working set per rank. Il j-blocking ottiene invece circa 100 GFLOPS su tutte le griglie a $P = 8$, mostrando una robustezza che lo schema base non possiede.
4. **Il tuning vale quanto la scelta del kernel.** La differenza fra la peggiore e la migliore configurazione di `cuda_warp_smem` (38,8% di perdita contro 1,5%) è dello stesso ordine della differenza fra i kernel custom, e molto maggiore della differenza fra `cuda_warp` e `cuda_warp_smem`. È la giustificazione a posteriori dell'ordine delle fasi.